Structural Augmented Reasoning for Language Models

by

Yubo Wang

A Thesis Submitted to
The Hong Kong University of Science and Technology
in Partial Fulfilment of the Requirements for
the Degree of Doctor of Philosophy
in Computer Science and Engineering

December 2026, Hong Kong

Structural Augmented Reasoning for Language Models

by Yubo Wang

Department of Computer Science and Engineering

The Hong Kong University of Science and Technology

Abstract

Language models depend on the evidence supplied to their prediction and generation processes. Tables can omit information needed for semantic annotation, classification labels can evolve under limited supervision, and answers can require relationships distributed across textual and visual sources. This thesis investigates how explicit knowledge structures can organize and select evidence for these settings.

The thesis develops four methods for structural augmentation. KGLink integrates knowledge graphs with a pre-trained language model to enrich column representations and bridge differences between graph types and annotation labels. GORAG supports dynamic few-shot classification through connected label retrieval and an online graph index that evolves with the task. AGRAG selects connected textual evidence by combining query-dependent node influence, semantic edge costs, and hybrid passage retrieval. MGRAG constructs query-specific multimodal graphs, matches query relations to semantic paths, and uses graph structure to guide the reuse and recomputation of cached model states.

Experiments across table annotation, dynamic classification, textual generation, and multimodal question answering show improved effectiveness over established baselines and efficiency gains in graph indexing and context processing. Ablation studies verify the contributions of the structural components. A related study of hyper-relational knowledge graphs examines information preservation. These studies demonstrate how explicit relationships improve evidence representation, retrieval, and computation for language models.

Authorization

I hereby declare that I am the sole author of the thesis.

I authorize the Hong Kong University of Science and Technology to lend this thesis to other institutions or individuals for the purpose of scholarly research.

I further authorize the Hong Kong University of Science and Technology to reproduce the thesis by photocopying or by other means, in total or in part, at the request of other institutions or individuals for the purpose of scholarly research.

_____

Yubo Wang

_____

Structural Augmented Reasoning for Language Models

by

Yubo Wang

This is to certify that I have examined the above PhD thesis
and have found that it is complete and satisfactory in all respects,
and that any and all revisions required by
the thesis examination committee have been made.

Prof. Lei Chen, Chair Professor, Thesis Supervisor

Head of Department

Department of Computer Science and Engineering

ACKNOWLEDGEMENTS

I thank my supervisor, Prof. Lei Chen, for his guidance during my doctoral research. I am grateful to Dr. Haoyang Li, Dr. Hao Xin, and Dr. Shimin Di for the discussions and collaboration that helped develop the work presented in this thesis.

TABLE OF CONTENTS

Title Page i

Abstract ii

Authorization iii

Signature Page iv

Acknowledgements v

Table of Contents vi

List of Figures x

List of Tables xii

Chapter 1 Introduction 1
- 1.1 Motivation 1
- 1.2 Research Scope and Structural Augmentation 2
  - 1.2.1 Evidence, structure, and model roles 2
  - 1.2.2 Reasoning through task-specific evidence 3
  - 1.2.3 The evidence interface as a design choice 3
- 1.3 Research Questions 3
  - 1.3.1 RQ1: External knowledge for table understanding 3
  - 1.3.2 RQ2: Evidence maintenance under evolving labels 4
  - 1.3.3 RQ3: Connected evidence for textual generation 4
  - 1.3.4 RQ4: Semantic matching and computation over multimodal evidence 5
- 1.4 Contributions and Evidence 5
- 1.5 Publications and Thesis Organization 7

Chapter 2 Background and Related Work 8
- 2.1 Knowledge Graphs and Language Models 8
- 2.2 Knowledge-enhanced Table Understanding 9
- 2.3 Retrieval-augmented Generation 10
- 2.4 Graph-based Evidence Retrieval 11
- 2.5 Multimodal Retrieval and Context Serving 12
- 2.6 Structural Information in Hyper-relational Knowledge Graphs 13
  - 2.6.1 Facts, Qualifiers, and Decomposition 14
  - 2.6.2 Qualifier Capture and Graph Reception 15
  - 2.6.3 Representative Published Evidence 15
- 2.7 Research Questions Emerging from the Background 16

Chapter 3 Knowledge-enriched Table Understanding with KGLink 17
3.1 Motivation and Task 17
3.2 Problem Formulation and Framework 18
3.3 KG-derived Candidate Types 19
3.3.1 Cell Mention Linking 19
3.3.2 Cross-column Neighborhood Filtering 20
3.3.3 Row Linking Quality and Input Selection 21
3.3.4 Candidate Types and Feature Sequences 22
3.4 Language Model Integration 23
3.4.1 Table Serialization and Column Representations 23
3.4.2 Contextual Type Representation Learning 23
3.4.3 Classification and Adaptive Task Weighting 25
3.5 Computational Structure 25
3.6 Experimental Evaluation 26
3.6.1 Datasets and Protocol 26
3.6.2 Main Annotation Results 27
3.6.3 Representation Learning and KG Evidence 28
3.6.4 Annotation without Retrieved Knowledge 28
3.6.5 Type Granularity in the Reported Cases 29
3.6.6 Row Selection and Input Budgets 29
3.6.7 Runtime in the Evaluated Configuration 30
3.7 Discussion and Chapter Summary 31

Chapter 4 Adaptive Graph Retrieval for Dynamic Classification with GORAG 32
4.1 Dynamic Few-shot Classification 32
4.1.1 Rounds and information availability 33
4.2 Weighted Graph Indexing 34
4.2.1 Keyword extraction and correlation scores 34
4.2.2 Keyword–label edge costs 35
4.2.3 Merging label rounds 35
4.3 Adaptive Candidate Label Retrieval 36
4.3.1 Tree and auxiliary-graph operations 36
4.4 Classification and Online Indexing 37
4.4.1 Computational organization 38
4.5 Experimental Design 38
4.5.1 Compared methods and controls 39
4.6 Classification Results 39
4.7 Ablations and Resource Use 42
4.7.1 Prompt tokens and graph growth 42
4.8 Zero-shot and Round Generalization 43
4.8.1 A retrieved classification case 45
4.9 Discussion and Chapter Summary 45

Chapter 5 Connected Evidence Selection for Language Models with AGRAG 47
5.1 Research Problem and Evidence Requirements 47
5.2 Framework and Graph Construction 48
5.2.1 The three-stage pipeline 48
5.2.2 Chunking and statistical entity extraction 49

5.2.3 Relations, passage nodes, and synonyms 50
5.3 Query Mapping and Influence-aware Retrieval 50
5.3.1 Mapping a query to triple facts 50
5.3.2 Node influence and edge costs 51
5.4 MCMI Formulation and Evidence Expansion 52
5.4.1 The original connected-evidence example 53
5.5 Hybrid Retrieval and Generation 54
5.6 Computational Properties 54
5.7 Experimental Evaluation 55
5.7.1 Tasks, corpora, and model settings 55
5.7.2 Metric interpretation 56
5.7.3 Answer quality across tasks 59
5.7.4 Indexing and querying costs 60
5.7.5 Component ablations 63
5.7.6 Additional resource setting and parameter study 63
5.8 Discussion and Chapter Summary 66

Chapter 6 Semantic Graph Matching for Multimodal Reasoning with MGRAG 67
6.1 Multimodal Evidence and Query Requirements 67
6.2 Framework and Evidence Representations 68
6.3 Coarse-grained Ranking 69
6.4 Query-aware Graph Construction 70
6.4.1 Representing the query 70
6.4.2 Extracting facts from ranked chunks 71
6.4.3 Matching as a stopping decision 72
6.5 Path-based Semantic Subgraph Matching 72
6.5.1 From edges to paths 72
6.5.2 Anchor selection and candidate mappings 73
6.5.3 Greedy semantic path growth 74
6.5.4 Published algorithmic properties 75
6.5.5 Serializing selected evidence 75
6.6 Graph-aware KV Recomputation 76
6.6.1 Reuse and selective updates 76
6.6.2 Topological features of facts 77
6.6.3 Combined recomputation score 77
6.7 Computational Cost 78
6.8 Experimental Protocol 78
6.9 Answer Quality and Structural Retrieval 79
6.9.1 Complete-system comparisons 79
6.9.2 Construction and matching substitutions 81
6.9.3 Queries requiring multiple hops 84
6.10 Cache Quality and End-to-end Costs 85
6.11 Ablations and Parameter Sensitivity 87
6.11.1 Component ablations 87
6.11.2 Adding components to other pipelines 87
6.11.3 Matching threshold and path length 88
6.12 Discussion and Chapter Summary 89

Chapter 7 Discussion 90
7.1 Where Structure Enters the Pipeline 90
7.2 Static, Online, and Query-dependent Structures 90
7.3 Evidence Quality and Computational Cost 91
7.4 Information Preservation and Semantic Granularity 93
7.5 Error Propagation Across Structural Decisions 93
7.6 Limitations and Open Questions 94

Chapter 8 Conclusion and Future Directions 95
8.1 Conclusion 95
8.2 Future Directions 96
8.2.1 Feedback for structure maintenance 96
8.2.2 Multimodal structure and provenance 96
8.2.3 Joint retrieval and context computation 96

References 97

LIST OF FIGURES

Figure 2.1 Original comparison of hyper-relational decomposition methods. Prune retains the main triple; Direct adds subject–qualifier links; Hyper adds combined relations; database reification introduces auxiliary objects. Reproduced from Fig. 1 of the CIKM study [5]. 14

Figure 3.1 Two annotation difficulties in the original KGLink examples. A graph description such as Athlete can differ from the target label Name, while dates beside a person's name provide little occupation information. Original subfigures reproduced from KGLink [1], Fig. 2. 18

Figure 3.2 KGLink's graph-processing and language-model components. Candidate-type text, column feature vectors, and a contextual type-representation objective connect external knowledge to annotation. Original image reproduced from KGLink [1], Fig. 3. 19

Figure 3.3 The original KG-processing pipeline. Cell linking is followed by entity filtering, row selection, and candidate-type generation. Reproduced from KGLink [1], Fig. 4. 20

Figure 3.4 Cross-column evidence in the original album-and-musician example. Graph links connect candidate interpretations of cell mentions in the same row. Original image reproduced from KGLink [1], Fig. 5. 21

Figure 3.5 Contextual type-representation training with masked and ground-truth table inputs. The true-type table is constructed during training. Original image reproduced from KGLink [1], Fig. 6. 24

Figure 3.6 Reported effect of the retained row budget k. The original two subfigures compare annotation quality and time cost on SemTab and VizNet. Reproduced from KGLink [1], Fig. 10. 30

Figure 3.7 Training and inference time reported on VizNet. Original image reproduced from KGLink [1], Fig. 7. 31

Figure 4.1 Two rounds of the dynamic classification task. New abuse categories extend the available labels. Reproduced from Figure 1 of GORAG [2]. 33

Figure 4.2 GORAG's offline indexing, query-dependent retrieval, classification, and online update. New query keywords are associated with the predicted class after classification. Reproduced from Figure 2 of GORAG [2]. 34

Figure 4.3 Total time in minutes for the one-shot four-round experiments with Qwen2.5-7B. The original R1–R4 legend in panel (b) applies to both panels. Reproduced from Figure 3 of GORAG [2]. 43

Figure 4.4 WOS classification across the reported round splits. Points 1–4 use the four-round split, 5–6 the six-round split, and 7–8 the eight-round split. The legend in panel (b) applies to both panels. Reproduced from Figure 4 of GORAG [2]. 44

Figure 4.5 A contextual-abuse example and its selected graph connections. Reproduced from Figure 5 of GORAG [2]. 45

Figure 5.1 The AGRAG pipeline. The original figure separates corpus preparation, query-conditioned graph retrieval, and hybrid passage retrieval. Reproduced from Figure 1 of the source paper [3]. 48
Figure 5.2 The original AGRAG evidence-expansion example. Red triples are query-mapped facts; orange triples show additional neighboring evidence in the illustrated MCMI structure. Reproduced from Figure 2 of the source paper [3]. 53
Figure 5.3 AGRAG parameter sensitivity on Novel (top) and Medical (bottom). The original panels vary the extraction threshold and maximum n-gram length and display task accuracy. Reproduced from Figure 3 of the source paper [3]. 65

Figure 6.1 Path-level correspondence in the MGRAG motivating example. A `grandfather_of` relation corresponds to two `father_of` edges. Original Figure 1, p. 2481 [4]. 68
Figure 6.2 The MGRAG pipeline. Ranked chunks support incremental query-aware graph construction and path-based matching. The matched graph also supplies features for selective KV recomputation. Original Figure 2, p. 2484 [4]. 69
Figure 6.3 Query and evidence graphs in path-based semantic matching. Placeholder entities express unknown query information; pseudo-relations connect passage nodes to extracted entities. Original Figure 3, p. 2485 [4]. 71
Figure 6.4 Graph features supplement attention in token recomputation scoring. The diagram retains the original graphical content. Original Figure 4, p. 2487 [4]. 76
Figure 6.5 MGRAG matching-parameter sensitivity. Original subfigures of Figure 5, p. 2491 [4]. 88

LIST OF TABLES

Table 1.1 Research questions, structural operations, and evidence locations. Each row corresponds to a separate study. 6

Table 1.2 Publications incorporated into the thesis. The hyper-relational study provides a separate background section; the four other studies form the core research chapters. 7

Table 2.1 Representative retrieval organizations and the evidence structures they expose. Each row refers to the cited method's design. 12

Table 2.2 Projection of the CIKM study's Table 6 onto Cleaned JF17k and WD50k. All model rows and the original metric precision are retained; the original bold and underline markings indicate best and second-best values. 16

Table 3.1 Column-type annotation results reported for KGLink. Scores and missing entries reproduce the original main-result table. 27

Table 3.2 Reported KGLink variant scores. The table reproduces the numeric columns and variant names of the original ablation table. 28

Table 3.3 Reported accuracy on the VizNet test subset whose tables have no extracted KG information. 29

Table 3.4 Reported comparison between linkage-ranked row selection and selection in original table order. 29

Table 4.1 Dataset statistics for the four-round experiments, transposed from the original table. IFS-Rel has 40 test samples per label. Round label counts refer to newly introduced labels. 39

Table 4.2 WOS classification accuracy with one support example per label. Qwen2.5-7B is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table. 40

Table 4.3 WOS classification accuracy with 5 support examples per label. Qwen2.5-7B is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table. 40

Table 4.4 WOS classification accuracy with 10 support examples per label. Qwen2.5-7B is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table. 40

Table 4.5 IFS-Rel classification accuracy with one support example per label. GPT-4o is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table. 40

Table 4.6 IFS-Rel classification accuracy with 5 support examples per label. GPT-4o is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table. 41

Table 4.7 IFS-Rel classification accuracy with 10 support examples per label. GPT-4o is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table. 41

Table 4.8 Ablation experiments on WOS for GORAG's variants. 42
Table 4.9 Token cost on IFS-Rel dataset with 1-shot setting. 43
Table 4.10 0-shot experiments for GORAG and RAG baselines on IFS-Rel dataset under 0-shot setting. 44

Table 5.1 Creative generation with Qwen2.5-14B-Instruct. Results from the AGRAG main experiment [3]. 57
Table 5.2 Contextual summarization with Qwen2.5-14B-Instruct. Results from the AGRAG main experiment [3]. 57
Table 5.3 Complex reasoning with Qwen2.5-14B-Instruct. Results from the AGRAG main experiment [3]. 58
Table 5.4 Fact retrieval with Qwen2.5-14B-Instruct. Results from the AGRAG main experiment [3]. 58
Table 5.5 Text classification with Qwen2.5-14B-Instruct. Results from the AGRAG main experiment [3]. 59
Table 5.6 Indexing / querying time in minutes. Qwen2.5-14B-Instruct; original paired measurements are preserved. 60
Table 5.7 Input / output tokens per query. Same source experiment as the preceding time table. 61
Table 5.8 Creative generation ablation with Qwen2.5-14B-Instruct. Values retain the original source ablation precision. 61
Table 5.9 Contextual summarization ablation with Qwen2.5-14B-Instruct. Values retain the original source ablation precision. 61
Table 5.10 Complex reasoning ablation with Qwen2.5-14B-Instruct. Values retain the original source ablation precision. 62
Table 5.11 Fact retrieval ablation with Qwen2.5-14B-Instruct. Values retain the original source ablation precision. 62
Table 5.12 Ablation time and token costs. Time is in minutes. Paired token columns retain the source I/G headings; its caption describes input/output token counts. 62
Table 5.13 Creative generation in the low-resource setting. Qwen3-4B-Instruct-2507 on two 2080 Ti GPUs; values and emphasis preserved. 64
Table 5.14 Contextual summarization in the low-resource setting. Qwen3-4B-Instruct-2507 on two 2080 Ti GPUs; values and emphasis preserved. 64
Table 5.15 Complex reasoning in the low-resource setting. Qwen3-4B-Instruct-2507 on two 2080 Ti GPUs; values and emphasis preserved. 64
Table 5.16 Fact retrieval in the low-resource setting. Qwen3-4B-Instruct-2507 on two 2080 Ti GPUs; values and emphasis preserved. 64
Table 5.17 Novel low-resource efficiency. Qwen3-4B-Instruct-2507 on two 2080 Ti GPUs. 64
Table 5.18 Medical low-resource efficiency. Qwen3-4B-Instruct-2507 on two 2080 Ti GPUs. 65

Table 6.1 Evaluation datasets, reproduced from published Table 3, p. 2488 [4]. T and I denote textual and image query modalities. The columns comprise MMQA and six MRAMG subsets. 78

Table 6.2 Answer quality with Qwen2.5-VL-7B-Instruct, reproduced from published Table 4, p. 2489 [4]. Vanilla., R1., Graph., Light., and Hippo. denote VanillaRAG, R1-Searcher, GraphRAG, LightRAG, and HippoRAG2. A dash means that the experiment exceeded three days; it is not a zero score. Bold and underlining retain the publication's highlighting. 80
Table 6.3 Graph-construction and graph-matching substitutions, transposed from published Table 7, p. 2490 [4]. All twelve method columns and all four metric values per dataset are retained. O.IE and STAT replace graph construction; the other alternatives replace matching. The shared backbone is Qwen2.5-VL-7B-Instruct. 81
Table 6.4 Results by query hop count, reorganized from published Table 8, p. 2490 [4]. Time is in minutes, as reported. Each original row is shown as two method rows, with the query count repeated. 84
Table 6.5 Cache-system comparison on MMQA, reproduced from published Table 6, p. 2489 [4]. V. and H. identify VanillaRAG and HippoRAG2 retrieval pipelines. TTFT is in seconds and total time is in hours. 85
Table 6.6 Module costs reproduced from published Table 5, p. 2489 [4]. Time is in minutes. KGC, QA, and E2E cells retain the published time/token pairs for graph construction, answering, and end-to-end processing. Match denotes matching time. Token counts and E2E totals are preserved as reported. 86
Table 6.7 MMQA component ablations, reproduced from published Table 9, p. 2490 [4]. Time is in hours. 87
Table 6.8 MMQA inverse ablations, reproduced from published Table 10, p. 2491 [4]. PSGM and GKV denote path-based semantic graph matching and graph-aware KV recomputation. Time is in hours. 88

Table 7.1 The roles of structure across the four studies. The entries summarize the methods in their respective research chapters. 91
Table 7.2 Main experimental conditions across the four studies. Hardware denotes the reported local setup; GORAG's IFS-Rel generation uses the GPT-4o service. Full task-specific controls appear in the corresponding research chapters. 92

CHAPTER 1

INTRODUCTION

1.1 Motivation

Language models use tables, text, and images to make predictions and generate answers. The required information is often incomplete or distributed across sources. A table may list people without identifying their occupations. A classification task may introduce labels absent from its initial training examples. A question may require facts from several passages or a relationship expressed jointly by text and images. Selecting the relevant evidence and preserving its connections are therefore part of the prediction problem.

We study how explicit knowledge structures can support evidence selection for language models. Knowledge graphs provide descriptions absent from a table. Task graphs maintain associations between keywords and classification labels. Evidence graphs connect facts across passages, while query graphs specify the relationships needed to answer a multimodal question. We develop methods that use these relationships to retrieve evidence and control how it is processed.

Our first problem is column type annotation. A column of people accompanied by birth and death dates may lack the information needed to identify their common occupation. Linking the names to a knowledge graph can supply that information, but the graph may assign several valid types to each person. These types may also differ from the annotation vocabulary. We propose KGLink to select relevant graph descriptions and learn their correspondence to the target labels [1].

Our second problem is dynamic few-shot text classification. New labels arrive over successive rounds, with only a few examples for each class. A fixed retrieval budget does not determine which labels are relevant to the current text, and uniformly indexing observed words can retain incidental associations. We propose GORAG to maintain weighted keyword–label relations and retrieve candidate labels by connecting query keywords [2].

Our third problem is evidence retrieval for textual question answering and generation. Individually relevant passages can repeat facts or leave their connections implicit. A short path between query anchors can also omit information needed for a summary. We propose AGRAG

to select a connected subgraph using query-dependent node influence and semantic edge costs. Hybrid passage retrieval supplies context beyond the selected graph [3].

Our fourth problem is multimodal question answering. A relation stated directly in a question may correspond to several relations in the evidence, and a graph constructed before seeing the query can omit relevant visual details. We propose MGRAG to build query-specific evidence graphs and match query relations to semantic paths. Graph features also guide the recomputation of cached model states when the retrieved content is used for answering [4].

These problems lead to a common research question. How can explicit relationships improve the selection and processing of evidence for language models? We investigate external knowledge selection, online index maintenance, connected textual retrieval, and semantic matching over multimodal evidence.

1.2 Research Scope and Structural Augmentation

1.2.1 Evidence, structure, and model roles

Structural augmentation uses explicit relationships among entities, types, keywords, labels, passages, or query components. We study knowledge-graph edges, keyword–label associations, passage links, and paths that express the relationships requested by a query.

Graphs record entities and their connections, while embeddings provide similarity scores or contextual features. We combine them by serializing graph descriptions for a table encoder, weighting connections for label retrieval, scoring nodes and edges for subgraph selection, and comparing path semantics for multimodal matching.

External evidence supplies information, while supervision determines how that information is used for prediction. KGLink learns column labels from annotated tables and retrieves descriptions from an external graph. GORAG initializes its index from labeled examples and updates it using subsequent predictions. AGRAG and MGRAG construct their evidence graphs from source documents.

KGLink fine-tunes a pre-trained encoder for column annotation. GORAG, AGRAG, and MGRAG use generative models for classification, extraction, filtering, or answer generation. The thesis thus covers both supervised representation learning and retrieval-augmented generation.

1.2.2 Reasoning through task-specific evidence

We evaluate the structural operations through annotation accuracy and F1, classification accuracy across label rounds, task-specific answer measures, and context-processing costs.

The four studies have separate implementations and evaluations. They address successive problems in obtaining external knowledge, maintaining an index, selecting connected text, and matching multimodal evidence.

We also examine information preservation in hyper-relational knowledge graphs. This study distinguishes qualifier capture from graph reception and develops FormerGNN to combine them. It appears in the background chapter and provides a basis for analyzing information lost during graph transformation.

1.2.3 The evidence interface as a design choice

The available information determines each retrieval decision. KGLink uses table content and linked descriptions, with a masked type position at inference. GORAG uses the support examples and index updates available before the current prediction. AGRAG selects from a fixed corpus graph for each query. MGRAG progressively extracts query-related facts from ranked multimodal chunks.

1.3 Research Questions

1.3.1 RQ1: External knowledge for table understanding

The first research question asks how graph-derived information and a pre-trained language model can be combined when a table lacks informative context and the graph's type vocabulary differs from the annotation task. The input consists of a table, an external knowledge graph, and labeled columns from the target task. The output is a semantic type for each target column.

Two difficulties motivate the question. Knowledge-graph linking can return multiple candidate descriptions at different granularities. The target task may require a different abstraction from a graph's available types. Meanwhile, a table encoder can only directly read the content in its input sequence. Surrounding dates or numeric values may provide little evidence about an entity's occupation or role. The method must expose external information while learning how it relates to the task's labels.

KGLink answers the question through cell linking, cross-column neighborhood filtering, row

selection, candidate-type construction, and graph-derived feature sequences. These components supply evidence to the table encoder. Contextual masked-type representation training relates that evidence to the target annotation vocabulary. Chapter 3 explains the method and reports the SemTab and VizNet experiments, ablations, no-linkage cases, and efficiency evidence.

1.3.2 RQ2: Evidence maintenance under evolving labels

The second question asks how a retrieval index can maintain task-specific relationships and select relevant labels as a few-shot classification problem evolves. The input includes the examples and labels available at the current round, an existing graph index, and an unseen text. The output is a label prediction and the prescribed index update for subsequent processing.

As labels arrive, the index must retain earlier classes and incorporate new examples. Query predictions can also supply associations for later retrieval. The order of prediction and insertion determines which information is available to each query.

GORAG answers this question through weighted keyword–label indexing, terminal-based connected retrieval, and online updates after classification. Its candidate set is selected for the current input rather than specified only by a fixed retrieval count. Chapter 4 describes the round and shot settings, the retrieval construction, and the CAD, WOS, and IFS-Rel results. The chapter also examines the cost of indexing and retrieval and the effect of the proposed components.

1.3.3 RQ3: Connected evidence for textual generation

The third question asks how to select evidence that reflects both its relevance to a query and the connections among its facts. The input is a text corpus and a query whose answer can involve several information items. The output includes an evidence subgraph, passage context, and the generated answer.

A graph-ranking score identifies relevant nodes but does not by itself specify an evidence structure. Conversely, a small structure connecting query anchors may omit additional facts needed for a summary. The system must also account for information that remains in the original text after graph extraction. These requirements motivate a combination of graph selection and passage retrieval.

AGRAG answers the question by constructing a graph with statistically selected entities, extracted relations, passage nodes, and synonym links. Query-mapped facts condition the influence scores; semantic similarity supplies edge costs. The method constructs an initial terminal-connecting structure and expands it according to the reported cost-score rule. A hybrid dense

and BM25 channel supplies additional passages. Chapter 5 evaluates these choices across textual QA tasks and classification.

1.3.4 RQ4: Semantic matching and computation over multimodal evidence

The fourth question asks how to construct and match evidence for a multimodal query when the query and evidence graphs express related information through different topologies, and how that structure can guide context recomputation. The input includes a multimodal corpus, a query, and the model states available for reuse. The output is a matched evidence subgraph and an answer generated under the selected recomputation policy.

A query can contain unknown entities, represented by semantically meaningful placeholders. Evidence may express the requested relation through intermediate entities. The relevant details can also depend on the question posed about an image. These conditions make both the construction of the graph and the matching operation query-dependent. Once selected evidence enters the generator, tokens associated with connected events supply information about which computations to refresh.

MGRAG answers the question through caption-based coarse ranking, incremental query-aware graph construction, and path-based semantic subgraph matching. It then combines query attention, graph-sequence attention, and graph topological features to score tokens for selective KV recomputation. Chapter 6 reports QA results on MMQA and six MRAMG subsets, comparisons of matching methods, component ablations, and cache quality–cost measurements.

1.4 Contributions and Evidence

We make four contributions corresponding to the research questions. Table 1.1 links each contribution to its method and evaluation.

1. **Knowledge-enriched column representations.** KGLink selects graph-derived evidence using table relationships and integrates that evidence with a pre-trained table encoder. Its contextual representation task connects the graph information to the annotation labels.
2. **Online structured label retrieval.** GORAG maintains weighted associations between keywords and labels and uses a connected retrieval procedure to generate input-specific candidate labels. Its online indexing incorporates information according to the dynamic classification protocol.

3. **Influence-aware connected evidence selection.** AGRAG combines statistical entity selection, query-conditioned influence, semantic edge costs, an expanded evidence subgraph, and hybrid passage retrieval.
4. **Query-aware multimodal matching and recomputation.** MGRAG incrementally constructs evidence graphs for individual queries, matches query relations to semantic paths, and uses graph information in selective KV recomputation.

Table 1.1: Research questions, structural operations, and evidence locations. Each row corresponds to a separate study.

RQ	Structural operation	Method location	Published evaluation
RQ1	KG-derived candidate types and features	Chapter 3, graph processing and model integration	SemTab and VizNet annotation, ablations, efficiency
RQ2	Weighted online keyword–label retrieval	Chapter 4, indexing and connected retrieval	CAD, WOS, IFS-Rel; rounds, shots, and costs
RQ3	Influence and cost-guided evidence subgraph	Sections 5.2–5.5	Section 5.7; QA and classification
RQ4	Query-aware construction, semantic paths, and KV scores	Chapter 6, matching and recomputation	MMQA and six MRAMG subsets, matching and cache studies

The published measurements provide quantitative answers to these research questions. On VizNet, KGLink obtains a weighted F1 of 96.07 compared with 95.06 for Doduo, using the percentage scale of the annotation experiment. In GORAG's fourth WOS round, accuracy with ten support examples per label is 0.4929 using Qwen2.5-7B; the one-shot result in the same round is 0.4420. AGRAG obtains Medical contextual-summarization coverage of 0.690 and accuracy of 0.604 with Qwen2.5-14B-Instruct. Its hybrid-retrieval ablation reduces coverage in that task to 0.420, linking the answer's information coverage to the passage channel. MGRAG reaches MMQA accuracy of 0.7442 with Qwen2.5-VL-7B-Instruct. The associated serving comparison reports 4.3 hours for MGRAG and 8.8 hours for vLLM with HippoRAG2.

The accompanying hyper-relational study analyzes structural information preservation and introduces FormerGNN to combine qualifier information with graph reception.

The author led the research presented in this thesis under the supervision of Prof. Lei Chen. The work benefited from discussions and collaboration with Dr. Haoyang Li, Dr. Hao Xin, and Dr. Shimin Di. Each research chapter identifies its coauthored publication source.

Table 1.2: Publications incorporated into the thesis. The hyper-relational study provides a separate background section; the four other studies form the core research chapters.

Publication	Venue	Thesis location
KGLink [1]	ICDE 2024	Chapter 3
Understanding the Embedding Models on Hyper-relational Knowledge Graph [5]	CIKM 2025	Chapter 2, structural information section
GORAG [2]	WWW 2026	Chapter 4
AGRAG [3]	ICDE 2026	Chapter 5
MGRAG [4]	PVLDB 2026	Chapter 6

1.5 Publications and Thesis Organization

The thesis draws on five publications. KGLink was published at ICDE 2024. *Understanding the Embedding Models on Hyper-relational Knowledge Graph* was published at CIKM 2025. GORAG was published at WWW 2026, AGRAG at ICDE 2026, and MGRAG in Volume 19 of the Proceedings of the VLDB Endowment. Table 1.2 records their roles in the thesis. Each research chapter begins with its own publication statement.

Chapter 2 establishes the shared concepts and compares related work according to evidence representation, retrieval, updates, matching, and context serving. It includes the hyper-relational study as an information-preservation perspective. Chapters 3–6 then develop the four research questions in sequence. Each chapter gives its problem, method, evaluation protocol, published evidence, and discussion.

Chapter 7 compares the points at which structure enters these pipelines. It examines static and query-dependent evidence, the relationship between quality and computational cost, and the conditions affecting interpretation. Chapter 8 returns to the research questions and identifies directions for future work.

CHAPTER 2

BACKGROUND AND RELATED WORK

This chapter reviews the foundations of knowledge graphs, language models, table annotation, graph retrieval, and multimodal context processing. We compare the information represented by each method and the operations used to retrieve it. Section 2.6 presents our study of information preservation in hyper-relational knowledge graphs.

2.1 Knowledge Graphs and Language Models

A knowledge graph represents entities and their relationships explicitly. In a conventional relational representation, a fact is a triple (s, r, o), where s is a subject entity, r a relation, and o an object entity. A set of these facts forms a directed graph whose edge labels identify relations. Wikidata provides a collaborative knowledge resource in which structured statements and their qualifiers can describe entities beyond the content of a single document [6]. A model accessing this graph can retrieve relations associated with a mentioned entity, including information absent from its immediate input.

The graph representation separates an entity's identity from the strings used to mention it. A table cell or query contains a mention, while an entity node refers to an indexed object. Entity linking establishes a correspondence between the two. A short name may match several entities, so a retrieval score and a contextual disambiguation rule can influence which facts become available. When a graph is extracted from text, repeated strings, aliases, and synonyms affect whether different passages share a node. These indexing choices determine possible paths before a query-specific selection algorithm begins.

Graph neighborhoods, paths, and subgraphs support different retrieval operations. Neighborhood aggregation enriches entity representations. Paths connect entities mentioned in a question. Subgraphs collect several relevant facts and their relationships. The edge type determines how a connection should be interpreted, whether it represents a fact, lexical similarity, or a keyword–label association.

Language models provide a complementary representation. The Transformer uses attention to compute token representations from their context [7]. BERT pre-trains bidirectional con-

textual representations and can be fine-tuned with a task-specific output layer [8]. In column annotation, contextual token vectors can represent a column and feed a classifier. Generative language models instead produce an answer sequence conditioned on the query and supplied context. This thesis uses the term language model for both roles and specifies the relevant prediction interface in each research chapter.

Fine-tuning modifies model parameters using labeled examples. Retrieval supplies evidence from an external index at prediction time. KGLink combines the two by fine-tuning an annotation model with graph-derived inputs. The later RAG systems retrieve evidence for a generator.

2.2 Knowledge-enhanced Table Understanding

Column-type annotation assigns semantic labels to table columns. Similar values can require different labels under different annotation vocabularies. A column of people may be labeled Name or assigned an occupation category. Surrounding columns and external facts can help determine the intended type.

Sherlock trains a neural type detector from statistical properties, character distributions, word embeddings, and paragraph vectors [9]. Sato adds topic features and structured prediction to model which column types occur together [10]. The two approaches differ in whether table context contributes to the prediction.

Doduo serializes multiple columns for a pre-trained language model and learns column-type and column-property annotations from their contextual representations [11]. Self-attention lets a column representation depend on values in other columns, subject to the input-length limit.

Knowledge-based methods obtain evidence through a different route. HNN combines learned table features with knowledge-base lookup and inter-column semantics [12]. A knowledge resource can identify a relation between mentioned objects that the table's strings do not explain. RECA uses related tables as an additional contextual source [13]. These approaches extend the information available within the target table through external semantic facts or evidence from other tables. Their retrieval units and preparation costs differ accordingly.

KGLink retrieves candidate entities for cell mentions and filters them through connections between columns [1]. Candidate-type text and a separate KG feature sequence enrich the column representation. A contextual type-representation objective learns the correspondence between graph descriptions and annotation labels. Chapter 3 presents the method and experiments.

These methods obtain context from different sources. Statistical features summarize column contents, contextual encoders model values jointly, RECA retrieves related tables, and

knowledge-based methods retrieve external facts. Their coverage determines which ambiguities they can resolve.

2.3 Retrieval-augmented Generation

Retrieval-augmented generation conditions a generative model on information selected from an external collection [14]. A retrieval stage maps the query to indexed evidence, and a generation stage uses the evidence to produce an answer. The distinction makes the external collection an explicit part of the prediction system. Changing its contents or its selection rule changes the context received by the generator even when the generator's parameters stay fixed.

A passage-based RAG pipeline divides a corpus into chunks, indexes them, and concatenates the retrieved chunks with the query. Small chunks can separate a fact from its explanation. Large chunks preserve more context but consume more input tokens. Overlap retains information across chunk boundaries at the cost of repeating some text.

Sparse retrieval scores lexical evidence. BM25 combines within-document term frequency with corpus frequency and document-length normalization [15]. It distinguishes a query term that occurs widely from one that is concentrated in fewer indexed items. Dense retrieval instead represents queries and passages as vectors and scores their correspondence in a learned space. Dense Passage Retrieval uses a dual-encoder framework with separate question and passage encoders [16]. The two representations provide different signals, so hybrid retrieval can select context using both lexical overlap and embedding similarity.

For a query q and indexed evidence item e, write the retrieval score abstractly as $s(q, e)$. A conventional fixed-budget result contains the highest-scoring k items. Evidence is scored at the chosen item granularity, and the retrieval cutoff controls how many items are returned. A graph-based system can alter either assumption. It can score relationships as well as items, or determine the context by a structural condition such as connecting query anchors. The resulting context is then a selected relational object with a serialization for the language model.

Retrieval evaluation measures whether the selected context contains supporting evidence. Generation evaluation measures the resulting answer through correctness, reference overlap, or model-based assessment. An answer improvement can arise from selecting better evidence, organizing the context, or using it more effectively.

An evolving collection adds a maintenance problem. Newly available documents, examples, or labels must enter the index before retrieval can use them. Updating stored evidence can involve new embeddings, new graph nodes, changed edge weights, or derived summaries. The

information supplied by an update also matters. A labeled support example carries a provided class, while a query processed online may carry only a model prediction. GORAG makes this distinction part of its dynamic classification protocol [2]. Its index expands as observations arrive, and retrieval uses the graph state available at the current prediction.

Retrieving more passages can increase both answer coverage and model-processing cost. The retrieval budget therefore affects the information available to the generator even when the corpus and model remain fixed.

The indexed object can differ from the retrieved context. A document can contribute several graph facts, a traversal can return candidate labels, and a prompt can combine selected relationships with source passages. Graph retrieval makes these connections available during evidence selection.

2.4 Graph-based Evidence Retrieval

Graph indexes connect entities, passages, labels, and other evidence items through extracted facts, shared mentions, or semantic similarity. Retrieval selects the nodes and connections relevant to the current query.

GraphRAG builds an entity graph and community summaries for query-focused summarization [17]. Community summaries provide a retrieval unit above individual passages and support questions about broader corpus themes.

HippoRAG combines language-model extraction, a knowledge graph, and personalized PageRank [18]. Query entities seed propagation through the graph, producing a ranking of related evidence. This ranking does not require all selected nodes to belong to one terminal-connecting structure.

LightRAG combines graph and vector representations, low- and high-level retrieval, and incremental index updates [19]. Its retrieval modes select specific entity information and broader relational or conceptual information.

Table 2.1 compares the evidence organization of these approaches with passage retrieval. It identifies the evidence object made available to each retrieval computation.

Personalized propagation ranks evidence by influence from query anchors. Terminal-connecting retrieval instead selects paths that include a specified set of nodes, admitting intermediate nodes when needed. The former locates relevant graph regions; the latter constructs connections among required evidence items.

A graph result must be converted into model input. Entity retrieval requires finding asso-

Table 2.1: Representative retrieval organizations and the evidence structures they expose. Each row refers to the cited method's design.

Method	Indexed representation	Query-related retrieval operation
DPR [16]	Dense passage representations	Question–passage similarity scoring
GraphRAG [17]	Entity graph and community summaries	Use of graph-derived summaries for query-focused summarization
HippoRAG [18]	Knowledge graph linked to passages	Personalized PageRank from query-related entities
LightRAG [19]	Graph and vector representations	Dual-level retrieval of specific and broader information

ciated source passages. Path retrieval requires serializing entities and relations. Community retrieval can return derived summaries. These choices determine which original details and relationships remain available to generation.

The thesis examines several structural retrieval questions within this landscape. GORAG retrieves labels by connecting keywords extracted from the query [2]. AGRAG selects a connected evidence subgraph while considering node influence and edge cost [3]. MGRAG tests semantic correspondence between query paths and evidence paths [4]. These operations have different output conditions. A set of highly ranked nodes need not contain paths that connect the query's requested facts. Conversely, a connecting path can introduce intermediate evidence whose direct query similarity is lower.

Passage nodes link extracted relations to their original wording. Combining graph serialization with source chunks gives the generator both connected facts and surrounding context. AGRAG uses this combination for text, and MGRAG retains source chunks containing the original images.

2.5 Multimodal Retrieval and Context Serving

Multimodal question answering requires evidence represented in more than one modality. MultiModalQA includes questions over text, tables, and images and studies combinations of evidence across them [20]. The evidence needed for a question can occur in different source objects. A relationship stated in text may identify the image whose visual content contains the answer. Retrieval must therefore consider both the relevance of individual objects and the relationship connecting them to the question.

Images can be ranked through captions or visual embeddings. Captions expose selected visual content in a text embedding space, while the original images preserve details for multimodal

generation. MGRAG ranks images through captions and supplies selected original chunks to the answering model [4].

MRAMG-Bench studies retrieval-augmented generation in which answers can combine text and images [21]. Its multimodal output requirement adds another distinction between finding supporting evidence and presenting the response. The MGRAG evaluation uses six MRAMG subsets together with MMQA. Chapter 6 specifies the retrieval and answer-assessment protocol used for these datasets.

Once evidence is retrieved, the system must process it as model context. In an autoregressive Transformer, the prefill stage computes representations for the supplied input, and decoding generates answer tokens. Key and value tensors from processed input positions can be cached for later attention computation [7, 22]. Reusing these tensors concerns the computation associated with selected evidence. It is a separate decision from choosing which evidence belongs in the context.

CacheGen addresses the storage and transfer of context caches through compression and streaming [23]. The relevant resource includes the cost of fetching cached tensors as well as computing context representations. CacheBlend studies combining precomputed caches for several chunks by selectively recomputing part of their key and value states [22]. Its motivation includes interactions between chunks that were not present when their caches were independently computed. These methods place context serving within the evidence pipeline, where retrieval order and the assembled input can affect the computation required for generation.

MGRAG supplements attention-based recomputation scores with features of the retrieved graph [4]. The graph therefore guides both context selection and the computation reused when that context is processed.

2.6 Structural Information in Hyper-relational Knowledge Graphs

This section presents the author's published work *Understanding the Embedding Models on Hyper-relational Knowledge Graph*, published at CIKM 2025 [5]. It examines how graph representations and embedding architectures preserve information used for link prediction. The study provides a separate perspective on structural information within the thesis. Its experiments concern KG embedding models, while the four research chapters examine language-model augmentation.

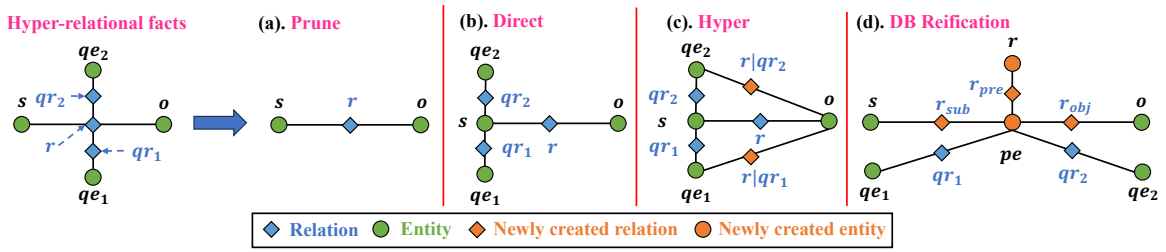


Figure 2.1: Original comparison of hyper-relational decomposition methods. Prune retains the main triple; Direct adds subject–qualifier links; Hyper adds combined relations; database reification introduces auxiliary objects. Reproduced from Fig. 1 of the CIKM study [5].

2.6.1 Facts, Qualifiers, and Decomposition

A hyper-relational fact extends a main triple with a set of qualifier pairs,

$$F = (s, r, o, \mathcal{Q}), \qquad \mathcal{Q} = \{(qr_i, qe_i)\}_{i=1}^{n}. \tag{2.1}$$

Here qr_i is a qualifier relation and qe_i its associated entity. Qualifiers attach information to a particular statement. For example, degree and major can distinguish two education statements about the same person. Their association with the intended main triple is part of the information represented by the fact. StarE retains the semantic roles of qualifiers and triples in message passing and introduces WD50K for hyper-relational link prediction [24].

The CIKM study asks how much of an HKGE model's performance comes from its base KGE architecture and how much comes from specialized qualifier processing. It evaluates a data-level alternative to modifying the model. Three decomposition methods convert a hyper-relational fact into ordinary triples while retaining different information. Prune keeps only the main triple. Direct adds triples connecting the subject to qualifier entities through their qualifier relations. Hyper adds further relations whose names combine the main relation with each qualifier relation and connect the qualifier entity to the main object.

Figure 2.1 reproduces the original comparison, including database reification. The study distinguishes main-triple information, intra-qualifier information, and inter-qualifier information. A decomposition can retain qualifier entities and relation labels while changing how their association with the main fact is expressed. Database reification introduces auxiliary entities and relations, also changing the original main-triple topology. These differences determine which neighborhoods and paths an ordinary KGE model receives after conversion.

Qualifier associations matter when a subject participates in several facts with the same relation. For example, a degree qualifies a particular university attendance statement. Separate subject–degree and subject–university triples retain the entities but change how that association

is represented. Hyper adds connections intended to preserve these associations. A qualifier integrator instead receives the main fact and its qualifier pairs together.

We compare conventional KGE models under these transformations with specialized HKGE models. Some GNN-based configurations obtain comparable scores on Cleaned JF17k and WD50k. Adding qualifier-derived edges changes the graph paths available to an encoder, so information preservation depends on both the transformed data and the model that processes it.

2.6.2 Qualifier Capture and Graph Reception

The analysis separates qualifier capture from graph reception. Qualifier capture concerns how the model represents qualifier pairs and their association with the main statement. Graph reception concerns how it receives information from the wider graph, including dependencies beyond one fact. Models operating on a fact in isolation have a different receptive field from graph encoders that propagate information across several hops. These properties are visible in the operations used to construct representations.

CompGCN jointly models entity and relation representations in a multi-relational graph [25]. NBFNet aggregates path information for link prediction [26]. Their graph computations offer different ways to receive neighborhood and path information after decomposition. The CIKM analysis also identifies an information-compression issue when qualifier information is aggregated into fixed-size main-triple representations. Wider graph reception and qualifier integration consequently concern complementary parts of the architecture. The study evaluates designs that retain both sources for prediction.

FormerGNN combines a Transformer-based qualifier integrator operating on the original hyper-relational facts with a graph encoder operating on the pruned main-triple graph. The graph encoder generates a topology representation for the subject. The model concatenates this representation with representations of the main triple and qualifier pairs and passes them to a Transformer decoder. This organization supplies qualifier and graph information jointly to the prediction stage. In the reported configuration, HyperFormer provides the qualifier integrator and NBFNet provides the graph encoder [26, 27].

2.6.3 Representative Published Evidence

Table 2.2 reproduces the Cleaned JF17k and WD50k columns of the study's Table 6. MRR measures mean reciprocal rank, and H@1 and H@10 measure whether the target entity appears within the corresponding rank cutoff. These are link-prediction measures, distinct from the an-

Table 2.2: Projection of the CIKM study's Table 6 onto Cleaned JF17k and WD50k. All model rows and the original metric precision are retained; the original bold and underline markings indicate best and second-best values.

	Cleaned JF17k			WD50k		
Model	MRR	H@1	H@10	MRR	H@1	H@10
StarE	0.381	0.284	0.577	0.346	0.268	0.494
QUAD	0.402	0.305	0.602	0.349	0.275	0.487
HAHE	0.405	0.310	0.585	0.360	0.288	0.496
HyperFormer	<u>0.425</u>	<u>0.328</u>	0.613	<u>0.372</u>	<u>0.293</u>	0.522
NBFNet + Prune	0.415	0.313	**0.630**	0.360	0.271	**0.530**
CompGCN + Hyper	0.389	0.294	0.590	0.350	0.277	0.488
FormerGNN	**0.432**	**0.335**	<u>0.624</u>	**0.377**	**0.299**	<u>0.527</u>

notation and answer metrics of the later chapters.

FormerGNN obtains MRR 0.432 on Cleaned JF17k and 0.377 on WD50k, compared with HyperFormer's 0.425 and 0.372. NBFNet + Prune has the highest H@10 in both selected datasets, at 0.630 and 0.530. The experiment uses a single NVIDIA A100-40GB GPU. Parameters follow the original repositories or are selected by grid search when dataset-specific settings are unavailable. For comparisons involving CompGCN, the study replaces its decoder with a Transformer to align it with the corresponding HKGE configurations.

The complete comparison also includes Wikipeople and FBAUTO, with different dataset characteristics and metric rankings. The study shows how changing the organization of graph facts changes the information available to a prediction model.

2.7 Research Questions Emerging from the Background

The preceding work identifies several places where structure can influence model inputs. Table understanding needs evidence that matches both the values and the task's type vocabulary. Dynamic classification needs an index that incorporates newly available label information and retrieves appropriate alternatives for the current text. Open-ended text generation needs a selection rule that can organize connected facts and retain their source context. Multimodal generation adds semantic matching across heterogeneous evidence and the cost of processing that evidence as model context.

CHAPTER 3

KNOWLEDGE-ENRICHED TABLE UNDERSTANDING WITH KGLINK

This chapter is based on the published paper *KGLink: A Column Type Annotation Method that Combines Knowledge Graph and Pre-Trained Language Model* [1]. We propose KGLink to annotate table columns using external graph knowledge. The method selects candidate types and graph-derived feature sequences, then learns their correspondence to the annotation labels through a contextual representation objective.

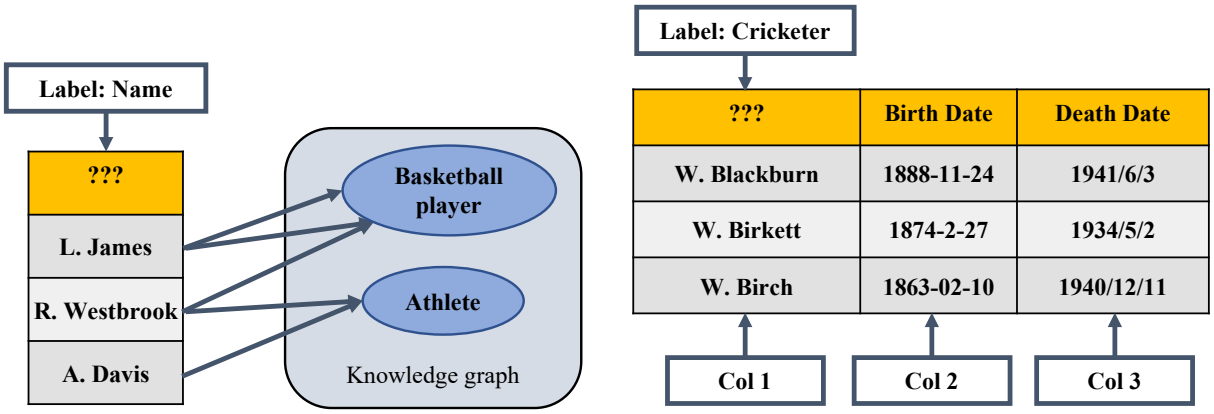
3.1 Motivation and Task

A table may contain too little context to determine a column type. For example, names of cricket players accompanied by birth and death dates do not reveal their occupation. Linking those names to a knowledge graph can supply occupation and sport relations that are absent from the table.

External knowledge introduces a second difficulty. The type expected by an annotation dataset need not coincide with a type attached to an entity in the graph. A column of basketball players can plausibly be described as *Basketball player*, *Athlete*, or *Name*. An annotation benchmark may use only the last of these labels. Retrieving a factually relevant graph type is consequently insufficient to determine the required output label. The model must learn which description the task expects. KGLink refers to this mismatch as the type granularity gap. The gap includes cases where the desired dataset label does not occupy a convenient ancestor position in the retrieved graph's type hierarchy.

We address missing context and type granularity through separate steps. Graph processing selects relevant external descriptions, while a supervised classifier learns the dataset labels. An auxiliary task trains a contextual representation of the target type so that the classifier can use graph descriptions even when their wording differs from the annotation vocabulary.

We evaluate single-label column annotation on SemTab and modified VizNet. The encoder predicts a dataset label from the cells, table context, and selected graph descriptions.



(a) Type granularity (b) Missing context

Figure 3.1: Two annotation difficulties in the original KGLink examples. A graph description such as Athlete can differ from the target label Name, while dates beside a person's name provide little occupation information. Original subfigures reproduced from KGLink [1], Fig. 2.

3.2 Problem Formulation and Framework

Let T be a table with row set R and column set C. A cell mention m_c^r is the content of row r and column c. The task assigns a label l_c from the dataset's type set D to each target column. All columns in the same table can contribute context to that decision. The external knowledge graph supplies entity labels and their one-hop relations. During training, column labels supervise both classification and the auxiliary representation task. During evaluation, those labels are absent from the model input.

KGLink has a graph-processing stage followed by a language-model stage. The graph stage retrieves candidate entities for cell mentions, filters these entities using connections between columns, ranks rows by their linkage quality, and derives candidate types from the remaining entities' neighborhoods. It also constructs a feature sequence for each target column. The language-model stage serializes the processed table, encodes the table and feature sequences, combines the resulting representations, and predicts the column labels. A second training input contains the true type labels and provides contextual supervision for representations generated from masked type positions.

Figure 3.2 shows the two stages and their training objectives.

Candidate types summarize descriptions supported by linked cells across a column. The feature sequence retains an entity and its neighboring relations from a selected cell. Both enter the learned classifier, which predicts a label from the dataset vocabulary.

Graph descriptions and cell values share the encoder input budget. We therefore select rows before serialization and split wide tables by columns under the input limits.

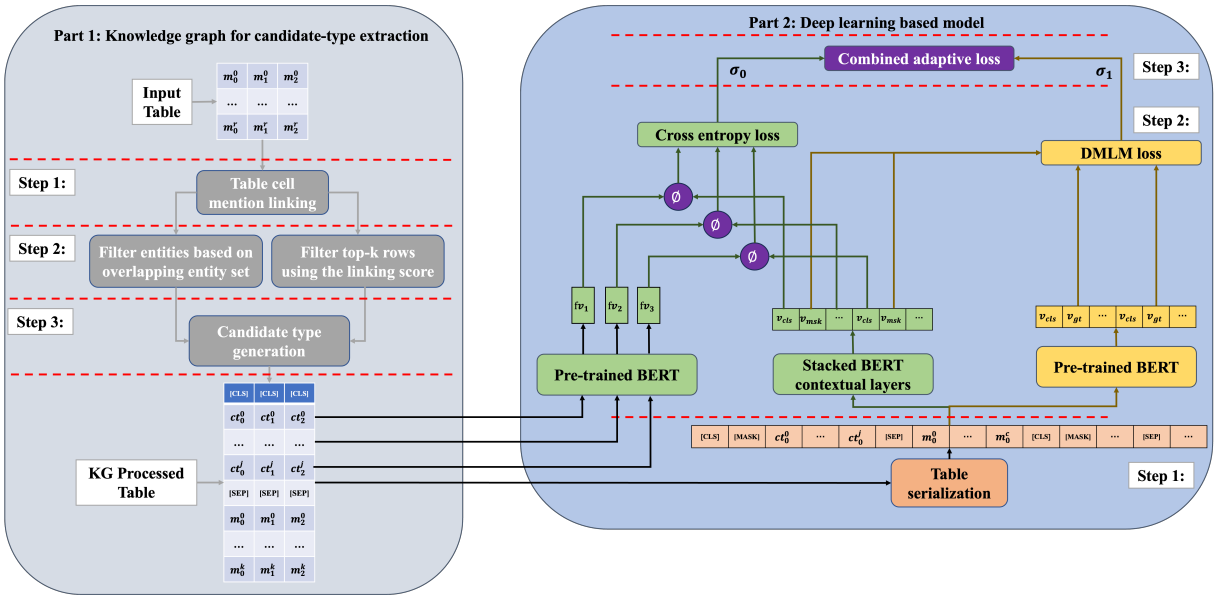


Figure 3.2: KGLink's graph-processing and language-model components. Candidate-type text, column feature vectors, and a contextual type-representation objective connect external knowledge to annotation. Original image reproduced from KGLink [1], Fig. 3.

3.3 KG-derived Candidate Types

3.3.1 Cell Mention Linking

The first step retrieves a candidate entity set $E_{m_c^r}$ for each suitable cell mention. KGLink uses BM25 to score the relation between the mention's words and indexed entity labels [15]. The implementation indexes Wikidata with Elasticsearch. The score incorporates query-term frequency, inverse document frequency, and label-length normalization. Multiple entities can match the same mention, so the retrieval output is a candidate set rather than a single disambiguated entity.

Short strings are often ambiguous. An album title can match an ordinary word, and a name can refer to several people. We use relationships between cells in the same row to filter these lexical candidates.

Named-entity detection identifies numeric and date mentions, which receive a linking score of zero. Failed lookups also receive zero. The original cell values remain in the encoder input, allowing classification when KG evidence is missing.

We retrieve at most ten entities per cell mention, then apply neighborhood filtering and row ranking before constructing the encoder inputs.

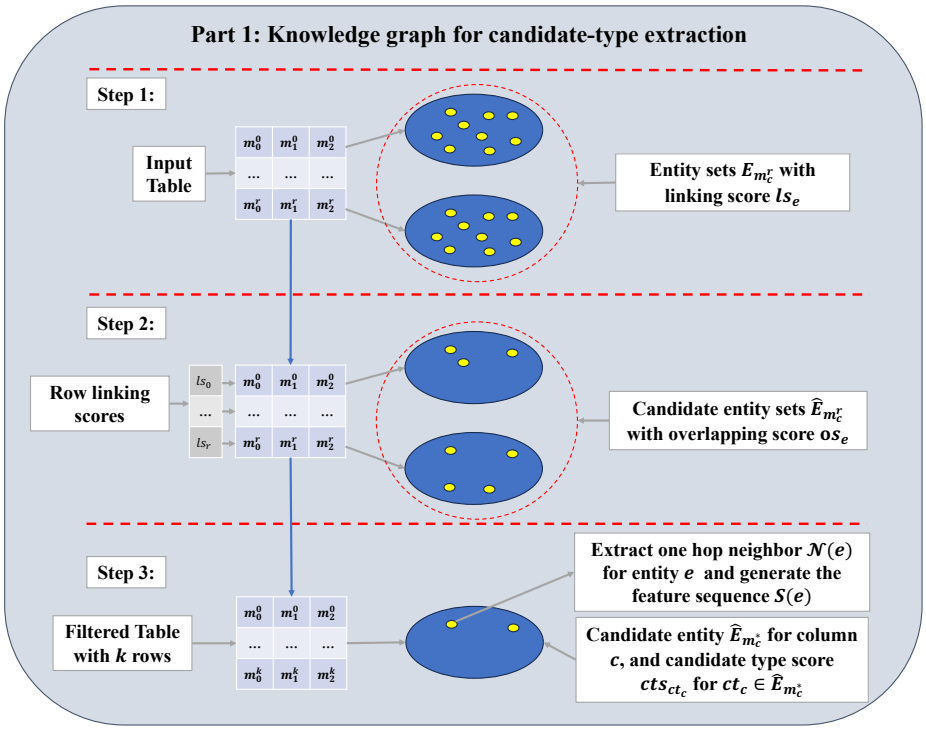


Figure 3.3: The original KG-processing pipeline. Cell linking is followed by entity filtering, row selection, and candidate-type generation. Reproduced from KGLink [1], Fig. 4.

3.3.2 Cross-column Neighborhood Filtering

Cells in the same row describe related objects. An album title beside a musician's name supplies a useful relation that either mention lacks in isolation. KGLink uses this relation by checking whether a candidate entity for one cell occurs in the one-hop neighborhood of candidates for another cell in the same row. Let $\mathcal{N}(E)$ denote the one-hop neighbors of a candidate entity set. The filtered set for a cell is

$$\widehat{E}_{m_{c_1}^r} = \bigcup_{c_2 \in C \setminus \{c_1\}} \left(E_{m_{c_1}^r} \cap \mathcal{N}(E_{m_{c_2}^r}) \right). \tag{3.1}$$

The union allows support from any other column. The intersection retains entities retrieved for the target mention that also have the required neighborhood support.

Figure 3.4 uses *Rust* and *Peter Steele*. A graph connection between an album candidate and the musician candidate supports their interpretation as the entities intended by the row. The role of the edge is to constrain which external facts are relevant to the table. KGLink subsequently associates each retained entity with an overlapping score that counts its neighborhood support from other columns. Entities with more such support contribute more to candidate-type selection.

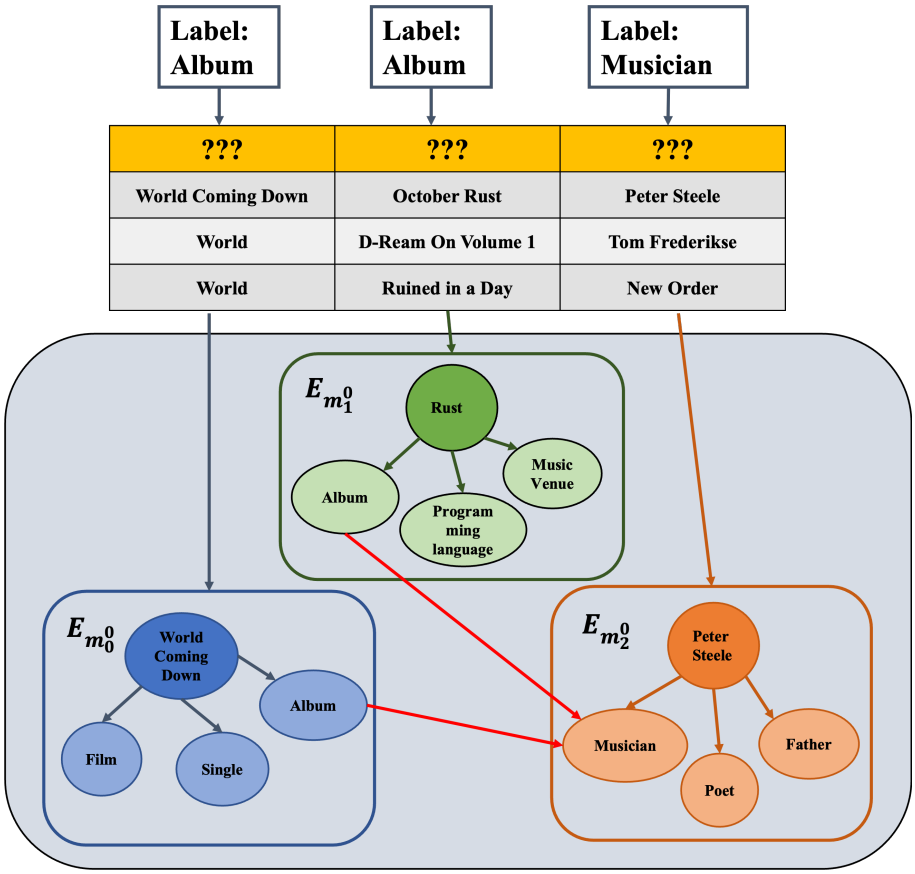


Figure 3.4: Cross-column evidence in the original album-and-musician example. Graph links connect candidate interpretations of cell mentions in the same row. Original image reproduced from KGLink [1], Fig. 5.

Filtering depends on both the initial entity candidates and the relations present in the knowledge graph. A missing relation can discard a useful candidate, while an incorrect candidate can survive through a matching contextual link. The encoder also receives the original table values and the separate feature sequence.

Filtering uses relationships between cells in the same row. Candidate-type generation then aggregates recurring descriptions across rows in a column.

3.3.3 Row Linking Quality and Input Selection

KGLink derives a cell score from the best-scoring retained entity. For a nonempty filtered candidate set, the score is

$$ls_{m_c^r} = \max_{e \in \hat{E}_{m_c^r}} ls_e, \tag{3.2}$$

where ls_e is the entity's BM25 linking score. Unlinked cells use zero as described in the evaluation protocol. A row's score sums the scores of its cells,

$$ls_r = \sum_{c \in C} ls_{m_c^r}. \tag{3.3}$$

Rows are sorted by this quantity, and only the top k rows are retained for prediction.

Linkage ranking favors rows with graph-supported candidates, while taking the first k rows follows the source order. The row-filter experiment compares these rules under the same input budget.

The highest-ranked row also supplies the selected cell used for feature-sequence generation. Row ranking therefore changes both the table values and the KG entity passed to the encoder.

3.3.4 Candidate Types and Feature Sequences

Candidate-type generation searches beyond an entity's explicit type attribute. A person's graph entry may be typed as *Human*, while *Musician* appears as an occupation neighbor. The latter can be more informative for a table column. KGLink collects filtered entities across the target column and examines their one-hop neighborhoods. It scores a potential type by aggregating the overlapping scores of entities from other rows that connect to that type. Repeated support within the column increases the candidate's score.

A label-based filter excludes candidates assigned the named-entity schema categories `PERSON` or `DATE`. The reported implementation uses spaCy for this step. This filter distinguishes a descriptive candidate type from a particular person's name or a date encountered in the neighborhood. KGLink retains the selected candidate-type labels as text and places them before the cell values in the processed column. The reported implementation generates up to three candidate types per row from the KG entities in each column. Their later contribution depends on how the encoder interprets them in the table context.

If no candidate type is returned, padding occupies the corresponding position. Numeric columns use numeric summary information in this part of the input, while their original values remain available in the table sequence.

The feature sequence provides a second representation of the retrieved knowledge. From the filtered and ranked table, KGLink selects a cell for each target column, chooses its highest-linking entity, and serializes that entity with its one-hop predicates and neighboring entities. If e is the selected entity, p a predicate, and o a neighboring object, the sequence consists of the entity label followed by predicate–object descriptions. Encoding this text yields a feature vector

for the column. A padding sequence is used when the column has no retrieved entity.

3.4 Language Model Integration

3.4.1 Table Serialization and Column Representations

KGLink adapts the multi-column serialization used by Doduo [11]. A special `[CLS]` token begins each column's segment. The segments are concatenated into one input sequence, followed by a `[SEP]` token. The contextual representation at each column's `[CLS]` position provides that column's vector. Because the sequence contains multiple columns, self-attention can use information outside the predicted column.

Writing the tokenized contents of column i as $\mathrm{tok}_i^1, \ldots, \mathrm{tok}_i^m$, the original multi-column serialization has the form

$$\begin{aligned} S(T) = {} & [\mathrm{CLS}]\ \mathrm{tok}_1^1 \cdots \mathrm{tok}_1^m\ [\mathrm{CLS}]\ \mathrm{tok}_2^1 \cdots \\ & [\mathrm{CLS}]\ \mathrm{tok}_{|C|}^1 \cdots \mathrm{tok}_{|C|}^m\ [\mathrm{SEP}]. \end{aligned} \tag{3.4}$$

Each column's `[CLS]` position produces a contextual vector using the serialized columns. Candidate-type text precedes the corresponding cell values, allowing both to interact with neighboring columns through self-attention. KGLink separately encodes the feature sequence into a column-specific vector and combines it with the contextual column vector through the composition function ϕ. The resulting representation is mapped into the dataset's classification space.

The principal experiments fine-tune BERT [8]. The model has a sequence limit of 512 tokens; the reported implementation limits each column to 64 tokens and each encoded table to eight columns. Tables with more columns are divided after candidate-type generation and are encoded separately. These limits define the context available to the classifier. In particular, retaining 25 table rows does not imply that every token in those rows survives serialization, since tokenization and the column limit also constrain the final sequence.

3.4.2 Contextual Type Representation Learning

The auxiliary task trains a type representation at a masked position. During training, KGLink constructs two versions of the processed table. The ground-truth version places the annotated type label before the candidate types in each target column. The masked version replaces that label with `[MASK]`. Both versions retain the column values and candidate-type context. En-

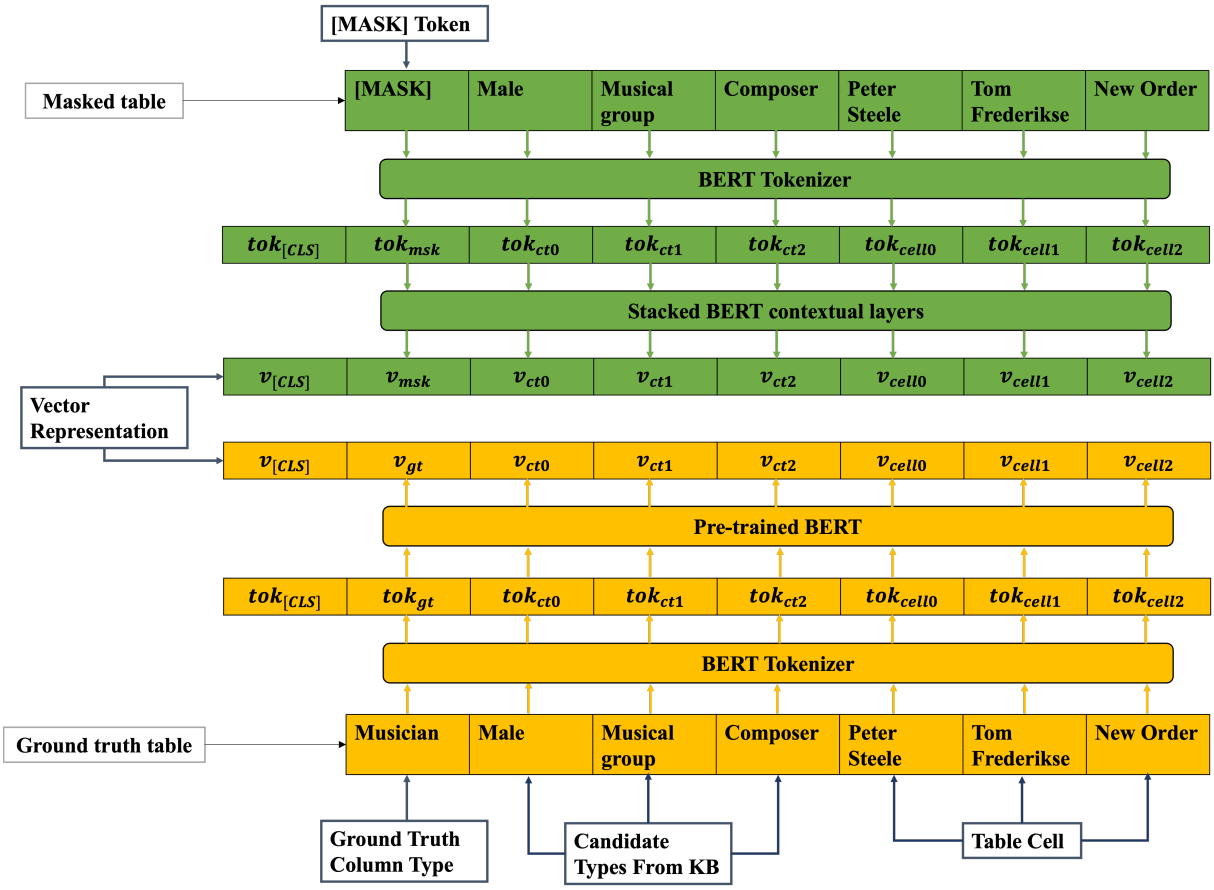


Figure 3.5: Contextual type-representation training with masked and ground-truth table inputs. The true-type table is constructed during training. Original image reproduced from KGLink [1], Fig. 6.

coding the two sequences produces contextual representations for the true type and its masked counterpart.

Figure 3.5 illustrates the two inputs and their contextual outputs.

This arrangement supplies supervision about the label's meaning in context. A label such as *Musician* is encoded together with the column values and graph-derived descriptions, rather than as an isolated word. The masked input must produce its type representation from the evidence that will be available at prediction time. Distilled masked language modeling provides the auxiliary objective [28]. The model projects contextual representations into vocabulary space with a temperature parameter, then uses the ground-truth and masked outputs in the reported training loss.

With contextual vector H and temperature $\mathcal{T}$, vocabulary projection gives

$$Y = \text{Softmax}\left(W_o \frac{H}{\mathcal{T}}\right), \tag{3.5}$$

where W_o projects into the vocabulary space. Denote the masked and ground-truth outputs by

Y_{msk} and Y_{gt}. The main paper writes the auxiliary objective as

$$\mathscr{L}_{\text{DMLM}}(Y_{\text{msk}}, Y_{\text{gt}}) = -\sum_{j=1}^{V} y^{j}_{\text{msk}} \log y^{j}_{\text{gt}}, \tag{3.6}$$

with vocabulary size V. The objective uses the contextualized representations produced from the two table inputs.

Training uses both true-type and masked inputs; evaluation uses the masked input. The model thus learns from annotated types during training and predicts from table values and retrieved descriptions at inference, including for numeric columns and columns without KG candidates.

The shared encoder is trained jointly for dataset-label classification and vocabulary-space type representation. The classification head produces the final column label.

3.4.3 Classification and Adaptive Task Weighting

Let z_c be the classification logits obtained from the combined column and feature representations. For a target label l_c, the classification loss is cross-entropy,

$$\mathscr{L}_{\text{CE}}(z_c, l_c) = -\log \frac{\exp\left(z_{c,l_c}\right)}{\sum_{j \in D} \exp\left(z_{c,j}\right)}. \tag{3.7}$$

KGLink jointly optimizes this loss and the auxiliary representation objective using uncertainty weighting [29]. With trainable task parameters σ_0 and σ_1, the combined objective is

$$\mathscr{L}_{\text{total}} = \frac{\mathscr{L}_{\text{DMLM}}}{2\sigma_0^2} + \frac{\mathscr{L}_{\text{CE}}}{2\sigma_1^2} + \log\left(\sigma_0 \sigma_1\right). \tag{3.8}$$

The temperature used in the representation objective is set to two in the reported experiments.

The trainable task weights balance the representation and classification losses during optimization.

3.5 Computational Structure

The graph-processing cost depends on table dimensions, candidate-set sizes, and neighborhood sizes. Linking traverses table cells. Cross-column filtering compares candidate information between columns within a row. Candidate-type scoring visits neighborhoods and aggregates

support across rows. The implementation's limits on retrieved entities and serialized columns constrain these operations in the evaluated setting. The external Wikidata index is prepared with Elasticsearch and supplies lexical retrieval scores.

The original analysis denotes maximum table rows and columns by R and C, the number of tables by T, the maximum retrieved entities by E, and their maximum one-hop neighborhood size by N. It reports the preprocessing expression

$$O(RCE(N + CET)) . \tag{3.9}$$

Neighborhood aggregation and cross-column candidate filtering contribute to preprocessing. Increasing the candidate-set limit adds graph work even when the encoder input remains capped.

Encoding cost depends on the serialized input length and the encoder's hidden dimension. KGLink encodes table inputs and column feature sequences; training also includes the ground-truth and masked constructions used by the representation task. The classification head maps column vectors into the label space, while the auxiliary projection operates over the language model's vocabulary. Thus, increasing the label set affects classification, whereas increasing the vocabulary-space projection concerns the representation objective.

Reducing the row count need not shorten a sequence already truncated to its token limit. Graph preprocessing and encoder computation can therefore respond differently to the same input-selection change.

3.6 Experimental Evaluation

3.6.1 Datasets and Protocol

The evaluation uses SemTab 2019 and a modified VizNet subset [10, 30]. SemTab includes tables derived from Wikipedia, DBpedia, and the T2Dv2 Gold Standard sources. The experiment uses rounds 1, 3, and 4, with 3,048 tables, 7,587 columns, and 275 column types. Round 2 has columns with multiple ground-truth labels and is excluded from this single-label comparison. The reported averages are 69 rows and 4.5 columns per table.

The modified VizNet corpus targets column annotation rather than the full range of visualization tasks in the original repository [31]. The experiment uses its multi-column portion so that methods can exploit relations between columns. Its annotation scheme is coarser than SemTab's and includes numeric types. The evaluated portion contains 32,265 tables, 73,034

Table 3.1: Column-type annotation results reported for KGLink. Scores and missing entries reproduce the original main-result table.

	SemTab		VizNet	
Model	Accuracy	Weighted F1	Accuracy	Weighted F1
MTab	**89.10**	-	38.21	-
TaBERT	72.69	71.21	94.68	94.07
Doduo	84.06	82.43	95.40	95.06
HNN	66.54	65.12	66.89	68.82
Sudowoodo	79.34	79.24	91.57	91.08
RECA	86.12	84.91	93.25	93.18
KGLink	87.12	**85.78**	**96.28**	**96.07**

annotated columns, and 77 types.

The data are split into training, validation, and testing portions in the ratio 7:1:2 while maintaining each class's original sample proportion. Each experiment is averaged over three runs. Accuracy measures exact label prediction, and weighted F1 weights class-level performance by sample frequency.

Training uses AdamW, an initial learning rate of 3×10^{-5}, and $\epsilon = 10^{-6}$. The learning rate decays linearly without warm-up. The reported maximum training lengths are 50 epochs on SemTab and 20 on VizNet, with early stopping. Batch size is 16 for both datasets; dropout is 0.1 for SemTab and 0.2 for VizNet. The implementation uses PyTorch and the Transformers library on an NVIDIA Tesla V100 GPU.

The baseline set includes MTab, TaBERT, Doduo, HNN, Sudowoodo, and RECA [11–13, 32–34]. TaBERT and Doduo use KGLink's experimental settings. HNN and RECA follow their original settings. Sudowoodo receives the full labeled training set in this comparison. VizNet labels are translated to Wikidata entities for MTab, and MTab's weighted F1 is unreported because of the difference in problem definition.

3.6.2 Main Annotation Results

Table 3.1 reports the main comparison. KGLink obtains accuracy 87.12 and weighted F1 85.78 on SemTab, and accuracy 96.28 and weighted F1 96.07 on VizNet. MTab has the highest SemTab accuracy, 89.10. KGLink has the highest reported weighted F1 on SemTab and the highest values of both reported metrics on VizNet.

Doduo provides a useful comparison for the language-model pathway because KGLink adapts its serialization and uses BERT in the main configuration. Doduo obtains 84.06 accuracy on SemTab and 95.40 on VizNet. KGLink improves on this baseline with graph-augmented

Table 3.2: Reported KGLink variant scores. The table reproduces the numeric columns and variant names of the original ablation table.

	SemTab		VizNet	
Variant	Accuracy	Weighted F1	Accuracy	Weighted F1
KGLink *w/o msk*	86.14	84.54	95.95	95.67
KGLink *w/o ct*	86.27	84.56	95.83	95.48
KGLink *w/o fv*	87.02	85.68	95.98	95.70
KGLink DeBERTa	**87.24**	**85.81**	**96.98**	**96.37**
KGLink	87.12	85.78	96.28	96.07

inputs and joint training.

The dataset dependence is also visible in MTab's results. Its accuracy is 89.10 on SemTab and 38.21 on VizNet under the label translation used in the experiment. These measurements motivate separating graph knowledge from the final annotation vocabulary.

3.6.3 Representation Learning and KG Evidence

Table 3.2 compares KGLink variants. The *w/o msk* variant removes the masked representation-generation task. Its scores are lower than the BERT-based full model on both datasets. The *w/o ct* variant removes the KG-derived information, including candidate types and the feature vector, while keeping the multitask language-model pathway. Its accuracy is 86.27 on SemTab and 95.83 on VizNet. Removing KG inputs reduces accuracy on both datasets.

The *w/o fv* variant removes the KG-derived feature vector, following the original method description. It obtains 87.02 accuracy and 85.68 weighted F1 on SemTab, and 95.98 accuracy and 95.70 weighted F1 on VizNet. Candidate types provide the remaining KG input in this configuration. The full model has higher scores on both datasets.

Replacing BERT with DeBERTa produces the fourth variant [35]. It obtains 87.24/85.81 on SemTab and 96.98/96.37 on VizNet. The architecture uses the same graph-processing design with a different encoder in this experiment.

3.6.4 Annotation without Retrieved Knowledge

The original study constructs a test subset from VizNet in which the entire table has no KG linkage. This condition is stricter than requiring only the target column to lack a link. With multi-column encoding, a target column can otherwise benefit from graph information retrieved for its neighbors. The subset contains 315 tables and 612 columns across 26 labels, comprising 556 numeric columns and 56 non-numeric columns.

Table 3.3: Reported accuracy on the VizNet test subset whose tables have no extracted KG information.

Model	Numeric accuracy	Non-numeric accuracy
KGLink	**97.04**	**90.92**
HNN	44.05	18.37
TaBERT	96.57	90.27
Doduo	96.28	89.50
RECA	96.89	61.54
Sudowoodo	96.21	67.72

Table 3.4: Reported comparison between linkage-ranked row selection and selection in original table order.

	SemTab		VizNet	
Selection	Accuracy	Weighted F1	Accuracy	Weighted F1
Top-k row filter	**87.12**	**85.78**	**96.28**	**96.07**
Original top-k rows	85.93	84.39	96.14	95.97

Table 3.3 compares annotation accuracy without retrieved KG evidence. It obtains 97.04 numeric accuracy and 90.92 non-numeric accuracy. TaBERT and Doduo also achieve high values on this subset. The table encoder supports annotation when external graph evidence is absent.

3.6.5 Type Granularity in the Reported Cases

Among SemTab classes with more than ten test samples, Athlete, Protein, and Film have the three largest reported accuracy increases when the representation-generation task is added.

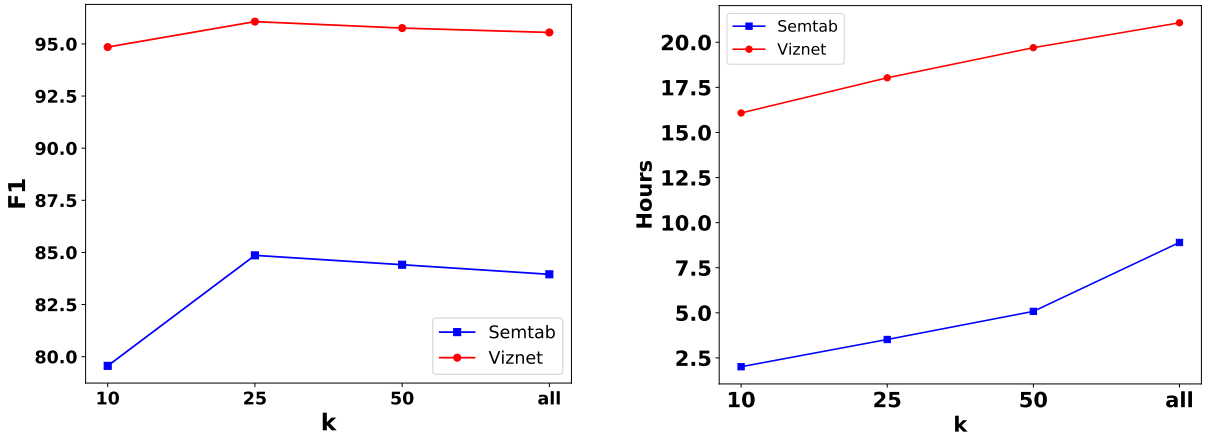
Protein columns can retrieve KG descriptions such as Gene or Enzymes, while Film columns can retrieve Television series, Scholarly article, or Book. These descriptions are related to the cell values but differ from the annotation labels. The contextual representation task learns how to use them jointly with the column contents.

The Athlete example shows the same granularity gap. KGLink retains the graph description as input and learns the annotation category from supervised examples.

3.6.6 Row Selection and Input Budgets

Table 3.4 shows that linkage-ranked rows improve annotation over taking the first rows on both datasets, with a larger change on SemTab. The filter also selects the cell used for the feature sequence, so the comparison measures both effects of row ranking.

The parameter study considers $k = 10, 25, 50$, and a condition labeled *all*. The source de-



(a) Weighted F1

(b) Time cost

Figure 3.6: Reported effect of the retained row budget k. The original two subfigures compare annotation quality and time cost on SemTab and VizNet. Reproduced from KGLink [1], Fig. 10.

scribes *all* as retaining at most 64 rows under its encoding configuration. The reported default is $k = 25$, selected from this study. The original curves compare weighted F1 and time cost as the retained row budget changes. Figure 3.6 shows the measured relation between input selection, weighted F1, and time cost.

The training-data study uses proportions $p = 0.2, 0.4, 0.6, 0.8, 1.0$ while keeping the test set unchanged. It compares KGLink with the version without the representation-generation task. The experiment measures data efficiency by varying available supervision against a fixed evaluation set.

3.6.7 Runtime in the Evaluated Configuration

Figure 3.7 reproduces the original VizNet runtime chart. KGLink's reported training and inference times are 16.50 and 1.53 hours. The corresponding RECA values are 80.00 and 9.00 hours. The figure also shows methods with shorter measured times, including Doduo at 1.96 hours for training and 0.07 hours for inference. These measurements locate KGLink's annotation results within the costs of the evaluated implementations. Its use of external retrieval and a second training objective carries computation beyond the table encoder alone.

The chart labels MTab's inference entry as N/A. Training and inference times describe different preparation and prediction procedures. The comparison uses VizNet and the original experimental hardware.

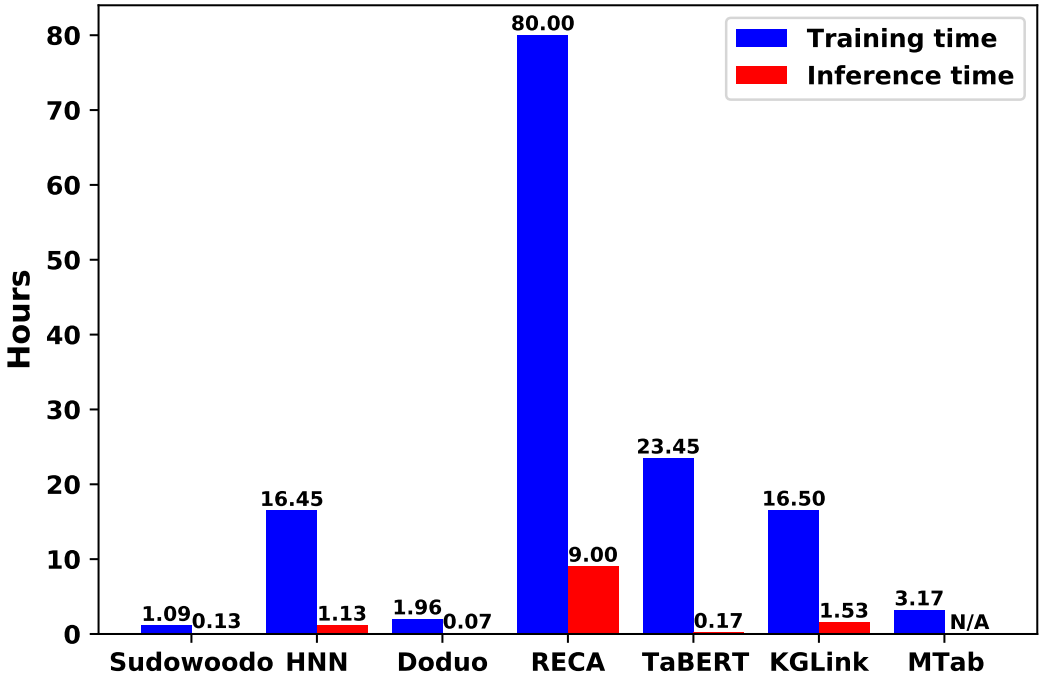


Figure 3.7: Training and inference time reported on VizNet. Original image reproduced from KGLink [1], Fig. 7.

3.7 Discussion and Chapter Summary

We proposed KGLink to supply missing table context and learn the correspondence between graph descriptions and annotation labels. Cross-column filtering selects entity candidates, recurring neighborhoods supply candidate types, and the contextual representation objective trains the encoder to use this information.

Removing either the masked representation task or KG information lowers the full BERT configuration's annotation scores. Linkage-based row selection also improves the results. On tables without retrieved KG evidence, the trained encoder continues to annotate from the table values.

Knowledge coverage and annotation vocabulary remain material conditions. The graph-processing stage can use only entities and relations available to its linker and neighborhood retrieval. The supervised predictor learns the type set represented in training. The evaluated datasets use single-label column annotation and bounded table inputs.

The next chapter studies a different classification problem, in which labels and examples arrive over successive rounds. It develops a task-specific graph that retrieves candidate labels and updates the index after each prediction.

CHAPTER 4

ADAPTIVE GRAPH RETRIEVAL FOR DYNAMIC CLASSIFICATION WITH GORAG

This chapter is based on *GORAG: Graph-based Online Retrieval Augmented Generation for Dynamic Few-shot Social Media Text Classification*, published in the Proceedings of the ACM Web Conference 2026 [2]. We propose GORAG to retrieve candidate labels for dynamic few-shot classification. It maintains a weighted graph of keyword–label associations, selects candidates by connecting query keywords, and inserts unseen terms after classification.

4.1 Dynamic Few-shot Classification

Dynamic classification must distinguish new categories from those introduced earlier. For example, an abuse classifier may initially recognize threatening language and later add identity insults. With few examples per class, the model must learn the new category while retaining access to earlier labels.

Few-shot classification provides a small labeled support set for each class. In the dynamic setting, new classes arrive in successive rounds [36]. The index must incorporate their examples while preserving evidence for the cumulative label set.

Adding more examples to the prompt can introduce irrelevant context as the label set grows. Passage retrieval alone also leaves the generator to choose among all labels. We instead retrieve a candidate label set for each query.

We represent keywords and labels in a weighted graph. A keyword can connect to several labels, with an edge cost for each association. Connecting several query keywords selects labels jointly from their neighborhoods.

GORAG also updates its index after classifying an unlabeled query. Keywords that were absent from the index can then become available to later queries. The predicted label associates the newly observed keywords with a class for subsequent retrieval. Support examples initialize the graph using their supplied labels.

GORAG combines weighted indexing, connected candidate-label retrieval, and online insertion. The following sections define these operations and evaluate their classification accuracy

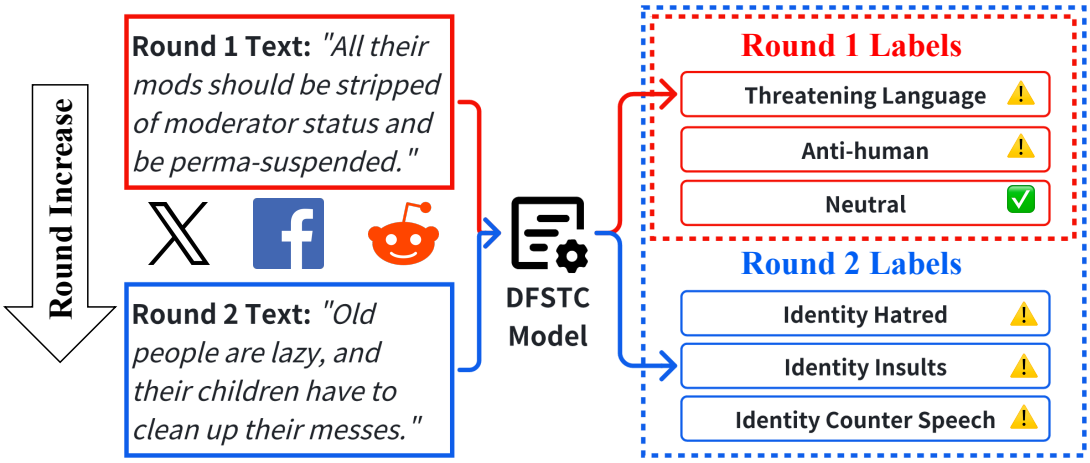


Figure 4.1: Two rounds of the dynamic classification task. New abuse categories extend the available labels. Reproduced from Figure 1 of GORAG [2].

and resource costs.

4.1.1 Rounds and information availability

Let $\mathcal{Y}_{\text{new}}^{r}$ denote the labels introduced at round r. The current label universe is

$$\mathcal{Y}^{r} = \bigcup_{i=1}^{r} \mathcal{Y}_{\text{new}}^{i}. \tag{4.1}$$

The few-shot support data provide k annotated texts per class, with k equal to 1, 5, or 10 in the main experiments. A query text t must be assigned a label from $\mathcal{Y}^{r}$. The superscript r identifies the current classification round.

Support texts carry supplied labels. Query texts become visible as the system processes them, and their predictions can be used for subsequent index updates. Corpus statistics used in keyword scoring refer to training and query texts seen so far. The online association nevertheless inherits prediction errors when the classifier chooses an incorrect label.

Semantic label names supply another source of information. Where names are available, GORAG uses a language model to generate representative descriptions or keywords for labels. These descriptions enter the classification context, and label-derived keywords can support graph initialization. The zero-shot experiment later in this chapter uses this source without any labeled examples.

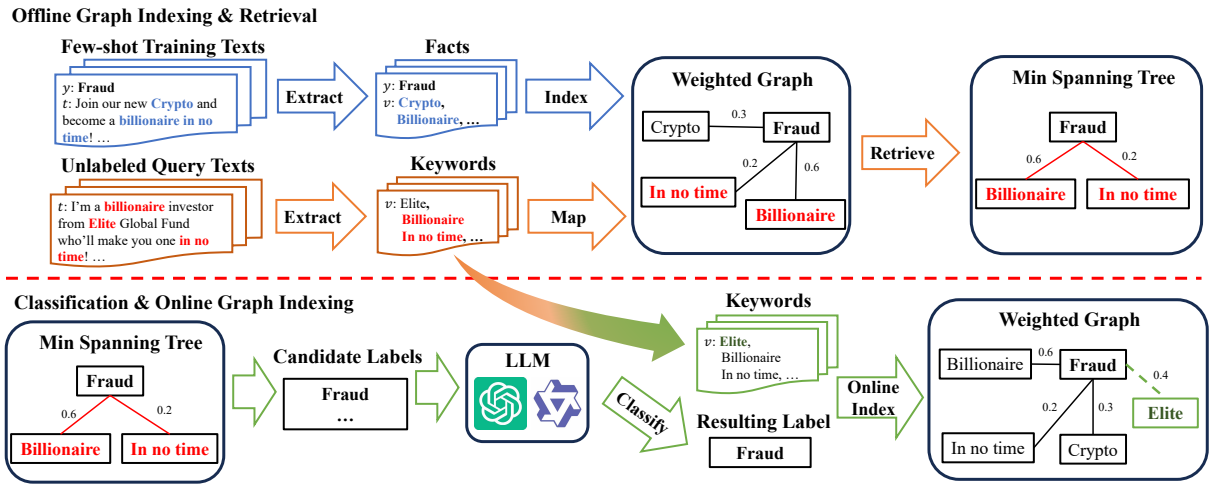


Figure 4.2: GORAG's offline indexing, query-dependent retrieval, classification, and online update. New query keywords are associated with the predicted class after classification. Reproduced from Figure 2 of GORAG [2].

4.2 Weighted Graph Indexing

GORAG maintains an undirected weighted graph $\mathcal{G}^r = (\mathcal{V}^r, \mathcal{E}^r, \mathcal{W}^r)$. Its node set contains keyword nodes and label nodes. A keyword node represents an extracted word or short phrase. A label node represents a target class. The representation places label alternatives inside the same structure that stores their lexical associations. Support documents contribute keyword–label associations to this representation.

A keyword–label edge is introduced when a keyword occurs in a text associated with that label. Repeated occurrences across texts contribute to the edge weight. A keyword shared by several classes can therefore have several label neighbors, with a different cost for each association. Query retrieval will use these costs when choosing connections between mapped keywords.

The graph stores keyword–label associations, while the classifier receives the original query, retrieved labels, and their descriptions. The full query retains syntax, word order, and discourse omitted by keyword extraction. The keyword-only ablation removes this full-text input.

4.2.1 Keyword extraction and correlation scores

GORAG scores terms using normalized TF–IDF [37]. Its extraction vocabulary includes words and phrases of fewer than three words. Let $\mathcal{T}_{\leq r}$ denote the corpus seen so far, and let $\mathrm{df}(v)$ be the number of its texts containing term v. The reported correlation score is

$$CS(v,t) = \frac{\mathrm{count}(v,t)}{|t| \log\left(|\mathcal{T}_{\leq r}| + 1\right)} \log \frac{|\mathcal{T}_{\leq r}| + 1}{\mathrm{df}(v) + 1}. \tag{4.2}$$

Here count(v, t) is the occurrence count of v in t, and $|t|$ is text length. The expression combines frequency within the text with frequency across the observed corpus. Its corpus term changes when additional texts are observed. Terms with $CS(v, t) > \tau$ are retained, with $\tau = 0.5$ in the original study. The method uses the score both to select keywords and to calculate association costs.

Short phrases preserve lexical combinations that unigram extraction would split. Only extracted terms can become retrieval terminals, although the classifier still receives the full query text.

4.2.2 Keyword–label edge costs

For keyword v and label y, let $\mathcal{T}^{r,v,y}$ be the seen texts that contain v and are associated with y. GORAG assigns the edge cost

$$w^r_{v,y} = \frac{\sum_{t_j \in \mathcal{T}^{r,v,y}} (1 - CS(v, t_j))}{|\mathcal{T}^{r,v,y}|}. \tag{4.3}$$

The complement converts a larger correlation score into a smaller contribution to connection cost. Averaging makes the edge depend on all contributing observations rather than on one occurrence. Offline observations use the supplied class of a support text; online observations use a predicted class when new query keywords are inserted.

The same keyword can have different edge costs for different labels. The unit-weight ablation replaces all costs with one while preserving adjacency, testing whether these association-specific costs help retrieval.

Each observed text contributes a correlation value to its keyword–label pair. Averaging these values summarizes repeated associations without concatenating every support text into the classification prompt.

4.2.3 Merging label rounds

At each round, we merge the new graph with the accumulated index by taking the unions of their node and edge sets. Earlier labels and their associations remain available for subsequent retrieval.

GORAG also introduces edges between newly introduced and earlier label nodes. These bridges support connections between parts of the graph indexed in different rounds. Let $\mathcal{M}^r_y$ denote the keyword neighbors of label y. For a new label y_n and an earlier label y_o, GORAG

reports

$$w^r_{y_n,y_o} = \frac{1}{2}\left(\frac{\sum_{v\in\mathscr{M}^r_{y_n}} w^r_{v,y_n}}{2|\mathscr{M}^r_{y_n}|} + \frac{\sum_{v\in\mathscr{M}^r_{y_o}} w^r_{v,y_o}}{2|\mathscr{M}^r_{y_o}|}\right). \tag{4.4}$$

The formula uses costs incident to both labels. These factors determine the relative cost of traversing a label bridge rather than a keyword association.

Label bridges connect graph regions introduced in different rounds. Unlike keyword edges, which record observed term–class associations, they are created during graph merging.

4.3 Adaptive Candidate Label Retrieval

For a query t, GORAG extracts keywords using the indexing procedure and partitions them according to whether they already occur in $\mathscr{V}^r$. Write these sets as $\mathscr{V}^t_{\text{exist}}$ and $\mathscr{V}^t_{\text{notexist}}$. The existing keywords become the terminals for current retrieval. The unseen keywords are held for online insertion after classification. The partition prevents the retrieval operation from treating an unindexed term as if it already had a known label neighborhood.

Retrieval seeks a low-cost connected structure that contains the mapped terminals. Label nodes encountered in this structure form the candidate set $\widehat{\mathscr{Y}}^r_t$. Connecting several terminals can select intermediate labels and keywords. The graph therefore determines the number of candidate labels through the returned structure, rather than requiring the same number of labels for all queries. A short query and a query containing several mutually distant indexed terms can produce different retrieval structures.

This terminal-connecting objective is related to the Steiner tree problem. In that problem, the required terminals are a subset of graph vertices, and the solution may use additional vertices to connect them. An ordinary minimum spanning tree instead spans all vertices of its input graph. These objects play distinct roles in GORAG's procedure. GORAG uses these intermediate structures to construct a heuristic connection among the query terminals.

4.3.1 Tree and auxiliary-graph operations

GORAG uses a greedy terminal-connecting procedure based on Mehlhorn's algorithm [38]. The mapped keywords serve as terminals. An auxiliary graph H represents connections between terminals, with edge weights given by path costs. A minimum spanning tree of H selects a collection of terminal connections. Expanding these connections into their underlying paths produces the graph structure used for candidate selection. The language model then receives the labels encountered in this structure.

Once the expanded structure S_t^r is obtained, the candidate set is

$$\widehat{\mathcal{Y}}_t^r = \mathcal{V}(S_t^r) \cap \mathcal{Y}^r. \tag{4.5}$$

Representative keywords and label descriptions are added to the classification prompt for the labels selected by this intersection.

4.4 Classification and Online Indexing

GORAG constructs a classification input by concatenating the query text, extracted query keywords, retrieved candidate labels, and representative keywords associated with those labels. With $\mathcal{K}^t$ denoting query keywords and $\mathcal{K}_{\widehat{\mathcal{Y}}_t^r}$ the representative label information, the input is

$$c_t = \text{Concat}\left(t, \mathcal{K}^t, \widehat{\mathcal{Y}}_t^r, \mathcal{K}_{\widehat{\mathcal{Y}}_t^r}\right). \tag{4.6}$$

A language model applies the classification instruction p_{classify} to this input and produces y_t^*. The graph narrows and describes the alternatives; the language model performs the final text classification.

After the prediction, each keyword in $\mathcal{V}_{\text{notexist}}^t$ is inserted into the graph and connected to y_t^*. The update can be written as

$$\mathcal{V}^r \leftarrow \mathcal{V}^r \cup \mathcal{V}_{\text{notexist}}^t, \tag{4.7}$$

$$\mathcal{E}^r \leftarrow \mathcal{E}^r \cup \{(v, y_t^*) : v \in \mathcal{V}_{\text{notexist}}^t\}. \tag{4.8}$$

New associations receive weights through the reported weighting mechanism. Later queries can then map terms that were previously absent from the index. The update changes the retrieval source without fine-tuning the language model's parameters.

The prediction precedes insertion, so a new keyword's association can affect later queries but not the prediction that created it. Incorrect predicted labels can propagate through those associations. The offline ablation suppresses online expansion, measuring its contribution relative to an index that receives no such query-dependent additions.

4.4.1 Computational organization

The graph separates work performed at a label update from work performed for each query. Offline indexing processes the supplied texts, creates or updates keyword associations, generates label information when available, and merges the round's graph. The cross-round label bridges introduce interactions between the newly indexed and earlier label regions. In the source's abstract operation count, construction is expressed as

$$O\big(|\mathcal{Y}^r||\mathcal{Y}^{r-1}| + |\mathcal{V}^r_{\text{new}}||\mathcal{D}^r|\big)\,. \tag{4.9}$$

The two terms count label merging and keyword–text processing. Language-model token costs additionally depend on prompt and output lengths and are measured in the token study.

Each query maps its keywords to the index, retrieves a connecting structure, and constructs a classification prompt. New terms are inserted after prediction, so the index can grow within a round. The prompt contains only the labels selected for the current query.

4.5 Experimental Design

We evaluate GORAG on CAD, WOS, and IFS-Rel. CAD contains social media posts annotated for contextual abuse [39]. WOS contains academic texts associated with research-area labels [40]. IFS-Rel supplies relation-classification examples derived from news text in an incremental few-shot benchmark [36]. These datasets exercise different label vocabularies and text lengths within the dynamic classification setting.

The four-round split introduces 13 labels across CAD's rounds, 133 across WOS, and 64 across IFS-Rel. The test collections contain 817, 9,396, and 2,560 examples, respectively. IFS-Rel uses 40 test examples per label. Its balanced test allocation differs from the changing class frequencies in the other collections.

The label counts in Table 4.1 describe labels introduced in each round. The classifier operates over their cumulative union. Thus the second WOS round adds 53 labels to the 32 labels already introduced, rather than replacing the earlier set by a new 53-label task. This distinction matters when relating the dataset statistics to the size of the classification instruction and to the graph state retained from earlier rounds.

The main few-shot study uses 1, 5, and 10 labeled examples per class. CAD and WOS RAG experiments use Qwen2.5-7B, while IFS-Rel uses GPT-4o. The source reports an Intel Xeon Gold 5220R CPU at 2.20GHz and a single NVIDIA A100-SXM4-40GB GPU for the

Table 4.1: Dataset statistics for the four-round experiments, transposed from the original table. IFS-Rel has 40 test samples per label. Round label counts refer to newly introduced labels.

Statistic	CAD	WOS	IFS-Rel
Average text tokens	154	200	105
Round 1 test texts	240	2,417	640
Round 1 new labels	4	32	16
Round 2 test texts	60	2,842	640
Round 2 new labels	3	53	16
Round 3 test texts	286	2,251	640
Round 3 new labels	3	30	16
Round 4 test texts	231	1,886	640
Round 4 new labels	3	18	16
Total test texts	817	9,396	2,560

experiments. IFS-Rel inference uses the GPT-4o service.

Classification accuracy requires an exact match between the model output and the ground-truth label name or number. An output that cannot be matched to the required class is not counted as correct.

4.5.1 Compared methods and controls

The study compares six baselines. Entailment constructs text–label entailment pairs and fine-tunes a classifier [36]. NaiveRAG retrieves text chunks from the indexed support information. Retrieved context is compressed by LongLLMLingua before classification [41]. GraphRAG, LightRAG, and HippoRAG2 supply graph-based retrieval alternatives [17, 19, 42]. The graph retrieval systems are adapted to return classification evidence.

Entailment uses RoBERTa-large for five training epochs, a batch size of 16, a learning rate of 10^{-6}, and Adam. LongLLMLingua is evaluated with compression rates of 0.75, 0.8, and 0.85, with 0.8 selected in the reported comparison. GraphRAG and LightRAG use local search. The source uses the nano-graphrag implementation for GraphRAG and NaiveRAG and the other baselines' official implementations.

4.6 Classification Results

The tables beginning with Tables 4.2 and 4.5 preserve the WOS and IFS-Rel portions of the main accuracy table. Each dataset is split into one-, five-, and ten-shot tables, with rounds as columns. Each value is a fractional accuracy, and the labels available to the model expand with the round.

Table 4.2: WOS classification accuracy with one support example per label. Qwen2.5-7B is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table.

Method	Round 1	Round 2	Round 3	Round 4
Entailment	0.3695	0.3994	0.4510	0.4030
NaiveRAG	0.3885	0.2267	0.1821	0.1653
LongLLMLingua	0.3806	0.2155	0.1770	0.1468
GraphRAG	0.3852	0.2213	0.1816	0.1641
LightRAG	0.3930	0.2202	0.1743	0.1625
HippoRAG2	0.4133	0.2552	0.2205	0.1906
GORAG	**0.4866**	**0.4790**	**0.4736**	**0.4420**

Table 4.3: WOS classification accuracy with 5 support examples per label. Qwen2.5-7B is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table.

Method	Round 1	Round 2	Round 3	Round 4
Entailment	0.3823	0.4471	0.4857	0.4387
NaiveRAG	0.3904	0.2154	0.1475	0.1556
LongLLMLingua	0.3823	0.2202	0.1567	0.1382
GraphRAG	0.3897	0.2197	0.1770	0.1634
LightRAG	0.3806	0.2216	0.1767	0.1626
HippoRAG2	0.4245	0.2656	0.2210	0.1860
GORAG	**0.4990**	**0.5109**	**0.4996**	**0.4717**

Table 4.4: WOS classification accuracy with 10 support examples per label. Qwen2.5-7B is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table.

Method	Round 1	Round 2	Round 3	Round 4
Entailment	0.4187	0.4222	0.4787	0.4442
NaiveRAG	0.3897	0.2187	0.1799	0.1649
LongLLMLingua	0.3901	0.2198	0.1608	0.1493
GraphRAG	0.3906	0.2219	0.1786	0.1625
LightRAG	0.3815	0.2145	0.1799	0.1632
HippoRAG2	0.4208	0.2666	0.2242	0.1915
GORAG	**0.5134**	**0.5284**	**0.5234**	**0.4929**

Table 4.5: IFS-Rel classification accuracy with one support example per label. GPT-4o is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table.

Method	Round 1	Round 2	Round 3	Round 4
Entailment	0.3391	0.4008	0.3061	0.2971
NaiveRAG	0.7343	0.5394	0.4719	0.5233
LongLLMLingua	0.8750	0.5734	0.4703	0.5819
GraphRAG	0.6406	0.4406	0.3828	0.4297
LightRAG	0.7578	0.4906	0.3802	0.4531
HippoRAG2	0.7313	0.4875	0.4297	0.4984
GORAG	**0.9688**	**0.5938**	**0.4844**	**0.6165**

Table 4.6: IFS-Rel classification accuracy with 5 support examples per label. GPT-4o is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table.

Method	Round 1	Round 2	Round 3	Round 4
Entailment	0.6046	0.5516	0.4703	0.4395
NaiveRAG	0.7344	0.5406	0.4719	0.4938
LongLLMLingua	0.8906	0.5922	0.4741	0.4903
GraphRAG	0.7813	0.4688	0.3734	0.4422
LightRAG	0.8281	0.5155	0.4010	0.4804
HippoRAG2	0.7375	0.5016	0.4328	0.5008
GORAG	**0.9688**	**0.6172**	**0.4948**	**0.4988**

Table 4.7: IFS-Rel classification accuracy with 10 support examples per label. GPT-4o is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table.

Method	Round 1	Round 2	Round 3	Round 4
Entailment	0.5859	0.5828	0.5193	0.4895
NaiveRAG	0.7406	0.5469	0.4688	0.5000
LongLLMLingua	0.9219	0.5806	0.5225	0.5038
GraphRAG	0.8125	0.4719	0.3141	0.3922
LightRAG	0.8594	0.5156	0.4427	0.4610
HippoRAG2	0.7406	0.4938	0.4438	0.4953
GORAG	**0.9688**	**0.6250**	**0.5344**	**0.5469**

On WOS with one support example per label, GORAG reports accuracies of 0.4866, 0.4790, 0.4736, and 0.4420 across the four rounds. Entailment reports 0.3695, 0.3994, 0.4510, and 0.4030 in the same columns. The corresponding GraphRAG values are 0.3852, 0.2213, 0.1816, and 0.1641. GORAG maintains higher accuracy than these baselines as the label set grows.

Additional support examples improve GORAG's WOS accuracy in the main table. At the fourth round, its one-, five-, and ten-shot values are 0.4420, 0.4717, and 0.4929. At the first round they are 0.4866, 0.4990, and 0.5134.

IFS-Rel uses GPT-4o. In its first round, GORAG reports 0.9688 for all three support sizes, while the fourth-round scores are 0.6165, 0.4988, and 0.5469. The five-shot fourth-round value is below the one-shot value, so increasing support size does not produce a monotonic improvement in every setting. LongLLMLingua reports 0.5819, 0.4903, and 0.5038 in these fourth-round columns. HippoRAG2 reports 0.4984, 0.5008, and 0.4953; its five-shot result exceeds GORAG's five-shot result in that column.

CAD uses Qwen2.5-7B. Its GORAG first-round accuracy is 0.6125 in all three support settings. The fourth-round one- and five-shot accuracies are 0.3133 and 0.3146. Accuracy decreases in later rounds as the cumulative label set grows.

WOS improves with additional support in the displayed rounds, whereas IFS-Rel is non-

Table 4.8: Ablation experiments on WOS for GORAG's variants.

Dataset	Model	Round			
		R1	R2	R3	R4
WOS	GORAG *offline&unit*	0.2979	0.2287	0.2645	0.2249
	GORAG *offline*	0.3063	0.2302	0.2455	0.2156
	GORAG *unit*	0.4706	0.4394	0.4407	0.3899
	GORAG *keyword*	0.4746	0.4606	0.4455	0.4030
	GORAG	**0.4862**	**0.4649**	**0.4814**	**0.4210**

monotonic across support sizes and CAD loses accuracy as labels accumulate.

4.7 Ablations and Resource Use

The WOS ablation study uses a one-shot Qwen2.5-7B setting. The offline variant disables online indexing. The unit variant assigns cost one to every graph edge. The offline-and-unit variant removes both features. The keyword variant uses the extracted keywords in place of the complete query text for classification.

In the ablation table, the full method reports 0.4862, 0.4649, 0.4814, and 0.4210 across rounds. The offline variant reports 0.3063, 0.2302, 0.2455, and 0.2156. The unit variant reports 0.4706, 0.4394, 0.4407, and 0.3899. Disabling online insertion causes a larger accuracy drop than replacing weighted edges with unit costs.

The offline-and-unit variant reports 0.2979, 0.2287, 0.2645, and 0.2249. Its third- and fourth-round values exceed those of the offline variant. The effects of removing these components are therefore not strictly additive across rounds. The keyword variant reports 0.4746, 0.4606, 0.4455, and 0.4030, below the full method in each displayed column. The full query provides information useful to the classifier in these experiments even though keywords determine the retrieval terminals.

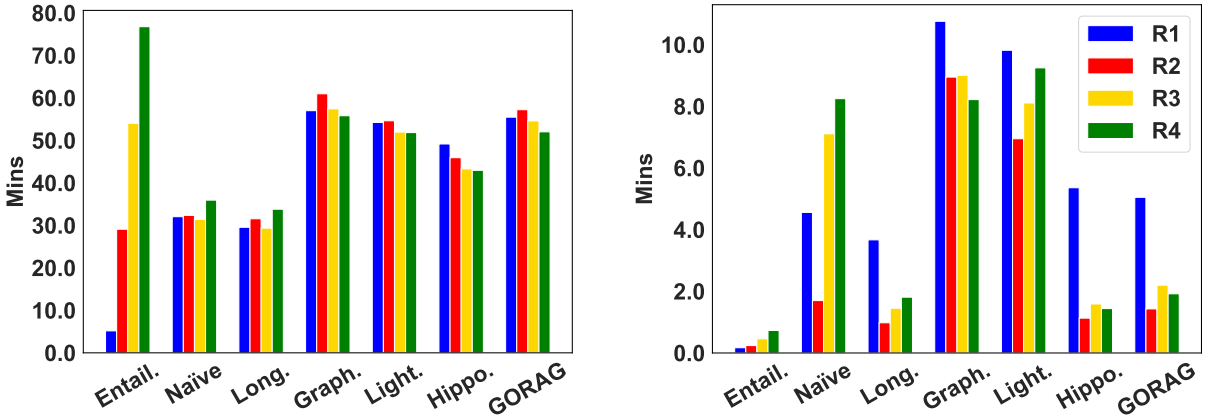
The publication gives different full-method values in the main and ablation WOS tables without explaining their provenance. Component comparisons here use the full-method row from the ablation table.

4.7.1 Prompt tokens and graph growth

The IFS-Rel one-shot token study reports average tokens per query and the corresponding US-dollar costs under the original study's pricing conditions. GORAG uses 557.5 tokens per query and reports a cost of $0.004. The direct GPT-4o comparison uses 634.6 tokens and reports $0.005. LongLLMLingua uses 2,771.3 tokens, NaiveRAG 4,537.4, LightRAG 7,866.3, and

Table 4.9: Token cost on IFS-Rel dataset with 1-shot setting.

Model	AVG Token per Query	AVG Cost per Query (USD)
GPT-4o	634.6	$0.005
LongLLMLingua	2771.3	$0.021
NaiveRAG	4537.4	$0.034
LightRAG	7866.3	$0.059
GraphRAG	9296.3	$0.070
GORAG	**557.5**	**$0.004**



(a) WOS. (b) CAD.

Figure 4.3: Total time in minutes for the one-shot four-round experiments with Qwen2.5-7B. The original R1–R4 legend in panel (b) applies to both panels. Reproduced from Figure 3 of GORAG [2].

GraphRAG 9,296.3. Their reported costs are $0.021, $0.034, $0.059, and $0.070, respectively.

GORAG's candidate-label context uses fewer tokens than the retrieval baselines in this one-shot experiment.

Graph growth is measured separately during the first IFS-Rel round with GPT-4o. The reported index contains 1,644 nodes and 1,771 edges at the 25-percent checkpoint, and 5,299 nodes and 5,503 edges at the final checkpoint. The corresponding reported memory values are 13.3 KB and 46.4 KB. These memory values cover the graph index alone.

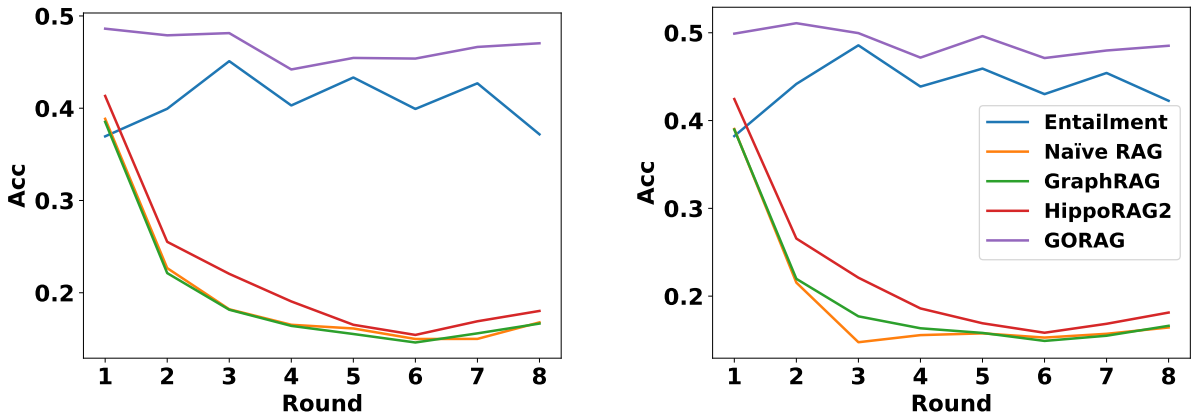
The original runtime study compares total time on WOS and CAD under the one-shot setting, using Qwen2.5-7B. Online insertion adds work after each prediction, while statistical keyword indexing and smaller model inputs can change other parts of the pipeline.

4.8 Zero-shot and Round Generalization

The zero-shot IFS-Rel experiment supplies label names and language-model-generated descriptions without annotated support texts. GORAG extracts keywords from these descriptions to

Table 4.10: 0-shot experiments for GORAG and RAG baselines on IFS-Rel dataset under 0-shot setting.

Model	R1	R2	R3	R4
GPT-4o [43]	0.9219	0.5109	0.4063	0.5936
LongLLMLingua [41]	0.9281	0.5297	0.4234	0.5806
LightRAG [19]	0.6718	0.3047	0.2135	0.0977
GraphRAG [17]	0.5625	0.1875	0.1458	0.0938
GORAG	**0.9328**	**0.5406**	**0.4375**	**0.6063**



(a) One-shot.

(b) Five-shot.

Figure 4.4: WOS classification across the reported round splits. Points 1–4 use the four-round split, 5–6 the six-round split, and 7–8 the eight-round split. The legend in panel (b) applies to both panels. Reproduced from Figure 4 of GORAG [2].

initialize its graph. The experiment consequently tests whether description-derived associations can support candidate selection and subsequent classification.

GORAG reports zero-shot accuracies of 0.9328, 0.5406, 0.4375, and 0.6063. The direct GPT-4o comparison reports 0.9219, 0.5109, 0.4063, and 0.5936. LongLLMLingua reports 0.9281, 0.5297, 0.4234, and 0.5806. The graph-based comparisons in the same table are lower under this initialization protocol. These values support the use of label descriptions as an initial retrieval source for this experiment.

The round-generalization study uses WOS split into four, six, and eight rounds under one- and five-shot settings. The displayed curve combines rounds one to four from the four-round split, rounds five and six from the six-round split, and rounds seven and eight from the eight-round split. These points represent results from different partitions of the label set. They do not form the trajectory of one index updated through eight consecutive rounds.

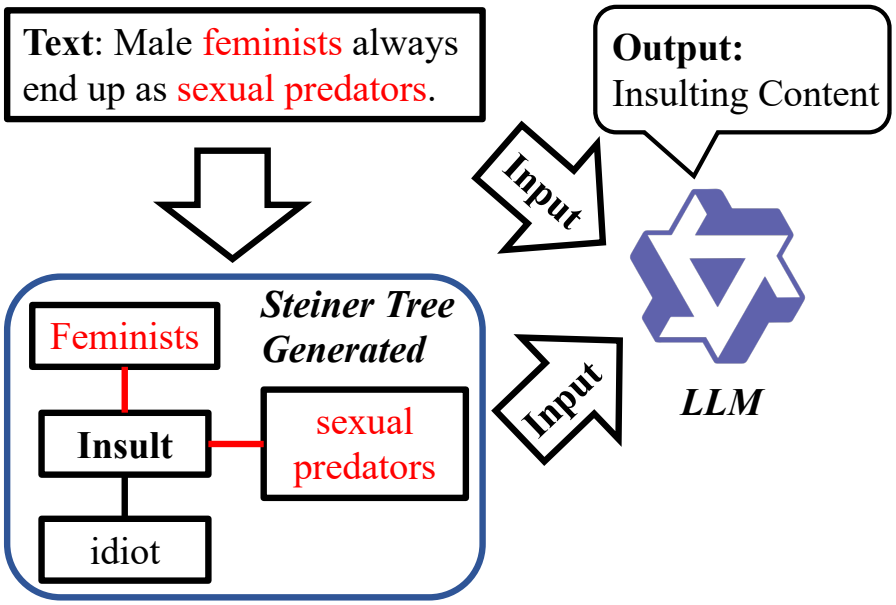


Figure 4.5: A contextual-abuse example and its selected graph connections. Reproduced from Figure 5 of GORAG [2].

4.8.1 A retrieved classification case

Figure 4.5 reproduces a contextual-abuse example from the original study. The query contains the terms highlighted in red. These terms map to keyword nodes, and the selected connecting structure passes through an insult label. The classifier outputs the abuse category shown in the figure. This example exposes the relationship between the extracted query evidence, the intermediate label node, and the final classification.

The insult label connects more than one highlighted query term. Other keyword neighbors remain outside the selected structure, showing how connected retrieval narrows the available label evidence.

4.9 Discussion and Chapter Summary

We proposed GORAG for dynamic few-shot classification. Weighted keyword–label connections determine the candidate set for each query, and the language model classifies the original text using those labels and their descriptions.

The offline ablation shows the contribution of expanding the index beyond the support sample. Because these additions carry predicted labels, the method couples current classification with the evidence available to later queries.

Keyword extraction restricts the query terms available as retrieval terminals. Sparse support data and limited label descriptions constrain initial graph coverage.

GORAG maintains classification evidence as labels and observed vocabulary evolve. Chapter 5 turns to open-ended answers, where retrieval must select connected facts and the passage context needed to explain them.

CHAPTER 5

CONNECTED EVIDENCE SELECTION FOR LANGUAGE MODELS WITH AGRAG

This chapter is based on *AGRAG: Advanced Graph-based Retrieval-Augmented Generation for LLMs*, published at ICDE 2026 [3]. We propose AGRAG to retrieve connected facts and supporting passages for language-model generation. It combines query-conditioned graph influence, semantic edge costs, and neighborhood expansion to collect evidence beyond the initially mapped facts.

5.1 Research Problem and Evidence Requirements

A question can require facts from several parts of a corpus. Independently retrieved passages may repeat one fact or leave the relationships between different facts implicit. A graph records these relationships, but retrieval must still select a subgraph that contains enough information for the answer.

We propose AGRAG to select connected evidence using query relevance and graph influence. Statistical extraction determines the entity vocabulary, semantic costs rank connections, and neighborhood expansion adds facts around the query anchors. A hybrid passage channel supplies context omitted by the graph representation.

Consider a corpus containing statements about an illness, the organ that it affects, and the routes through which it spreads. A query may match facts mentioning the illness and the organ. Connecting those facts can identify a small initial structure, but answering the question may require a neighboring fact that specifies another anatomical component. The original example in Figure 5.2 illustrates this distinction. The example motivates expansion beyond an initial terminal-connecting structure.

The input is a text corpus $\mathcal{T}$ and a natural-language query q. The corpus is divided into chunks $\mathcal{T}_c$. The indexed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ contains entity nodes, passage nodes, and several kinds of edges. Query processing identifies an initial set of relevant triple facts, assigns node influence scores and edge costs, and produces a subgraph for generation. A hybrid retriever supplies additional passages from the same corpus. The output is an answer conditioned on the

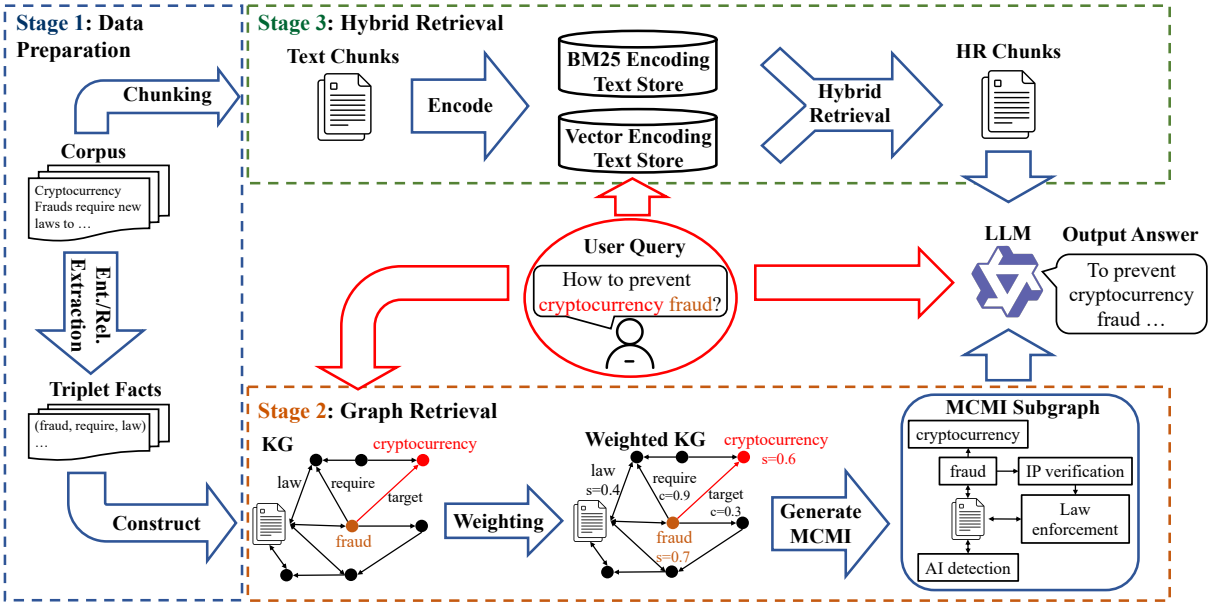


Figure 5.1: The AGRAG pipeline. The original figure separates corpus preparation, query-conditioned graph retrieval, and hybrid passage retrieval. Reproduced from Figure 1 of the source paper [3].

query, a textual serialization of the retrieved graph, and the hybrid passages.

Entity–relation edges represent facts, while passage nodes link those facts to source text. We supply both the selected graph and retrieved passages to the generator so that it can use relational connections alongside their surrounding explanations.

The chapter evaluates the resulting pipeline on tasks with different answer requirements. Fact retrieval emphasizes individual facts. Complex reasoning combines facts. Contextual summarization requires coverage of several evidence items. Creative generation evaluates responses grounded in retrieved material while allowing more flexible output. Text classification supplies a further setting in which graph retrieval identifies candidate labels.

5.2 Framework and Graph Construction

5.2.1 The three-stage pipeline

AGRAG separates data preparation, graph retrieval, and hybrid retrieval. Data preparation builds the graph and the passage indexes. Graph retrieval conditions the graph on a query through semantic mapping and influence scores, then selects a connected evidence subgraph. Hybrid retrieval combines dense passage similarity with BM25. Figure 5.1 reproduces the original architecture.

Corpus indexing performs chunking, entity and relation extraction, passage linking, and

dense and sparse indexing. Query processing maps the question to facts, computes influence scores and edge costs, expands the evidence subgraph, and retrieves hybrid passages.

The statistical component replaces the entity extraction step used in the compared graph pipelines. Language models remain involved in relation extraction, filtering raw mapped triples, and generating answers. The entity-extraction ablation changes the nodes supplied to subsequent relation extraction and retrieval.

5.2.2 Chunking and statistical entity extraction

The corpus is split into overlapping text chunks,

$$\mathscr{T}_c = \text{Split}(\mathscr{T}, l_t, l_o), \tag{5.1}$$

where l_t is the chunk length and l_o is the overlap. The reported main setting uses 256 tokens per chunk and an overlap of 32 tokens. Overlap allows adjacent chunks to share text at their boundaries; the graph subsequently connects chunks through the entities they contain.

AGRAG considers words and phrases up to a maximum n-gram length b. For a candidate term v in chunk t, the extraction rule modifies TF–IDF [37] to define the score

$$\text{ER}(v,t) = \frac{\text{count}(v,t)}{|t| \log\big(|\mathscr{T}_c| + 1\big)} \log \frac{|\mathscr{T}_c| + 1}{|\{t_j \in \mathscr{T}_c \,:\, v \in t_j\}| + 1}. \tag{5.2}$$

The first factor depends on within-chunk frequency and length. The second distinguishes terms according to their distribution across chunks. The extracted set is

$$\mathscr{V}_t = \{v \,:\, \text{ER}(v,t) > \tau\}, \tag{5.3}$$

with maximum n-gram length b and threshold τ controlling candidate boundaries and selection. The main paper reports $b = 3$ and $\tau = 0.5$.

Statistical extraction selects strings present in the corpus. Term frequency does not directly measure factual importance, so a generic expression can score differently from a rare but relevant phrase. We examine the n-gram boundary and threshold in the sensitivity study.

Shared entities connect passages in the graph. These links are useful when the extracted strings identify compatible entities across their occurrences; ambiguous mentions can instead create misleading connections.

5.2.3 Relations, passage nodes, and synonyms

For each chunk, the language model receives the selected entity set and the chunk text, and extracts relations,

$$\mathscr{E}_t = \mathrm{LLM}(\mathscr{V}_t, t, p_{\mathrm{re}}). \tag{5.4}$$

The prompt p_{re} asks for relations between the extracted entities in the chunk. The resulting entity–relation–entity triples provide the graph's factual edges. Statistical extraction selects entity candidates from observed terms. The model then extracts relations conditioned on those candidates and their context.

Chunks are also inserted as passage nodes. A *Contains* edge connects a passage to an entity extracted from it. These links preserve the association between the symbolic representation and the source chunk. They also allow paths to cross passages through shared entities. A passage contributes contextual evidence to graph retrieval without being reduced to a single isolated fact.

The graph includes synonym edges between entity nodes whose embedding similarity exceeds a threshold φ. Such an edge expresses an embedding-based correspondence between strings. It permits retrieval to connect different expressions that would otherwise form separate nodes. Extracted relations come from the language model's reading of a chunk, whereas synonym edges come from a similarity test.

AGRAG additionally introduces a pseudo node linked to passage nodes. The reported cost of each pseudo edge is 10. This connection gives the graph a route between otherwise disconnected portions while penalizing its use relative to semantically relevant factual edges. Pseudo edges provide connectivity between passages without asserting a factual relation.

5.3 Query Mapping and Influence-aware Retrieval

5.3.1 Mapping a query to triple facts

At query time, AGRAG first identifies facts that are semantically close to the query. A text encoder $\mathscr{M}$ maps the query and a textual rendering of each triple to vectors. With $\mathbf{q} = \mathscr{M}(q)$ and $\mathbf{f} = \mathscr{M}(f)$, the mapping score is

$$\mathrm{MS}(\mathbf{q}, \mathbf{f}) = \cos(\mathbf{q}, \mathbf{f}). \tag{5.5}$$

The highest-scoring k_a facts form the raw mapped set $\mathscr{F}_{\text{raw}}$. The main setting uses $k_a = 5$. This stage locates candidate facts in the indexed graph before graph propagation begins.

A model-based filtering step then evaluates the raw facts with respect to the query,

$$\mathscr{F} = \text{LLM}(\mathscr{F}_{\text{raw}}, q, p_{\text{tf}}). \tag{5.6}$$

The filtered set $\mathscr{F}$ supplies query anchors. Dense mapping produces the candidate list, and the model checks its relevance to the question. The filtering ablation removes this second step.

Facts around the anchors can contribute to an answer even when their direct query similarity is lower. We propagate influence from the mapped facts to score this surrounding evidence and use semantic edge costs when selecting its connections.

5.3.2 Node influence and edge costs

AGRAG uses a personalized PageRank-style propagation [44] to assign query-dependent influence scores,

$$\mathbf{s} = \lim_{i \to \infty} \mathbf{s}^{(i)}, \qquad \mathbf{s}^{(i)} = (1 - d)\mathbf{p} + d\mathbf{P}\mathbf{s}^{(i-1)}. \tag{5.7}$$

Here $\mathbf{p}$ is determined by nodes associated with the filtered mapped facts, and $\mathbf{P}$ is the graph propagation matrix specified in the source. The reported setting uses $d = 0.5$ and stops the iteration when the change is below 10^{-7}. The score of a node v is $s_v = \mathbf{s}[v]$. Passage-node scores are multiplied by 0.05 to balance their contribution relative to entity nodes.

The personalization vector assigns one to nodes participating in the mapped facts and zero to the other nodes. The graph operator is specified by

$$\mathbf{P}[u][v] = \begin{cases} 1/|\mathscr{N}(u)|, & (u, v) \in \mathscr{E}, \\ 1/|\mathscr{V}|, & \text{otherwise.} \end{cases} \tag{5.8}$$

Nonadjacent node pairs receive the constant term in this operator. The recurrence uses this matrix with the damping value in Equation 5.7, followed by passage-node scaling.

We compute node influence on the indexed graph before expansion. These scores remain assigned to the nodes while the retrieval procedure selects which nodes and edges to include.

Direct similarity scores a triple against the question. Influence also reflects its connections to the query anchors, allowing evidence connected to several mapped facts to receive retrieval weight.

For an edge $e = (u, v)$ associated with fact f_e, AGRAG defines

$$c_e = \frac{1 - \mathrm{MS}(\mathbf{q}, \mathbf{f}_e)}{2}. \tag{5.9}$$

Higher similarity gives a lower factual edge cost. Subgraph selection combines these edge costs with node influence scores.

Pseudo edges retain their assigned penalty, providing an expensive route when factual connections alone cannot connect the anchors.

5.4 MCMI Formulation and Evidence Expansion

Let $\mathscr{V}_{\text{term}}$ contain the nodes participating in the mapped facts. A feasible subgraph contains these terminals and connects each terminal to a designated root terminal. The main paper defines the Minimum Cost Maximum Influence (MCMI) objective as

$$r(\mathscr{G}') = \sum_{e=(u,v)\in\mathscr{E}'} \frac{c_e}{s_u + s_v}. \tag{5.10}$$

The objective scales each included edge cost by the influence of its endpoints. We construct an initial terminal-connecting tree, then expand its neighborhood using marginal cost-to-score comparisons.

The initial structure connects the query-mapped information. The source constructs an auxiliary graph on the terminals, uses shortest-path costs between them, selects a tree on that auxiliary structure, and expands its edges back to paths in the evidence graph. This stage supplies connections among anchors instead of returning the anchors as an unordered set. The detailed theorem in the main paper assigns its 2-approximation statement to this MCST construction stage.

Subsequent expansion examines neighboring candidates. A candidate node v connected by an edge of cost $c_{(\mathrm{MCMI},v)}$ is considered through the ratio

$$\frac{c_{(\mathrm{MCMI},v)}}{s_v}. \tag{5.11}$$

The procedure selects the lowest such ratio and compares it with the current subgraph's reported cost-score quantity. The acceptance condition is

$$\frac{c_{(\mathrm{MCMI},v)}}{s_v} < r(\mathscr{G}_{\mathrm{MCMI}}). \tag{5.12}$$

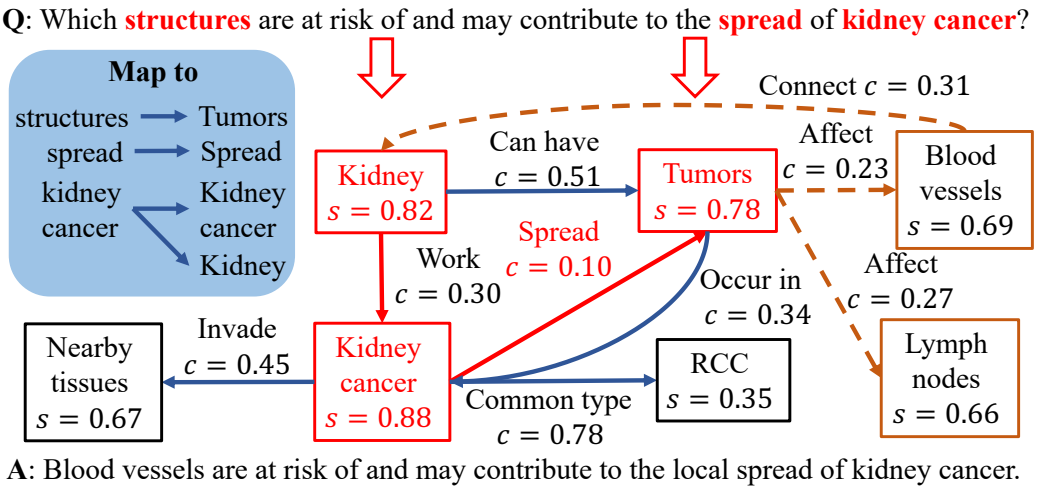


Figure 5.2: The original AGRAG evidence-expansion example. Red triples are query-mapped facts; orange triples show additional neighboring evidence in the illustrated MCMI structure. Reproduced from Figure 2 of the source paper [3].

The candidate and its connecting edge are added when this condition holds. The subgraph quantity is recalculated and the procedure considers the updated neighborhood. Expansion stops when the lowest candidate ratio is greater than or equal to the current subgraph quantity. The comparison uses the node score of the prospective addition, whereas Equation 5.10 uses the two endpoints of each included edge. The candidate ratio therefore ranks possible additions using their assigned node scores, while the current subgraph is evaluated by Equation 5.10.

Expansion adds neighboring facts that can supply answer components absent from the initial tree. The resulting structure can contain cycles, as shown in the MCMI example, and its selected relations are serialized for generation.

5.4.1 The original connected-evidence example

The running example in Figure 5.2 makes the difference between connecting anchors and collecting an answer visible. Its mapped triples include *(Kidney, Work, Kidney cancer)* and *(Kidney, Spread, Tumors)*, using the relation strings printed in the original illustration. Those triples locate the query's subject region. The expanded structure additionally contains relations involving blood vessels, neighboring tissues, and other entities shown in the graph. The original illustrated answer identifies blood vessels as the requested component.

In this example, the added relation supplies an answer component absent from the mapped facts. Influence scores and edge costs determine whether expansion includes that neighboring evidence.

The theoretical claims have distinct scopes. The main paper reports NP-hardness for the

stated subgraph optimization problem and a 2-approximation result for the initial terminal-connecting stage. The expansion procedure adds further evidence after that stage. The ablation replaces the expanded subgraph with the initial Steiner-tree representation.

The graph returned by retrieval is serialized as $g = \mathrm{StrConcat}(\mathscr{V}_{\mathrm{MCMI}}, \mathscr{E}_{\mathrm{MCMI}})$. This representation contains the selected entities and relations. It makes the selected relationships available within the model's input sequence.

5.5 Hybrid Retrieval and Generation

A graph index captures extracted relationships, whereas many details remain in the original text. AGRAG therefore also retrieves chunks through a hybrid score,

$$\mathrm{HS}(q, t) = \frac{\mathrm{MS}(\mathbf{q}, \mathbf{t}) + \mathrm{BM25}(q, t)}{2}, \tag{5.13}$$

where $\mathbf{t}$ is the dense representation of chunk t. The top k_r chunks according to this score form $\mathscr{T}_{\mathrm{HR}}$. The source reports $k_r = 5$ in the main setting.

Dense retrieval scores semantic similarity, while BM25 scores lexical overlap [15]. We average them and combine the selected passages with the graph serialization.

The final answer is generated through

$$A_q = \mathrm{LLM}(q, g, \mathscr{T}_{\mathrm{HR}}, p_{\mathrm{gen}}). \tag{5.14}$$

The generator receives the question, selected graph facts, and hybrid passages. Graph-only and hybrid-only ablations test the contributions of the two evidence channels.

For classification, the retrieval interface is adapted to return candidate labels associated with passages. One training passage per label supplies the indexed corpus. The generator selects a label from the retrieved candidate set. This classification index is fixed, whereas GORAG updates its index online over label rounds in Chapter 4.

5.6 Computational Properties

AGRAG's work is distributed across corpus indexing and query processing. Statistical extraction and passage-link construction operate over the corpus. Query processing includes triple mapping, filtering, graph propagation, subgraph selection, hybrid retrieval, and generation.

Language-model calls contribute to relation extraction during indexing and to filtering and answer generation online.

The source's graph-operation analysis assigns $O(|\mathscr{V}||\mathscr{T}_c|)$ time to entity extraction and passage association. Graph propagation is expressed in terms of node and edge counts, and semantic edge weighting visits the indexed edges. The Mehlhorn-based initial tree stage [38] is reported with $O(|\mathscr{E}| + |\mathscr{V}| \log |\mathscr{V}|)$ complexity. Expansion additionally depends on the selected subgraph and its neighborhood. These expressions characterize graph operations; the measured end-to-end results include model inference under the experimental configuration.

A fixed corpus index can be reused across queries. Mapping, filtering, graph selection, and generation recur for each question, so index preparation can be amortized while query latency still depends on graph processing and generated output length.

The index stores entity strings, relations, embeddings, passage associations, and graph adjacency. Its memory use depends on their representations. The efficiency experiments measure time and tokens.

5.7 Experimental Evaluation

5.7.1 Tasks, corpora, and model settings

We evaluate AGRAG on Novel and Medical from GraphRAG-bench [45], together with WOS [40] and IFS-Rel [36] for classification. Novel contains 5,140 chunks, 63,887 graph entities, and 163,169 graph relations. Medical contains 1,030 chunks, 12,426 entities, and 52,054 relations. The reported classification indexes contain 24 chunks for WOS and 11 for IFS-Rel, with 9,396 and 2,560 evaluated texts and 133 and 64 labels, respectively.

The Novel evaluation contains 67 creative-generation queries, 362 contextual-summarization queries, 610 complex-reasoning queries, and 971 fact-retrieval queries. The corresponding Medical counts are 166, 289, 509, and 1,098.

The main backbone is Qwen2.5-14B-Instruct [46], deployed through vLLM [47]. The machine has an Intel Xeon Gold 5220R CPU and one NVIDIA A100 GPU with 80 GB memory. Contriever [48] supplies dense representations for AGRAG and HippoRAG2; the other baselines use bge-large-en-v1.5 according to the reported configuration. Qwen3-Reranker-0.6B performs raw-triplet filtering. The retrieval top-k setting is 5; AGRAG's raw mapped triple count and hybrid chunk count are also 5.

GraphRAG uses local search. LightRAG uses hybrid search for the QA tasks, with its

classification-specific configuration retained where applicable. The comparison includes NaiveRAG, LongLLMLingua [41], RAPTOR [49], SiReRAG [50], G-Retriever [51], MIXRAG [52], GraphRAG [17], LightRAG [19], HippoRAG2 [42], and Fast-GraphRAG [53]. These methods represent passage retrieval, compression, hierarchical organization, connected subgraph selection, neighborhood retrieval, and graph propagation.

5.7.2 Metric interpretation

Classification accuracy is the proportion of predicted labels matching gold labels. Classification recall is macro recall. For the other tasks, the reported accuracy is a model-assessed score with values 0, 0.5, or 1 for inaccurate, partially accurate, or accurate answers. These quantities appear under the same ACC heading in the paper but evaluate different outputs.

Complex reasoning and fact retrieval additionally report the source's ROUGE-L measure [54]. Contextual summarization and creative generation report coverage, the fraction of required reference evidence represented in the answer. Creative generation also reports faithfulness, the fraction of answer claims supported by retrieved context. The evaluation uses the RAGAS framework [55, 56] and the task-specific protocol. Coverage concerns inclusion of required information, whereas faithfulness concerns support for generated claims.

For generated answer sequence G and reference sequence Y, the source defines its reported lexical score as

$$\mathrm{ROG}(G, Y) = \frac{\mathrm{LCS}(G, Y)}{|G|}. \tag{5.15}$$

The denominator is the generated answer length. This is the source's reported ROUGE-L quantity, rather than an F1 combination of longest-common-subsequence precision and recall.

Let E denote the reference evidence items and let $M(e, R)$ indicate whether response R covers item e. Coverage is

$$\mathrm{COV}(R, E) = \frac{|\{e \in E : M(e, R)\}|}{|E|}. \tag{5.16}$$

For the answer's claim set A and retrieved context C, let $S(a, C)$ indicate that claim a is supported. Faithfulness is

$$\mathrm{FS}(A, C) = \frac{|\{a \in A : S(a, C)\}|}{|A|}. \tag{5.17}$$

Coverage measures required reference information, while faithfulness measures support for the claims actually generated. Omitting a required fact lowers coverage; adding an unsupported claim lowers faithfulness. These judgments use the model-based assessment procedure.

Table 5.1: Creative generation with Qwen2.5-14B-Instruct. Results from the AGRAG main experiment [3].

Method	Novel FS	Novel ACC	Novel COV	Medical FS	Medical ACC	Medical COV
NaiveRAG	0.205	0.307	0.384	0.176	0.577	0.554
L.LLMLingua	0.213	0.316	0.377	0.225	0.575	0.496
RAPTOR	0.365	0.333	0.373	0.314	0.526	0.553
SiReRAG	0.324	0.328	0.365	0.295	0.544	0.456
G-retriever	0.294	0.167	0.200	0.169	0.371	0.468
MIXRAG	0.200	0.286	0.337	0.255	0.351	0.306
GraphRAG	0.427	0.310	0.339	0.345	0.475	0.325
LightRAG	0.201	0.314	0.329	0.189	0.567	0.549
HippoRAG2	0.496	0.295	0.325	0.347	0.563	0.451
F.GraphRAG	0.286	0.287	0.376	0.217	0.500	0.396
AGRAG	**0.513**	**0.339**	**0.386**	**0.356**	**0.598**	**0.589**

Table 5.2: Contextual summarization with Qwen2.5-14B-Instruct. Results from the AGRAG main experiment [3].

Method	Novel COV	Novel ACC	Medical COV	Medical ACC
NaiveRAG	0.745	0.621	0.677	0.576
L.LLMLingua	0.687	0.627	0.657	0.583
RAPTOR	0.760	0.626	0.633	0.583
SiReRAG	0.754	0.617	0.592	0.560
G-retriever	0.594	0.404	0.302	0.395
MIXRAG	0.695	0.538	0.409	0.464
GraphRAG	0.758	0.620	0.625	0.506
LightRAG	0.753	0.628	0.625	0.585
HippoRAG2	0.735	0.586	0.496	0.560
F.GraphRAG	0.706	0.528	0.507	0.587
AGRAG	**0.778**	**0.643**	**0.690**	**0.604**

Table 5.3: Complex reasoning with Qwen2.5-14B-Instruct. Results from the AGRAG main experiment [3].

Method	Novel ROG	Novel ACC	Medical ROG	Medical ACC
NaiveRAG	0.120	0.447	0.117	0.635
L.LLMLingua	0.092	0.434	0.080	0.633
RAPTOR	0.290	0.460	0.185	0.649
SiReRAG	0.307	0.459	0.269	0.646
G-retriever	0.146	0.441	0.102	0.396
MIXRAG	0.109	0.386	0.156	0.434
GraphRAG	0.246	0.449	0.250	0.606
LightRAG	0.223	0.448	0.255	0.637
HippoRAG2	0.314	0.453	0.332	0.622
F.GraphRAG	0.237	0.371	0.262	0.643
AGRAG	**0.319**	**0.467**	**0.343**	**0.665**

Table 5.4: Fact retrieval with Qwen2.5-14B-Instruct. Results from the AGRAG main experiment [3].

Method	Novel ROG	Novel ACC	Medical ROG	Medical ACC
NaiveRAG	0.117	0.431	0.182	0.604
L.LLMLingua	0.115	0.420	0.105	0.591
RAPTOR	0.133	0.465	0.132	0.609
SiReRAG	0.311	0.451	0.349	0.558
G-retriever	0.199	0.209	0.148	0.326
MIXRAG	0.102	0.366	0.160	0.463
GraphRAG	0.240	0.445	0.217	0.521
LightRAG	0.253	0.460	0.283	0.607
HippoRAG2	0.310	0.470	0.343	0.598
F.GraphRAG	0.318	0.439	0.352	0.610
AGRAG	**0.324**	**0.490**	**0.357**	**0.628**

Table 5.5: Text classification with Qwen2.5-14B-Instruct. Results from the AGRAG main experiment [3].

Method	WOS ACC	WOS REC	IFS-Rel ACC	IFS-Rel REC
NaiveRAG	0.237	0.257	0.351	0.197
L.LLMLingua	0.274	0.231	0.378	0.220
RAPTOR	0.409	0.290	0.293	0.212
SiReRAG	0.430	0.294	0.325	0.220
G-retriever	0.188	0.129	0.311	0.185
MIXRAG	0.244	0.235	0.362	0.203
GraphRAG	0.253	0.201	0.279	0.202
LightRAG	0.273	0.224	0.265	0.220
HippoRAG2	0.424	0.270	0.389	0.246
F.GraphRAG	0.283	0.245	0.307	0.221
AGRAG	**0.438**	**0.307**	**0.403**	**0.251**

5.7.3 Answer quality across tasks

The main results are split by task in Tables 5.1–5.5. On Novel creative generation, AGRAG reports faithfulness 0.513, accuracy 0.339, and coverage 0.386. HippoRAG2 reports faithfulness 0.496, while RAPTOR reports accuracy 0.333 and NaiveRAG coverage 0.384.

On Medical creative generation, AGRAG reports faithfulness 0.356, accuracy 0.598, and coverage 0.589. NaiveRAG has accuracy 0.577 and coverage 0.554. HippoRAG2 has faithfulness 0.347.

For contextual summarization, AGRAG obtains coverage and accuracy of 0.778 and 0.643 on Novel, and 0.690 and 0.604 on Medical. Graph facts and passage context jointly supply information distributed across the corpus.

For complex reasoning, the reported ROUGE-L and accuracy values are 0.319 and 0.467 on Novel and 0.343 and 0.665 on Medical. For fact retrieval they are 0.324 and 0.490 on Novel and 0.357 and 0.628 on Medical. The tasks use different query collections and answer formats.

Classification accuracy and recall are 0.438 and 0.307 on WOS and 0.403 and 0.251 on IFS-Rel. SiReRAG reports WOS accuracy 0.430 and recall 0.294, while HippoRAG2 reports IFS-Rel accuracy 0.389 and recall 0.246. AGRAG's classifier receives graph-derived candidate labels under the source protocol.

Table 5.6: Indexing / querying time in minutes. Qwen2.5-14B-Instruct; original paired measurements are preserved.

Method	Novel	Medical	WOS	IFS-Rel
NaiveRAG	13 / 371	7 / 245	2 / 292	2 / 225
L.LLMLingua	51 / 360	37 / 305	4 / 358	3 / 315
RAPTOR	464 / 250	117 / 319	30 / 162	24 / 121
SiReRAG	85 / 233	40 / 207	26 / 139	18 / 117
G-Retriever	19 / 119	12 / 145	8 / 158	3 / 164
MIXRAG	17 / 388	11 / 453	5 / 257	2 / 193
GraphRAG	347 / 205	194 / 158	85 / 240	64 / 217
LightRAG	166 / 221	116 / 164	42 / 211	31 / 197
HippoRAG2	95 / 134	46 / 130	19 / 140	15 / 127
F.GraphRAG	106 / 198	48 / 173	29 / 186	27 / 164
AGRAG	65 / 142	16 / 138	16 / 132	14 / 124

5.7.4 Indexing and querying costs

Table 5.6 separates indexing time, querying time, input tokens, and output tokens. For Novel, AGRAG reports 65 minutes for indexing and 142 minutes for querying, compared with HippoRAG2's 95 and 134 minutes. For Medical the corresponding values are 16 and 138 minutes for AGRAG and 46 and 130 minutes for HippoRAG2. This comparison shows lower index preparation time alongside a higher reported query total for AGRAG.

The table also contains methods with lower costs in particular columns. NaiveRAG has lower indexing times, while G-Retriever has a shorter Novel querying total.

On WOS and IFS-Rel, AGRAG reports querying times of 132 and 124 minutes and input token counts of 3,970 and 1,890 per query. The corresponding HippoRAG2 values are 140 and 127 minutes and 5,066 and 2,440 tokens. Output lengths differ between methods.

Table 5.7: Input / output tokens per query. Same source experiment as the preceding time table.

Method	Novel	Medical	WOS	IFS-Rel
NaiveRAG	9,437 / 1,708	7,473 / 379	7,292 / 29	4,521 / 16
L.LLMLingua	8,466 / 1,750	5,686 / 390	5,861 / 31	2,757 / 14
RAPTOR	7,243 / 777	4,817 / 695	6,007 / 41	3,807 / 33
SiReRAG	7,759 / 800	4,918 / 701	6,012 / 47	3,821 / 36
G-Retriever	6,344 / 116	6,290 / 157	6,762 / 12	4,002 / 21
MIXRAG	6,083 / 237	4,238 / 230	4,193 / 31	3,597 / 29
GraphRAG	12,686 / 2,109	8,385 / 1,017	11,809 / 351	9,009 / 287
LightRAG	13,145 / 1,233	9,535 / 426	11,510 / 227	7,667 / 199
HippoRAG2	8,138 / 1,073	6,264 / 595	5,066 / 93	2,440 / 102
F.GraphRAG	7,490 / 1,252	6,475 / 384	4,957 / 35	2,217 / 20
AGRAG	6,487 / 1,140	5,145 / 413	3,970 / 101	1,890 / 93

Table 5.8: Creative generation ablation with Qwen2.5-14B-Instruct. Values retain the original source ablation precision.

Method	Novel FS	Novel ACC	Novel COV	Medical FS	Medical ACC	Medical COV
AGRAG w. HR Only	0.491	0.322	0.378	0.303	0.526	0.549
AGRAG w. LLM ER	0.428	0.302	0.365	0.352	0.526	0.587
AGRAG w.o. HR	0.519	0.306	0.343	0.342	0.521	0.570
AGRAG w.o. MCMI	0.488	0.332	0.358	0.351	0.530	0.575
AGRAG w.o. Cle.	0.478	0.311	0.361	0.336	0.517	0.569
AGRAG	0.513	0.339	0.386	0.356	0.598	0.589

Table 5.9: Contextual summarization ablation with Qwen2.5-14B-Instruct. Values retain the original source ablation precision.

Method	Novel COV	Novel ACC	Medical COV	Medical ACC
AGRAG w. HR Only	0.739	0.522	0.535	0.583
AGRAG w. LLM ER	0.755	0.552	0.600	0.566
AGRAG w.o. HR	0.677	0.534	0.420	0.578
AGRAG w.o. MCMI	0.748	0.592	0.600	0.588
AGRAG w.o. Cle.	0.712	0.597	0.616	0.577
AGRAG	0.778	0.643	0.690	0.604

Table 5.10: Complex reasoning ablation with Qwen2.5-14B-Instruct. Values retain the original source ablation precision.

Method	Novel ROG	Novel ACC	Medical ROG	Medical ACC
AGRAG w. HR Only	0.303	0.453	0.334	0.636
AGRAG w. LLM ER	0.320	0.465	0.333	0.624
AGRAG w.o. HR	0.318	0.434	0.324	0.598
AGRAG w.o. MCMI	0.324	0.465	0.329	0.633
AGRAG w.o. Cle.	0.301	0.450	0.334	0.628
AGRAG	0.319	0.467	0.343	0.665

Table 5.11: Fact retrieval ablation with Qwen2.5-14B-Instruct. Values retain the original source ablation precision.

Method	Novel ROG	Novel ACC	Medical ROG	Medical ACC
AGRAG w. HR Only	0.323	0.487	0.355	0.619
AGRAG w. LLM ER	0.322	0.480	0.353	0.613
AGRAG w.o. HR	0.314	0.487	0.344	0.615
AGRAG w.o. MCMI	0.322	0.470	0.359	0.621
AGRAG w.o. Cle.	0.313	0.478	0.354	0.620
AGRAG	0.324	0.490	0.357	0.628

Table 5.12: Ablation time and token costs. Time is in minutes. Paired token columns retain the source I/G headings; its caption describes input/output token counts.

Method	Novel time I/Q	Novel tokens I/G	Medical time I/Q	Medical tokens I/G
AGRAG w. HR Only	11.7 / 137.9	5,117 / 193	11.4 / 117.9	5,215 / 199
AGRAG w. LLM ER	83.3 / 161.2	8,030 / 887	31.9 / 148.8	5,457 / 414
AGRAG w.o. HR	61.7 / 138.9	6,986 / 741	9.4 / 114.4	4,820 / 315
AGRAG w.o. MCMI	63.9 / 130.4	6,710 / 757	9.5 / 125.0	5,230 / 331
AGRAG w.o. Cle.	65.6 / 140.1	6,988 / 154	9.1 / 142.9	5,252 / 158
AGRAG	65.6 / 142.0	6,988 / 639	9.1 / 145.3	5,252 / 306

5.7.5 Component ablations

The ablation compares hybrid retrieval alone, language-model entity extraction, graph retrieval without hybrid context, an initial tree without MCMI expansion, and retrieval without the raw-triplet filtering step.

The hybrid-only variant has Novel creative-generation faithfulness 0.491, accuracy 0.322, and coverage 0.378. Full AGRAG reports 0.513, 0.339, and 0.386. For Novel contextual summarization, hybrid-only coverage and accuracy are 0.739 and 0.522, compared with 0.778 and 0.643 for the complete system. Adding graph retrieval improves creative-generation scores and summarization accuracy.

Removing hybrid retrieval produces a different pattern. Novel creative-generation faithfulness is 0.519, exceeding the complete system's 0.513, while accuracy and coverage decrease to 0.306 and 0.343. Medical contextual-summarization coverage decreases from 0.690 to 0.420. The graph-only variant can produce well-grounded claims while supplying less of the required information.

Replacing statistical entity extraction with model-based extraction gives Novel creative-generation faithfulness 0.428 and accuracy 0.302, with indexing time 83.3 minutes in the ablation setting. The complete configuration reports 65.6 minutes in that table. This experiment changes entity extraction while retaining the rest of the method.

The variant without MCMI expansion uses a Steiner-tree representation. Its Novel contextual-summarization accuracy is 0.592, compared with 0.643 for full AGRAG, and its Medical creative-generation accuracy is 0.530, compared with 0.598. Some fact-retrieval metrics are close or favor the tree variant. Expansion improves these summarization and creative-generation scores, while the initial tree remains competitive on fact retrieval.

Removing raw-triplet filtering gives Novel contextual-summarization coverage 0.712 and accuracy 0.597. These are below 0.778 and 0.643 for full AGRAG. Filtering the mapped triples improves coverage and accuracy.

The main efficiency and ablation tables report different Medical indexing and querying totals. Component comparisons use the measurements in the ablation table.

5.7.6 Additional resource setting and parameter study

The low-resource experiment uses Qwen3-4B-Instruct-2507 on two NVIDIA 2080 Ti GPUs. Both the backbone and accelerators differ from the main experiment, so their individual effects are not isolated.

Table 5.13: Creative generation in the low-resource setting. Qwen3-4B-Instruct-2507 on two 2080 Ti GPUs; values and emphasis preserved.

Method	Novel FS	Novel ACC	Novel COV	Medical FS	Medical ACC	Medical COV
SiReRAG-L	0.324	0.412	0.299	0.421	0.758	0.475
HippoRAG2-L	0.433	0.481	0.335	0.425	0.755	0.433
AGRAG-L	**0.441**	**0.519**	**0.354**	**0.436**	**0.777**	**0.490**

Table 5.14: Contextual summarization in the low-resource setting. Qwen3-4B-Instruct-2507 on two 2080 Ti GPUs; values and emphasis preserved.

Method	Novel COV	Novel ACC	Medical COV	Medical ACC
SiReRAG-L	0.647	0.673	0.536	0.738
HippoRAG2-L	0.696	0.657	0.522	0.731
AGRAG-L	**0.745**	**0.706**	**0.545**	**0.751**

Table 5.15: Complex reasoning in the low-resource setting. Qwen3-4B-Instruct-2507 on two 2080 Ti GPUs; values and emphasis preserved.

Method	Novel ROG	Novel ACC	Medical ROG	Medical ACC
SiReRAG-L	0.260	0.426	0.308	0.698
HippoRAG2-L	0.280	0.507	0.298	0.681
AGRAG-L	**0.293**	**0.552**	**0.312**	**0.739**

Table 5.16: Fact retrieval in the low-resource setting. Qwen3-4B-Instruct-2507 on two 2080 Ti GPUs; values and emphasis preserved.

Method	Novel ROG	Novel ACC	Medical ROG	Medical ACC
SiReRAG-L	0.271	0.416	0.343	0.661
HippoRAG2-L	0.288	0.513	0.338	0.654
AGRAG-L	**0.297**	**0.560**	**0.351**	**0.701**

Table 5.17: Novel low-resource efficiency. Qwen3-4B-Instruct-2507 on two 2080 Ti GPUs.

Method	Index min	Query min	Input tokens	Output tokens
SiReRAG-L	52.0	183.5	7,898	**1,003**
HippoRAG2-L	55.7	**131.2**	7,964	1,354
AGRAG-L	**44.7**	132.1	**6,625**	1,172

Table 5.18: Medical low-resource efficiency. Qwen3-4B-Instruct-2507 on two 2080 Ti GPUs.

Method	Index min	Query min	Input tokens	Output tokens
SiReRAG-L	18.1	182.3	**5,113**	827
HippoRAG2-L	21.8	**162.4**	5,577	584
AGRAG-L	**14.4**	167.3	5,282	**534**

In that setting, AGRAG-L reports Novel creative-generation accuracy 0.519 and contextual-summarization accuracy 0.706. Its Novel index/query totals are 44.7 and 132.1 minutes. Medical totals are 14.4 and 167.3 minutes. HippoRAG2-L has shorter query totals in both corpora, while AGRAG-L has shorter indexing totals.

The parameter study varies the maximum n-gram length over $\{1, 3, 5, 7\}$ and the extraction threshold over $\{0.1, 0.3, 0.5, 0.7\}$. The original heatmaps are reproduced in Figure 5.3. They show accuracy across the tested combinations for Novel and Medical. The main experiment uses $b = 3$ and $\tau = 0.5$.

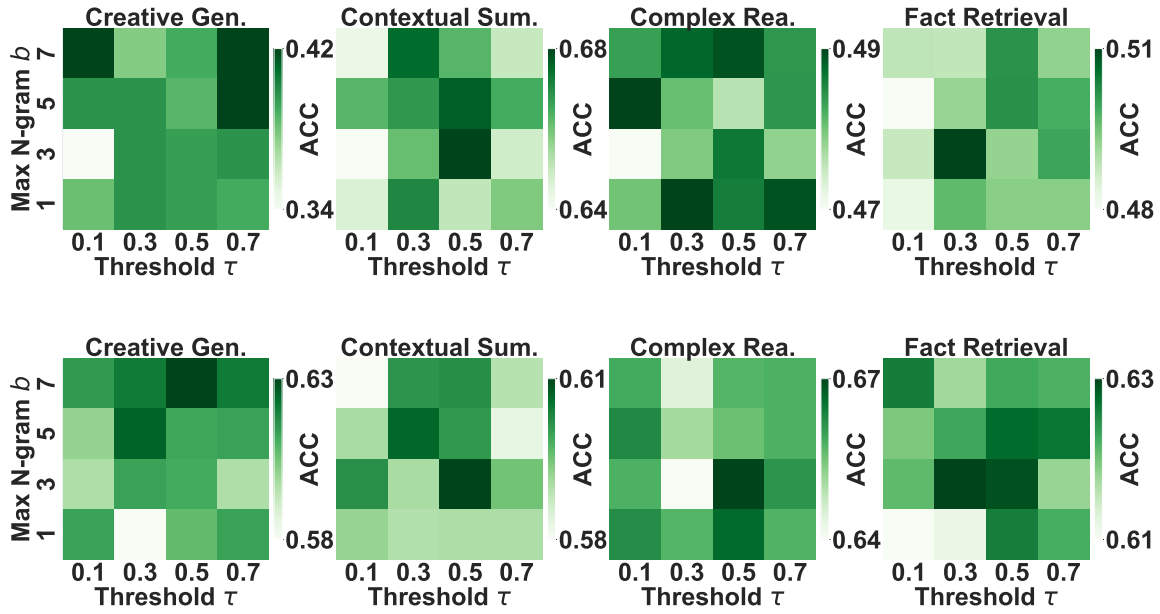


Figure 5.3: AGRAG parameter sensitivity on Novel (top) and Medical (bottom). The original panels vary the extraction threshold and maximum n-gram length and display task accuracy. Reproduced from Figure 3 of the source paper [3].

5.8 Discussion and Chapter Summary

We proposed AGRAG to retrieve connected facts while retaining their source context. Query-conditioned influence and semantic edge costs guide subgraph expansion, and hybrid passage retrieval supplements the selected graph.

Removing hybrid context can retain high faithfulness while reducing coverage. Graph facts can support the claims that are generated yet omit information needed for the full answer. Expansion beyond the initial tree also helps summarization and creative generation, while its effect on fact retrieval is smaller.

Relations are model-extracted, query anchors are model-filtered, and several QA metrics use model assessment. Errors in the extracted relations or filtered anchors can change the evidence supplied to generation.

AGRAG addresses connected evidence selection for textual tasks. Chapter 6 extends the problem to multimodal evidence, where a query relation may match a path with a different topology.

CHAPTER 6

SEMANTIC GRAPH MATCHING FOR MULTIMODAL REASONING WITH MGRAG

This chapter is based on *MGRAG: Semantic Subgraph Matching and Graph-Aware Caching for Multimodal Retrieval-Augmented Generation*, published in the *Proceedings of the VLDB Endowment*, volume 19, number 9, pages 2480–2493, 2026 [4]. We propose MGRAG to construct query-specific multimodal graphs, match query relations to semantic paths, and guide selective recomputation of cached model states.

6.1 Multimodal Evidence and Query Requirements

Multimodal questions can require relationships distributed across text and images. A passage may name a person, an image may show an action, and another passage may link that person to an event. Retrieving the items independently can leave the required connection unresolved. We construct query-related graphs and match their paths to the relationships requested by the question.

This chapter considers a corpus $\mathscr{D} = \mathscr{D}_{\text{txt}} \cup \mathscr{D}_{\text{img}}$ and a query q that can contain text, an image, or both. A retrieval module selects evidence from the corpus, and a multimodal language model generates an answer using that evidence. The research question is how to construct and retrieve a query-relevant relational representation while controlling the cost of preparing and processing the context. The answer-generation task remains grounded in the original documents and images. The graph supplies an additional representation of their query-related content.

The first challenge is extracting the right facts. A corpus-wide graph is constructed before the question is known, and its extraction prompt may omit a visual detail needed later. We instead include the query in extraction and incrementally construct an evidence graph from ranked chunks.

The second decision concerns what constitutes a match. A query and its supporting documents need not use the same decomposition of a fact. The published motivating example describes a data graph containing two `father_of` edges and a query containing one `grandfather_of` edge. The requested relation is represented by a two-hop path in the data. Re-

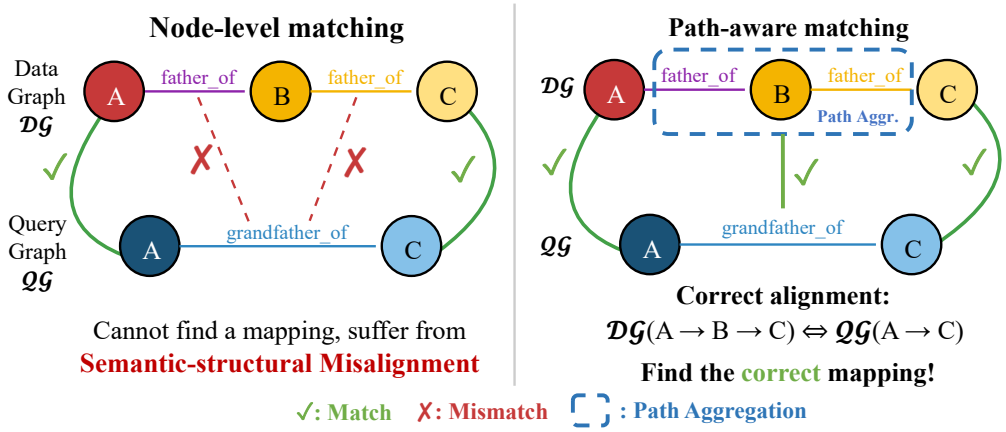


Figure 6.1: Path-level correspondence in the MGRAG motivating example. A `grandfather_of` relation corresponds to two `father_of` edges. Original Figure 1, p. 2481 [4].

quiring the same edge topology would exclude this correspondence. Figure 6.1 reproduces the example. MGRAG evaluates semantic representations of paths so that a query relation can be supported by a sequence of evidence relations.

The third decision concerns the cost of processing retrieved evidence. Graph construction already sends evidence to a language model to extract entities and relations. Some of that content appears again when the matched graph and its source chunks are used for answering. Reusing the earlier key-value (KV) states can reduce repeated computation. Their suitability depends on the new query and context, so MGRAG selectively recomputes token states. It combines attention scores with topological features of the facts associated with those tokens. Structure thus enters both evidence selection and the subsequent use of the selected context.

6.2 Framework and Evidence Representations

Figure 6.2 shows three stages. Coarse-grained ranking orders text chunks and images according to their relevance to the query. Fine-grained retrieval consumes this ranked list, extracts query-related facts, and searches the growing graph for a semantic match. Graph-aware KV recomputation then determines which cached token states to update for answer generation. The stopping decision in the middle stage connects ranking, graph construction, and matching. Each additional chunk can change whether the graph contains a suitable correspondence.

Two graphs serve different purposes. The query graph $\mathcal{QG}_q = (\widehat{\mathcal{V}}_q, \widehat{\mathcal{E}}_q)$ describes the entities and relations requested by q, including placeholders for unknown information. The evidence graph $\mathcal{KG}_q = (\mathcal{V}_q, \mathcal{E}_q)$ contains facts extracted from the chunks considered for that query. A mapping connects query entities to evidence entities. Paths between the mapped endpoints sup-

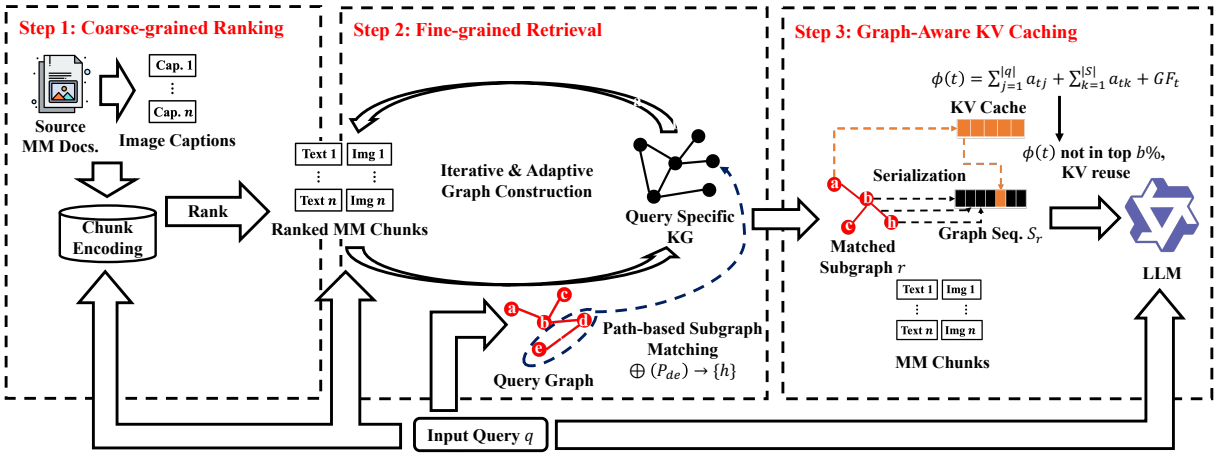


Figure 6.2: The MGRAG pipeline. Ranked chunks support incremental query-aware graph construction and path-based matching. The matched graph also supplies features for selective KV recomputation. Original Figure 2, p. 2484 [4].

ply candidate evidence for the query relations.

For generation, MGRAG combines the graph sequence S_r, the original chunks $\mathscr{C}_q$ consumed during construction, and the query q. The chunks retain source text and original images, including details omitted from the extracted triples.

6.3 Coarse-grained Ranking

MGRAG first divides textual documents into overlapping chunks. The published setting uses a chunk length $l_c = 256$ tokens and overlap $l_o = 32$ tokens. Images form the other members of the chunk collection, giving $\mathscr{C} = \mathscr{T} \cup \mathscr{D}_{\text{img}}$, where $\mathscr{T}$ denotes the text chunks. This collection is the input to the coarse retrieval stage. Ranking identifies the order in which content will be examined, rather than directly defining the final graph context.

The implementation uses the text encoder bge-base-en-v1.5 [57]. Text chunks are encoded directly. An image is represented for ranking by its caption, which can be supplied by the dataset or generated by a language model. Both types of evidence can therefore be compared in the text embedding space. Let $\mathscr{M}$ denote that encoder and $\mathbf{h}_c$ the representation of a candidate chunk. For an image chunk, the input to $\mathscr{M}$ is its caption. This detail matters because the ranking representation expresses the image through language, whereas the later extraction and answering stages can use its visual content.

For a query containing text q_{txt} and an image caption q_{cap}, MGRAG averages the two representations,

$$\mathbf{h}_q = \frac{\mathscr{M}(q_{\text{txt}}) + \mathscr{M}(q_{\text{cap}})}{2}. \tag{6.1}$$

A query with one modality uses its corresponding textual representation. The distance to a candidate is

$$d(q, c) = \|\mathbf{h}_q - \mathbf{h}_c\|_2, \tag{6.2}$$

and the chunks are ordered by increasing distance. Equations 6.1 and 6.2 define the query representation and ranking distance.

Fine-grained retrieval processes chunks in coarse-ranking order. Each chunk can add facts needed for a match, and construction stops when matching succeeds or the iteration budget is exhausted.

Caption quality affects which images receive an early position. A caption can omit a visual detail that becomes relevant during graph construction, or foreground content unrelated to the query. The published CLIP substitution experiment tests an alternative coarse representation within this pipeline. Section 6.11 reports the encoder comparison.

6.4 Query-aware Graph Construction

6.4.1 Representing the query

We extract entities and relations from the query. Known entities retain their descriptions, while requested values use semantic placeholders such as *Person* or *Year*. The matcher can compare these placeholders with evidence entities before the answer values are known.

Figure 6.3 shows the query and evidence graph representations. The placeholder boxes describe missing knowledge in the query. Passage nodes connect extracted entities back to their chunks. These connections let the graph carry both relational content and its association with the original multimodal evidence. In a multimodal corpus, a passage node can represent an image as well as a text chunk.

The original example asks which year an Irandhir Santos film was released when its poster shows people doing martial arts. The query graph connects the actor to an unknown film, the film to a poster, and a person on the poster to an action. It also includes a release-year requirement. The evidence graph contains named film candidates and a visual description of a man performing a martial-arts kick in midair. The depicted film, *The Assailant*, is connected to the year 2009.

The actor name leads to several film candidates. The poster action distinguishes the requested film, whose release relation then supplies the year. Matching must therefore connect the actor, film, visual action, and release date.

The film placeholder aligns with a candidate film entity, and the year placeholder marks the

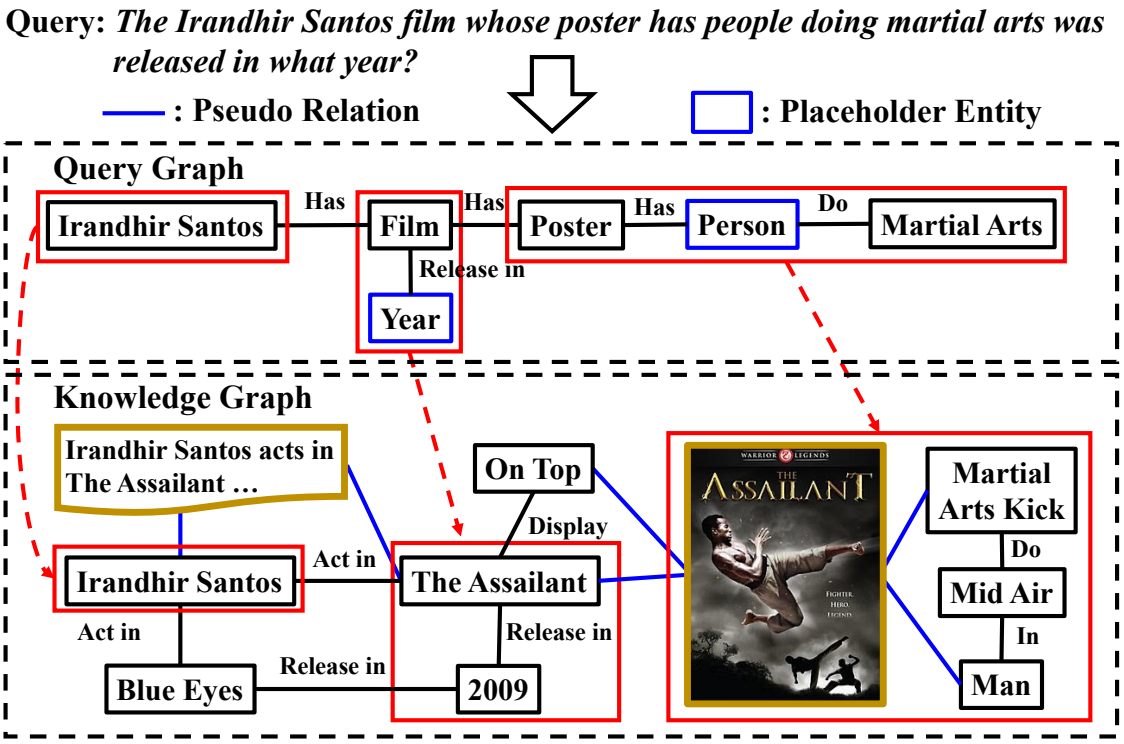


Figure 6.3: Query and evidence graphs in path-based semantic matching. Placeholder entities express unknown query information; pseudo-relations connect passage nodes to extracted entities. Original Figure 3, p. 2485 [4].

requested release value. Path matching checks the relationships connecting those entities.

Extraction errors in query placeholders or relations can change the evidence accepted by the matcher.

6.4.2 Extracting facts from ranked chunks

For a chunk c, the extraction model receives the chunk, query, and extraction prompt p_{ext}. Its output contains query-related entities and relations,

$$(\mathcal{V}_q^c, \mathcal{E}_q^c) = \text{LLM}(c, q, p_{\text{ext}}). \tag{6.3}$$

Including q makes the extraction selective. The model is asked to identify events and relations relevant to the current question, rather than to enumerate every describable fact in the chunk. For an image, this permits attention to query-related visual events; for text, it permits selection of the entities and relationships needed to describe the requested evidence.

The chunk is also added as a passage node. MGRAG connects this node to the entities extracted from it using the pseudo-relation `consists_in`, retaining the published relation name and orientation from chunk to entity. If $\mathcal{C}_q^{(i)}$ is the prefix of ranked chunks consumed after

iteration i, the cumulative graph contains

$$\mathscr{V}_q^{(i)} = \bigcup_{c \in \mathscr{C}_q^{(i)}} (\mathscr{V}_q^c \cup \{c\}), \tag{6.4}$$

$$\mathscr{E}_q^{(i)} = \bigcup_{c \in \mathscr{C}_q^{(i)}} \left(\mathscr{E}_q^c \cup \{(c, \texttt{consists_in}, v) : v \in \mathscr{V}_q^c\} \right). \tag{6.5}$$

The iteration-indexed notation makes explicit that the unions range over the chunks already processed.

Passage nodes connect extracted facts to their source chunks and can link facts from the same source item.

6.4.3 Matching as a stopping decision

After each graph update, MGRAG attempts path-based semantic subgraph matching against the query graph. A valid match ends the construction loop. If the matcher returns an empty result, the next ranked chunk is used to enrich the graph. The loop is bounded by the maximum number of construction iterations μ. Thus, the amount of graph construction depends on both the ranking order and the matching outcome.

If a required endpoint or path is missing, another chunk can supply it. Matching thus determines whether construction continues, using the relational requirements of the query graph.

The iteration cap limits how far construction proceeds through the ranked list. A valid match can stop it earlier, while relevant evidence below the cap remains unexamined.

6.5 Path-based Semantic Subgraph Matching

6.5.1 From edges to paths

A relation in the query graph can be expressed by several relations in the evidence graph. MGRAG assigns text embeddings to entity and relation descriptions. For a path $P(u, v)$ containing relations $r_1, \ldots, r_k$, it defines a semantic representation by aggregating the relation embeddings,

$$\mathbf{h}_{P(u,v)} = \bigoplus_{j=1}^{k} \mathbf{h}_{r_j}. \tag{6.6}$$

The aggregation can use mean pooling to represent an evidence path. Greedy extension additionally compares the current path, candidate edge, and candidate endpoint with the query

relation.

Let $I : \widehat{\mathscr{V}_q} \to \mathscr{V}_q$ be a candidate entity mapping. For a query edge $(u, \hat{r}, v)$, the candidate evidence paths run from $I(u)$ to $I(v)$. Denote this collection by $\mathscr{P}(I(u), I(v))$. The matching objective is

$$\max_{I} \sum_{(u,\hat{r},v)\in\widehat{\mathscr{E}_q}} \max_{P\in\mathscr{P}(I(u),I(v))} \cos\big(\mathbf{h}_{\hat{r}}, \mathbf{h}_P\big). \tag{6.7}$$

Candidate mappings must satisfy the semantic threshold θ for the mapped entities and for every query relation's supporting path,

$$\cos\big(\mathbf{h}_u, \mathbf{h}_{I(u)}\big) > \theta \qquad \text{for all } u \in \widehat{\mathscr{V}_q}, \tag{6.8}$$

$$\max_{P\in\mathscr{P}(I(u),I(v))} \cos\big(\mathbf{h}_{\hat{r}}, \mathbf{h}_P\big) > \theta \qquad \text{for all } (u, \hat{r}, v) \in \widehat{\mathscr{E}_q}. \tag{6.9}$$

The constraints validate each query edge against an associated evidence path. The selected evidence is the subgraph formed from the mapped entities and accepted paths, which can include intermediate entities absent from the query graph.

For a fixed entity mapping, the inner maximization in Equation 6.7 selects a path for each query relation. The outer search compares mappings using these path scores. Similar endpoints can therefore produce different matching scores when their connecting paths express different relations.

The returned subgraph is the union of the accepted paths, which can share nodes and edges. Intermediate entities allow a relation stated compactly in the question to match a longer description in the evidence.

In the family-relation example, two father relations support one grandfather relation. The path similarity must pass validation, and its endpoints must also align with the query entities.

6.5.2 Anchor selection and candidate mappings

MGRAG first computes pairwise cosine similarities between query entities and evidence entities. The highest-similarity pair supplies an anchor entity in the query graph and its candidate counterpart in the evidence graph. If no valid match is obtained from that anchor, the method tries other anchors. For the selected query anchor, Algorithm 1 considers each evidence counterpart exceeding the threshold and enumerates mappings consistent with that correspondence.

Candidate mappings are enumerated using a similarity-based modification of VF2 [58]. VF2 supplies a graph-matching search procedure; the MGRAG adaptation uses the threshold θ in semantic feasibility checks. Each candidate is then validated relation by relation through greedy

Algorithm 1 Path-based semantic matching in MGRAG

Require: Query graph $\mathcal{QG}_q$, query anchor γ, evidence graph $\mathcal{KG}_q$, threshold θ

Initialize valid results $\mathcal{R} \leftarrow \varnothing$
for each $v_a \in \mathcal{V}_q$ with $\cos\left(\mathbf{h}_{v_a}, \mathbf{h}_\gamma\right) > \theta$ **do**
  Enumerate candidate mappings consistent with $I(\gamma) = v_a$
  for each candidate mapping I **do**
    Initialize the subgraph with mapped query entities; set $s \leftarrow 0$
    for each query edge $(u, \hat{r}, v)$ **do**
      Search greedily for a semantically valid path from $I(u)$ to $I(v)$
      if the path search fails **then**
        Reject I and stop validating this mapping
      else
        Add the path's nodes and edges to the candidate subgraph
        Add its cosine similarity to s
      end if
    end for
    if every query edge has a valid path **then**
      Add the subgraph and score $s/|\widehat{\mathcal{E}}_q|$ to $\mathcal{R}$
    end if
  end for
end for
Return the highest-scoring result, or $\varnothing$ if $\mathcal{R}$ is empty

path search. A candidate entity correspondence can consequently be rejected because one of its required relations lacks an acceptable evidence path.

For each accepted mapping, the algorithm takes the union of the paths that support its query edges. It averages the edge-wise semantic similarities to obtain a score s. The highest-scoring accepted subgraph is returned. Dividing by the number of query edges does not change the ranking of candidate mappings for the same query. It provides an average semantic score with a common range across queries whose graph sizes differ.

6.5.3 Greedy semantic path growth

The path search begins at the mapped source entity and maintains its current endpoint x. For each neighboring entity y and connecting edge (x, r, y), MGRAG evaluates the candidate extension using

$$\cos\left(\mathbf{h}_P \oplus \mathbf{h}_{e(x,r,y)} \oplus \mathbf{h}_y, \mathbf{h}_{\hat{e}(u,\hat{r},v)}\right). \tag{6.10}$$

Among extensions exceeding θ, it selects the highest-scoring one, appends that edge to the path, and updates the endpoint to y. This repeats until the path reaches the mapped target. The search fails when no qualifying extension is available. The experiments also impose a maximum path

length l_p to bound expansion.

A path must reach the mapped target and pass the semantic checks. Greedy selection can follow a locally attractive extension that blocks a valid route, so search failure does not imply that no valid path exists. The algorithm returns the best accepted candidate it finds.

A mapping is accepted only when every query edge has a valid path. Accepted paths may have different lengths. The construction loop then stops; otherwise, it adds the next ranked chunk and repeats matching.

6.5.4 Published algorithmic properties

The publication states that path-based semantic subgraph matching is NP-hard. Its proof sketch reduces subgraph isomorphism to a restricted instance with uniform embeddings and unit path lengths. The result motivates the use of semantic path growth and constrained mapping enumeration. The complexity of enumerating mappings remains relevant even when each individual extension is selected greedily.

The published Theorem 2 gives a threshold-based quality condition for a valid returned match. Every matched query relation has a supporting path with cosine similarity at least θ. Under the strict acceptance test in Equation 6.9, the average score of an accepted mapping exceeds θ as well. This condition describes the semantic validation of returned evidence. It is conditional on finding a valid match and does not require the algorithm to recover one whenever such a subgraph exists.

The technical report discusses the threshold in terms of a bottleneck transport cost [59]. With a per-edge cost of one minus cosine similarity, the threshold limits the largest accepted local cost. Section 6.9 evaluates graph-matching substitutions and answer quality.

6.5.5 Serializing selected evidence

After matching, MGRAG serializes the selected triples into S_r, separating facts with `[SEP]`. This sequence is combined with q and the original chunks consumed during construction. The serialization supplies an explicit list of entities and relationships to the answering model. Original chunks remain available because extraction is a selective description of the evidence, and some linguistic or visual details may lie outside the triples.

Construction and answering process overlapping content. We reuse cached states from construction and recompute selected tokens for the answering context.

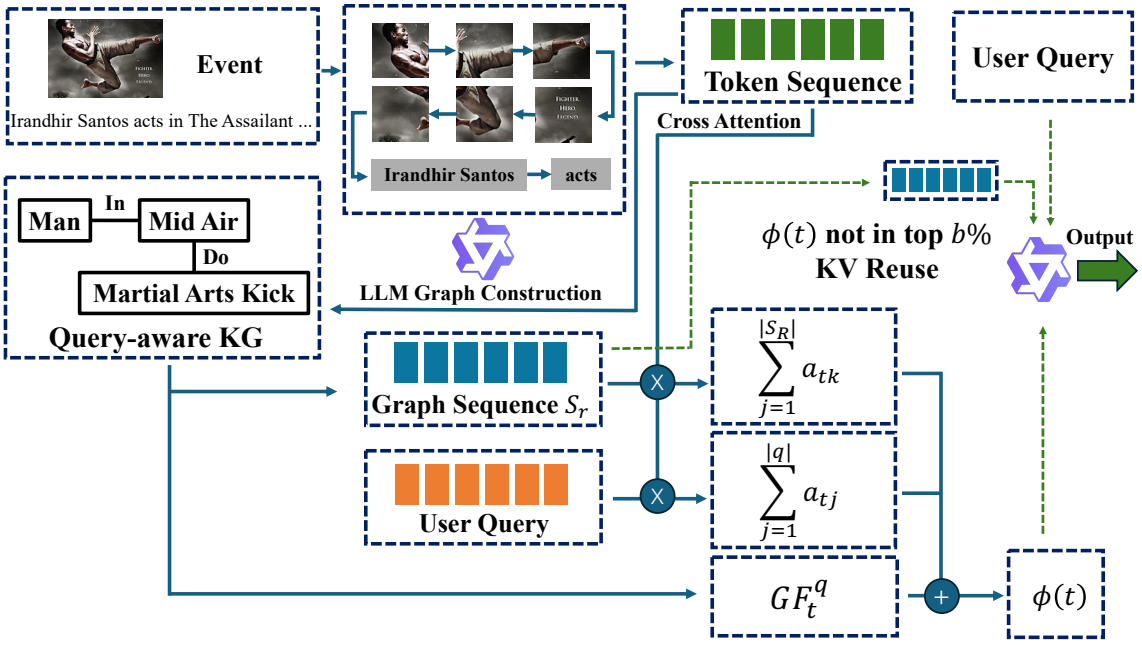


Figure 6.4: Graph features supplement attention in token recomputation scoring. The diagram retains the original graphical content. Original Figure 4, p. 2487 [4].

6.6 Graph-aware KV Recomputation

6.6.1 Reuse and selective updates

MGRAG reuses key and value states from graph construction when their content appears in the answering context. Because the assembled context differs from the earlier extraction input, it recomputes selected token states and reuses the rest.

Let $\phi(t)$ be the recomputation score of token t. At each transformer layer, MGRAG recomputes the tokens in the highest $b\%$ of scores and reuses the cached states for the remaining tokens. The published description uses a fixed-size cache with least-recently-used eviction. Its proposed policy concerns the ranking of token states for selective recomputation. The budget is represented by b; the numerical experiment tables do not specify a fixed value for that budget.

Figure 6.4 shows how the graph contributes. Query-related events are represented by entities and relations. The score combines attention to the query, attention to the graph sequence, and a feature derived from the topology of the associated fact. The goal of the added term is to give tokens access to an event-level structural signal alongside token-level attention.

6.6.2 Topological features of facts

MGRAG uses degree, betweenness, eigenvector, and closeness centrality to characterize a node. Degree centrality measures its normalized number of adjacent connections. Betweenness characterizes its participation in paths connecting other nodes. Eigenvector centrality weights connections by the centrality of their neighbors. Closeness uses distances to other graph nodes. The published method averages these four quantities,

$$f(u) = \frac{dc(u) + bc(u) + ec(u) + cc(u)}{4}. \tag{6.11}$$

For a token associated with a triple (u, r, v), its graph feature is the mean of the endpoint scores,

$$GF_t^q = \frac{f(u) + f(v)}{2}. \tag{6.12}$$

The token-to-fact association follows the entity and relation token spans in the serialized representation. Tokens referring to the same fact therefore receive a shared structural component.

The structural feature measures an entity's position in the query-specific graph. Nodes connecting several extracted facts can receive different scores from isolated endpoints.

The graphs contain fewer than twenty entities on average in the published evaluation. We compute the four centrality measures on these query-specific graphs.

6.6.3 Combined recomputation score

Let a_{tj} denote attention between token t and token j, and let $|q|$ and $|S_r|$ denote the query and serialized-graph lengths. The score is

$$\phi(t) = \sum_{j=1}^{|q|} a_{tj} + \sum_{k=1}^{|S_r|} a_{tk} + GF_t^q. \tag{6.13}$$

The score adds attention to the question, attention to the serialized graph, and the token's graph feature. Tokens with higher combined scores receive priority for recomputation.

The publication compares the policy with vLLM prefix caching [47] and CacheBlend [22]. Their retrieval pipelines and cache variants are identified separately in the evaluation. The inverse ablations also evaluate graph-aware recomputation within an existing retrieval pipeline. These comparisons expose answer-quality and time changes under several compositions of retrieval and caching.

6.7 Computational Cost

The published offline encoding cost is $O(|\mathscr{C}|d_{\text{emb}}l_c^2)$, with embedding dimension d_{emb}. Using k processed chunks, query-graph vertex and edge sets $\widehat{\mathscr{V}}$ and $\widehat{\mathscr{E}}$, evidence-graph sets $\mathscr{V}$ and $\mathscr{E}$, maximum path length l_p, and language-model hidden dimension d_{LLM}, its reported online bound is

$$\begin{aligned} O\big(&d_{\text{emb}}kl_c^2 + l_p|\widehat{\mathscr{E}}|\,|\mathscr{V}|^{|\widehat{\mathscr{V}}|+1} + |\mathscr{V}||\mathscr{E}| \\ &+ d_{\text{LLM}}(|S_r|^2 + |q|^2)\big). \end{aligned} \tag{6.14}$$

The mapping-enumeration term is exponential in query-graph size. Query-specific construction and a bounded path length restrict the graphs processed in the evaluated setting. Actual module costs provide the complementary evidence about where runtime is spent. In particular, graph construction calls the language model repeatedly, whereas matching operates on the extracted graph. Section 6.10 reports these costs individually.

6.8 Experimental Protocol

We evaluate MGRAG on MMQA [20] and the Arxiv, Manual, Web, Wiki, Wit, and Recipe subsets of MRAMG-Bench [21]. Table 6.1 preserves the published dataset statistics. MMQA includes textual and image queries. The six MRAMG subsets contain textual queries over multimodal corpora. Their query-hop ranges and corpus sizes differ, so results are reported by dataset.

Table 6.1: Evaluation datasets, reproduced from published Table 3, p. 2488 [4]. T and I denote textual and image query modalities. The columns comprise MMQA and six MRAMG subsets.

Statistic	MMQA	Wiki	Wit	Recipe	Arxiv	Manual	Web
Query Modality	T / I	T	T	T	T	T	T
Queries	951	500	600	2,360	200	390	750
Texts	218,285	538	639	1,528	101	40	1,500
Images	57,058	538	639	8,569	337	2,607	1,500
Corpus Size	22.5M	680k	478k	1.1M	136k	420k	215k
Query Hops	1–2	2–5	2–4	1–6	3–4	2–6	2–3
Avg Query Hops	1.5	3.0	2.8	3.3	3.1	3.7	2.4

The shared generation backbone is Qwen2.5-VL-7B-Instruct [60]. The experiments run on an Intel Xeon Gold 5220R CPU at 2.20 GHz and one NVIDIA A100-SXM4 GPU with 40 GB memory. The retrieval amount is set to five, text chunks use 256 tokens, and overlap is 32 tokens. HippoRAG2 uses Contriever for dense passage retrieval [48]; the other reported embedding

configurations use bge-base-en-v1.5. Similarity-based retrieval represents images through their captions.

VanillaRAG is implemented with LangChain and FAISS [61]. The GraphRAG implementation is nano-graphrag [62], and the remaining baseline systems use their released implementations. The protocol selects hybrid search for LightRAG and local search for GraphRAG. vLLM accelerates the tested baseline systems, while MGRAG uses its graph-aware cache procedure. Other baseline hyperparameters follow the implementations' defaults.

Four metrics characterize answers. Semantic similarity (SIM.) is the cosine similarity between bge-base-en-v1.5 representations of generated and reference answers. Accuracy (ACC.) is the reported proportion of correct answers. ROUGE-L (ROG.) evaluates longest-common-subsequence overlap [54], and BLEU-1 evaluates modified unigram precision [63]. The paper uses the RAGAS evaluation framework [56]. The tables preserve its metric names and precision, including the label BLEU used for BLEU-1.

The publication reports point estimates for these experiments.

The complete-system baselines use different retrieval units. VanillaRAG returns dense-retrieved chunks, and R1-Searcher incorporates reinforcement learning into search and question answering. GraphRAG organizes its graph into communities and prepares summaries; LightRAG retrieves graph elements and neighborhoods through its selected search mode. HippoRAG2 adds passage nodes and uses Personalized PageRank for retrieval.

Component substitutions evaluate graph construction, semantic matching, and cache policy within MGRAG. Inverse ablations add matching and recomputation to other retrieval systems.

6.9 Answer Quality and Structural Retrieval

6.9.1 Complete-system comparisons

Table 6.2 reproduces the complete-system results. MGRAG has the highest value for all four metrics on each dataset. On MMQA, its accuracy is 0.7442, compared with 0.7053 for VanillaRAG and 0.6335 for HippoRAG2. Its semantic similarity is 0.8533, ROUGE-L is 0.6763, and BLEU-1 is 0.7215. GraphRAG and LightRAG have missing MMQA entries because those experiments exceeded three days.

Table 6.2: Answer quality with Qwen2.5-VL-7B-Instruct, reproduced from published Table 4, p. 2489 [4]. Vanilla., R1., Graph., Light., and Hippo. denote VanillaRAG, R1-Searcher, GraphRAG, LightRAG, and HippoRAG2. A dash means that the experiment exceeded three days; it is not a zero score. Bold and underlining retain the publication's highlighting.

Dataset	Metric	Vanilla.	R1.	Graph.	Light.	Hippo.	MGRAG
MMQA	SIM.	0.8362	0.5478	-	-	0.8381	**0.8533**
MMQA	ACC.	0.7053	0.3567	-	-	0.6335	**0.7442**
MMQA	ROG.	0.5942	0.1041	-	-	0.6659	**0.6763**
MMQA	BLEU	0.6645	0.0817	-	-	0.5917	**0.7215**
Arxiv	SIM.	0.8664	0.7051	0.7943	0.7404	0.8293	**0.8760**
Arxiv	ACC.	0.6638	0.3775	0.2350	0.3813	0.4688	**0.6800**
Arxiv	ROG.	0.2713	0.1189	0.1553	0.1580	0.2331	**0.2726**
Arxiv	BLEU	0.2452	0.1092	0.1690	0.1370	0.1281	**0.2498**
Manual	SIM.	0.8166	0.6046	0.5453	0.7255	0.6852	**0.8221**
Manual	ACC.	0.5881	0.2071	0.1436	0.4096	0.2641	**0.6173**
Manual	ROG.	0.3509	0.1078	0.0974	0.1595	0.2047	**0.3839**
Manual	BLEU	0.2767	0.0740	0.0718	0.1385	0.1314	**0.3093**
Web	SIM.	0.6937	0.6620	0.5470	0.5513	0.5642	**0.7009**
Web	ACC.	0.4703	0.4200	0.1053	0.2000	0.3607	**0.5123**
Web	ROG.	0.1508	0.1142	0.1133	0.0905	0.0616	**0.1664**
Web	BLEU	0.0326	0.0785	0.1031	0.0206	0.0133	**0.1456**
Wiki	SIM.	0.8824	0.5478	0.6966	0.7517	0.8204	**0.8838**
Wiki	ACC.	0.7800	0.3567	0.2000	0.3660	0.6500	**0.7845**
Wiki	ROG.	0.3559	0.1418	0.1361	0.2008	0.3042	**0.3579**
Wiki	BLEU	0.3053	0.1146	0.1007	0.1863	0.1851	**0.3174**
Wit	SIM.	0.7851	0.6110	0.6211	0.6973	0.7230	**0.8033**
Wit	ACC.	0.6675	0.6017	0.1500	0.2483	0.5138	**0.6967**
Wit	ROG.	0.3255	0.1307	0.1319	0.2416	0.2712	**0.3589**
Wit	BLEU	0.2095	0.0934	0.1142	0.1928	0.1200	**0.2382**
Recipe	SIM.	0.8599	0.6155	0.8166	0.8118	0.7792	**0.8673**
Recipe	ACC.	0.6525	0.2180	0.2771	0.3334	0.4917	**0.6814**
Recipe	ROG.	0.3714	0.1065	0.1745	0.2128	0.3303	**0.4220**
Recipe	BLEU	0.2599	0.0773	0.1527	0.1648	0.1941	**0.3102**

The advantage over the strongest baseline varies by dataset and metric. On Wiki, accuracy is 0.7845 for MGRAG and 0.7800 for VanillaRAG, while their semantic similarities are 0.8838

and 0.8824. On Web, the corresponding accuracies are 0.5123 and 0.4703. On Recipe, MGRAG reports ROUGE-L of 0.4220 compared with 0.3714 for VanillaRAG.

Several graph-based baselines score below VanillaRAG in this multimodal protocol. For example, on Manual, GraphRAG, LightRAG, and HippoRAG2 report accuracies of 0.1436, 0.4096, and 0.2641, compared with VanillaRAG's 0.5881.

6.9.2 Construction and matching substitutions

Table 6.3 reorganizes every value from the published graph-construction and matching comparison. Stanford OpenIE [64] and the statistical extraction alternative STAT replace the construction mechanism. VF2 and ISMAGS [58, 65] provide exact-matching alternatives. SM, VJ, RRWM, and IPFP represent approximate graph matching [66–69]. BGE, SLOTAlign, and FGWAlign supply semantic alternatives [57, 70, 71]. The table preserves the publication's three-decimal precision for this experiment.

Table 6.3: Graph-construction and graph-matching substitutions, transposed from published Table 7, p. 2490 [4]. All twelve method columns and all four metric values per dataset are retained. O.IE and STAT replace graph construction; the other alternatives replace matching. The shared backbone is Qwen2.5-VL-7B-Instruct.

Dataset	Method	SIM.	ACC.	ROG.	BLEU
MMQA	O.IE	0.642	0.571	0.510	0.483
MMQA	STAT	0.655	0.602	0.519	0.500
MMQA	VF2	0.851	0.735	0.673	0.620
MMQA	ISMAGS	0.833	0.706	0.638	0.590
MMQA	SM	0.832	0.702	0.632	0.582
MMQA	VJ	0.812	0.715	0.646	0.606
MMQA	RRWM	0.838	0.698	0.647	0.599
MMQA	IPFP	0.773	0.633	0.601	0.585
MMQA	BGE	0.832	0.703	0.635	0.586
MMQA	SLOT	0.838	0.700	0.650	0.602
MMQA	FGW	0.837	0.697	0.647	0.598
MMQA	MGRAG	0.853	0.744	0.676	0.722
Arxiv	O.IE	0.618	0.526	0.177	0.138
Arxiv	STAT	0.640	0.576	0.184	0.145
Arxiv	VF2	0.871	0.650	0.270	0.239
Arxiv	ISMAGS	0.868	0.643	0.266	0.231

Table 6.3 (continued)

Dataset	Method	SIM.	ACC.	ROG.	BLEU
Arxiv	SM	0.866	0.670	0.263	0.244
Arxiv	VJ	0.872	0.667	0.269	0.245
Arxiv	RRWM	0.874	0.675	0.257	0.243
Arxiv	IPFP	0.814	0.581	0.212	0.203
Arxiv	BGE	0.874	0.668	0.270	0.244
Arxiv	SLOT	0.872	0.653	0.269	0.233
Arxiv	FGW	0.868	0.651	0.264	0.230
Arxiv	MGRAG	0.876	0.680	0.273	0.250
Manual	O.IE	0.520	0.459	0.245	0.209
Manual	STAT	0.509	0.443	0.234	0.182
Manual	VF2	0.817	0.612	0.377	0.306
Manual	ISMAGS	0.817	0.612	0.382	0.307
Manual	SM	0.813	0.614	0.378	0.303
Manual	VJ	0.808	0.608	0.372	0.298
Manual	RRWM	0.811	0.614	0.377	0.303
Manual	IPFP	0.800	0.594	0.369	0.289
Manual	BGE	0.815	0.615	0.375	0.303
Manual	SLOT	0.813	0.611	0.375	0.301
Manual	FGW	0.820	0.600	0.374	0.292
Manual	MGRAG	0.822	0.617	0.384	0.309
Web	O.IE	0.466	0.290	0.052	0.020
Web	STAT	0.485	0.298	0.101	0.095
Web	VF2	0.677	0.507	0.156	0.088
Web	ISMAGS	0.675	0.509	0.155	0.079
Web	SM	0.676	0.507	0.156	0.056
Web	VJ	0.679	0.508	0.160	0.056
Web	RRWM	0.675	0.503	0.157	0.066
Web	IPFP	0.666	0.493	0.149	0.055
Web	BGE	0.687	0.510	0.156	0.095
Web	SLOT	0.676	0.498	0.154	0.116
Web	FGW	0.676	0.480	0.147	0.096
Web	MGRAG	0.701	0.512	0.166	0.146

Table 6.3 (continued)

Dataset	Method	SIM.	ACC.	ROG.	BLEU
Wiki	O.IE	0.504	0.498	0.230	0.200
Wiki	STAT	0.525	0.510	0.253	0.218
Wiki	VF2	0.877	0.752	0.352	0.300
Wiki	ISMAGS	0.879	0.754	0.355	0.303
Wiki	SM	0.878	0.750	0.353	0.298
Wiki	VJ	0.873	0.751	0.349	0.297
Wiki	RRWM	0.878	0.743	0.355	0.301
Wiki	IPFP	0.869	0.735	0.348	0.296
Wiki	BGE	0.878	0.746	0.355	0.301
Wiki	SLOT	0.879	0.749	0.356	0.301
Wiki	FGW	0.878	0.747	0.352	0.300
Wiki	MGRAG	0.884	0.785	0.358	0.317
Wit	O.IE	0.492	0.465	0.233	0.210
Wit	STAT	0.489	0.453	0.205	0.198
Wit	VF2	0.799	0.693	0.354	0.234
Wit	ISMAGS	0.797	0.694	0.355	0.231
Wit	SM	0.784	0.686	0.353	0.229
Wit	VJ	0.786	0.691	0.356	0.227
Wit	RRWM	0.795	0.679	0.347	0.230
Wit	IPFP	0.792	0.678	0.346	0.231
Wit	BGE	0.789	0.689	0.356	0.230
Wit	SLOT	0.793	0.693	0.351	0.227
Wit	FGW	0.794	0.691	0.349	0.223
Wit	MGRAG	0.803	0.697	0.359	0.238
Recipe	O.IE	0.551	0.425	0.211	0.202
Recipe	STAT	0.545	0.401	0.198	0.172
Recipe	VF2	0.859	0.679	0.413	0.304
Recipe	ISMAGS	0.861	0.668	0.407	0.302
Recipe	SM	0.862	0.678	0.411	0.305
Recipe	VJ	0.857	0.672	0.413	0.295
Recipe	RRWM	0.852	0.670	0.406	0.298
Recipe	IPFP	0.847	0.661	0.395	0.221

Table 6.3 (continued)

Dataset	Method	SIM.	ACC.	ROG.	BLEU
Recipe	BGE	0.864	0.674	0.415	0.308
Recipe	SLOT	0.861	0.675	0.412	0.306
Recipe	FGW	0.860	0.677	0.409	0.305
Recipe	MGRAG	0.867	0.681	0.422	0.310

On MMQA, the full method reports accuracy 0.744, compared with 0.571 for OpenIE and 0.602 for STAT. Query-aware construction outperforms these extraction alternatives on MMQA.

Among MMQA matching alternatives, VF2 reaches accuracy 0.735, while MGRAG reaches 0.744. VF2 also has ROUGE-L of 0.673 compared with 0.676. On Web, BGE reaches accuracy 0.510 and MGRAG 0.512. On Wiki, ISMAGS reaches 0.754 and MGRAG 0.785. MGRAG leads the displayed metric rows, with smaller MMQA and Web accuracy margins than the Wiki accuracy margin.

6.9.3 Queries requiring multiple hops

Table 6.4 reports results grouped by MRAMG query-hop count. For one-hop questions, VanillaRAG reports accuracy 0.613 and MGRAG 0.624. For six-hop questions, the reported values are 0.661 and 0.738. MGRAG also reports higher ROUGE-L and BLEU-1 for each hop group. VanillaRAG is faster in every hop group. The one-hop times are 0.13 and 0.35 minutes, and the six-hop times are 0.18 and 0.43 minutes.

Table 6.4: Results by query hop count, reorganized from published Table 8, p. 2490 [4]. Time is in minutes, as reported. Each original row is shown as two method rows, with the query count repeated.

Hops	Queries	Method	Time	ACC.	ROG.	BLEU
1-hop	201	VanillaRAG	**0.13**	0.613	0.379	0.240
1-hop	201	MGRAG	0.35	**0.624**	**0.425**	**0.286**
2-hop	1,237	VanillaRAG	**0.10**	0.588	0.304	0.193
2-hop	1,237	MGRAG	0.37	**0.601**	**0.335**	**0.230**
3-hop	2,357	VanillaRAG	**0.16**	0.633	0.329	0.238
3-hop	2,357	MGRAG	0.41	**0.659**	**0.355**	**0.260**
4-hop	499	VanillaRAG	**0.19**	0.697	0.338	0.259
4-hop	499	MGRAG	0.44	**0.712**	**0.373**	**0.293**

Table 6.4 (continued)

Hops	Queries	Method	Time	ACC.	ROG.	BLEU
5-hop	286	VanillaRAG	**0.18**	0.678	0.319	0.205
5-hop	286	MGRAG	0.45	**0.733**	**0.392**	**0.282**
6-hop	220	VanillaRAG	**0.18**	0.661	0.305	0.176
6-hop	220	MGRAG	0.43	**0.738**	**0.377**	**0.249**

The six-hop group contains 220 queries, whereas the three-hop group contains 2,357. The comparison shows a larger accuracy gap on the longest-hop group than on the shortest-hop group, alongside additional processing time.

6.10 Cache Quality and End-to-end Costs

Table 6.5 preserves the MMQA cache-system comparison. MGRAG reports accuracy 0.7442 and total time 4.3 hours. The vLLM and CacheBlend variants paired with VanillaRAG report accuracies 0.6593 and 0.6601, with total time 4.5 hours for both. Paired with HippoRAG2, they report accuracies 0.6335 and 0.6267, with total times of 8.8 and 8.3 hours.

Table 6.5: Cache-system comparison on MMQA, reproduced from published Table 6, p. 2489 [4]. V. and H. identify VanillaRAG and HippoRAG2 retrieval pipelines. TTFT is in seconds and total time is in hours.

System	SIM.	ACC.	ROG.	BLEU	TTFT	Time
vLLM (V.)	0.8315	0.6593	0.6277	0.5867	3.2 s	4.5 h
CacheBlend (V.)	0.8315	0.6601	0.6417	0.5964	**3.1 s**	4.5 h
vLLM (H.)	0.8381	0.6335	0.6659	0.5917	4.6 s	8.8 h
CacheBlend (H.)	0.8415	0.6267	0.6636	0.5917	4.4 s	8.3 h
MGRAG	**0.8533**	**0.7442**	**0.6763**	**0.7215**	3.2 s	**4.3 h**

Time to first token (TTFT) gives a different view from total time. CacheBlend with VanillaRAG reports 3.1 seconds, while MGRAG and vLLM with VanillaRAG report 3.2 seconds. MGRAG combines the highest answer-quality values with a total time of 4.3 hours; CacheBlend with VanillaRAG has the shortest TTFT.

The inverse-ablation table also measures quality when graph-aware recomputation is added after path matching. Within the HippoRAG2 composition, accuracy changes from 0.7372 with path matching to 0.7442 with both components, while the total time changes from 4.1 to 4.3

hours. Selective recomputation improves answer quality in this composition with an increase in total time.

Section 6.11 compares these component additions within the HippoRAG2 and VanillaRAG pipelines.

Table 6.6 reports time and token costs by module. On MMQA, graph construction accounts for 193.0 minutes, matching for 0.14 minutes, and answer generation for 8.9 minutes; the reported end-to-end time is 248.1 minutes. The construction and generation token entries are 3,085 and 1,431, with end-to-end token entry 4,516. The listed module times do not sum to the reported end-to-end total; the publication does not reconcile the difference in this breakdown.

Table 6.6: Module costs reproduced from published Table 5, p. 2489 [4]. Time is in minutes. KGC, QA, and E2E cells retain the published time/token pairs for graph construction, answering, and end-to-end processing. Match denotes matching time. Token counts and E2E totals are preserved as reported.

Dataset	Queries	KGC	Match	QA	E2E
MMQA	951	193.0 / 3,085	0.14	8.9 / 1,431	248.1 / 4,516
Arxiv	200	61.9 / 3,946	0.23	16.2 / 2,944	83.9 / 6,890
Manual	390	76.1 / 4,285	0.04	23.9 / 2,556	109.7 / 6,841
Web	750	143.8 / 4,623	0.27	42.9 / 3,101	196.5 / 7,723
Wiki	500	164.2 / 6,629	0.30	52.1 / 6,267	241.5 / 12,896
Wit	600	140.6 / 7,660	0.21	42.6 / 6,539	196.5 / 14,199
Recipe	2,360	525.7 / 4,351	1.74	191.3 / 3,173	808.7 / 7,525

Graph construction is the largest listed time component on all seven datasets. Recipe reports 525.7 minutes for construction, 1.74 for matching, and 191.3 for answering, with end-to-end time 808.7 minutes. On the smaller Arxiv query set, the corresponding values are 61.9, 0.23, and 16.2 minutes, with end-to-end time 83.9. Repeated extraction accounts for most of the listed computation time despite the matcher's exponential worst-case bound.

Although mapping enumeration has an exponential bound, matching processes small query-specific graphs in these experiments. The measured bottleneck is repeated multimodal extraction. Reducing the number of processed chunks or the cost per extraction is therefore a target for runtime improvement.

6.11 Ablations and Parameter Sensitivity

6.11.1 Component ablations

The MMQA ablations in Table 6.7 change matching, coarse encoding, and pipeline composition. Removing graph matching constructs the query-specific graph from a fixed number of chunks and reports accuracy 0.7260, compared with 0.7442 for MGRAG. Replacing caption-based coarse ranking with direct CLIP encoding [72] reports accuracy 0.7376. The Vanilla MGRAG variant applies path matching and graph-aware caching within the VanillaRAG pipeline with $k = 5$; it reports accuracy 0.6887.

Table 6.7: MMQA component ablations, reproduced from published Table 9, p. 2490 [4]. Time is in hours.

Model	SIM.	ACC.	ROG.	BLEU	Time
Without graph matching	0.8362	0.7260	0.6657	0.5774	4.4 h
With CLIP encoder	0.8513	0.7376	0.6744	0.7008	4.5 h
Vanilla MGRAG, $k = 5$	0.8260	0.6887	0.6266	0.5773	4.7 h
MGRAG	**0.8533**	**0.7442**	**0.6763**	**0.7215**	**4.3 h**

The full system also has the largest SIM., ROUGE-L, and BLEU-1 entries in this table. Its reported time is 4.3 hours, compared with 4.4 hours without graph matching, 4.5 hours with CLIP, and 4.7 hours for Vanilla MGRAG. Caption-based ranking performs better than the CLIP substitution on MMQA with this pipeline and backbone.

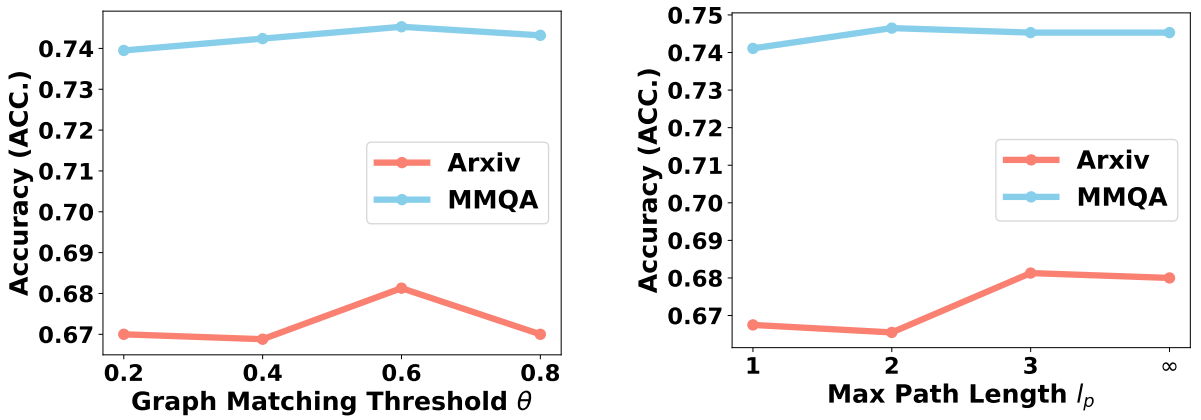
6.11.2 Adding components to other pipelines

Table 6.8 retains the inverse ablations. Adding graph-aware KV recomputation to HippoRAG2 changes accuracy from 0.6335 to 0.6855 and time from 7.5 to 7.6 hours. Adding path-based semantic matching gives accuracy 0.7372 and time 4.1 hours. Combining both gives accuracy 0.7442 and time 4.3 hours. The latter configuration shares MGRAG's reported values because the publication identifies its retrieval and generation setting with the full method once both components are incorporated.

For VanillaRAG, the base accuracy is 0.7053 at 2.6 hours. Adding path matching gives 0.7175 at 4.7 hours, and adding graph-aware recomputation as well gives 0.7232 at 4.8 hours. Thus, the component additions increase quality in these measured compositions, while their time changes depend on the host pipeline.

Table 6.8: MMQA inverse ablations, reproduced from published Table 10, p. 2491 [4]. PSGM and GKV denote path-based semantic graph matching and graph-aware KV recomputation. Time is in hours.

Model	SIM.	ACC.	ROG.	BLEU	Time
Hippo.	0.8381	0.6335	0.6659	0.5917	7.5 h
Hippo.+GKV	0.8392	0.6855	0.6651	0.7013	7.6 h
Hippo.+PSGM	0.8416	0.7372	0.6720	0.7128	4.1 h
Hippo.+PSGM+GKV	**0.8533**	**0.7442**	**0.6763**	**0.7215**	4.3 h
Vanilla.	0.8362	0.7053	0.5942	0.6645	**2.6 h**
Vanilla.+PSGM	0.8460	0.7175	0.6466	0.6973	4.7 h
Vanilla.+PSGM+GKV	0.8485	0.7232	0.6563	0.7001	4.8 h
MGRAG	**0.8533**	**0.7442**	**0.6763**	**0.7215**	4.3 h



(a) Similarity threshold θ.

(b) Maximum path length l_p.

Figure 6.5: MGRAG matching-parameter sensitivity. Original subfigures of Figure 5, p. 2491 [4].

6.11.3 Matching threshold and path length

Figure 6.5 reproduces the published parameter curves. The experiments vary the semantic threshold θ and maximum path length l_p, including lengths $1, 2, 3$, and unrestricted length. The reported discussion identifies $\theta = 0.6$ and $l_p = 3$ as effective choices in these tests.

The threshold changes which entity correspondences and path extensions can pass validation. A higher value demands closer embedding agreement, while a lower value permits more candidates. The path-length cap determines how many relations may be used to express a query relation. Its effect therefore concerns both representational flexibility and search.

6.12 Discussion and Chapter Summary

We proposed MGRAG to construct and match multimodal evidence for individual queries. Query-aware extraction selects relevant facts, semantic path matching connects them to the query, and graph-aware recomputation reuses processing from construction during answer generation.

MGRAG obtains the highest values across the seven datasets' four answer metrics. Construction and matching substitutions show the benefits of query-dependent extraction and path semantics. The hop-group results show larger quality gains for questions requiring more relational steps, with added runtime. Cache and inverse-ablation results show how recomputation changes quality and cost within different retrieval pipelines.

Caption omissions and extraction errors can exclude relevant facts before matching. An accepted match ends construction, so an incorrect correspondence can prevent examination of later chunks. The construction and path-length caps further restrict the search.

Semantic paths allow a query relation to match evidence expressed through several intermediate facts. The same graph can guide which cached token states are recomputed. The next chapter compares these uses of structure with external knowledge selection, online indexing, and connected textual retrieval.

CHAPTER 7

DISCUSSION

The four methods use relationships to make different evidence decisions. KGLink selects external descriptions for table columns, GORAG retrieves labels from an evolving index, AGRAG collects connected facts and passages, and MGRAG matches multimodal evidence and guides its processing. We compare when these decisions occur, how their errors propagate, and what they imply for answer quality and computation.

7.1 Where Structure Enters the Pipeline

KGLink filters linked entities before the encoder reads the table. Its graph processing determines which candidate types and feature descriptions enter the model, while supervised training learns how those descriptions correspond to annotation labels.

GORAG selects candidate labels before classification. Since predicted keyword–label associations are inserted into the index, one prediction can change the evidence retrieved for a later query.

AGRAG selects a connected subgraph for each query from a reusable corpus graph. Its hybrid retriever adds passages containing context absent from the selected relations.

MGRAG makes query dependence part of graph construction. Matching controls whether another chunk is processed, and graph features determine which cached token states receive recomputation priority.

The selected structure controls both which facts reach the model and how their relationships remain visible. MGRAG further uses that structure to allocate context-processing computation.

7.2 Static, Online, and Query-dependent Structures

KGLink reuses an external knowledge graph and adapts evidence selection to each table. Learning changes how the model uses the selected descriptions; it does not rebuild the source graph.

GORAG updates a task index as labels and text arrive. Associations from earlier predictions become available to later queries, so classification performance depends on the sequence of

Table 7.1: The roles of structure across the four studies. The entries summarize the methods in their respective research chapters.

Study	Structure source	Selected or represented evidence	Model role
KGLink	Existing knowledge graph and table links	Candidate types and graph-derived column features	Fine-tuned table encoding and type prediction
GORAG	Labeled examples and prescribed online updates	Connected keyword–label evidence and candidate labels	Label-description generation and classification
AGRAG	Corpus terms, extracted relations, and passage links	Query-conditioned connected subgraph and hybrid passages	Relation extraction, filtering, and generation
MGRAG	Query and ranked multimodal chunks	Semantic path matches, original chunks, and graph features	Query-aware extraction, QA, and selective KV recomputation

index states.

AGRAG prepares the corpus graph once, then changes query anchors, node influence, edge costs, and subgraph selection for each question. This amortizes graph preparation across repeated queries.

MGRAG extracts facts for each query as it consumes ranked chunks. Matching can stop construction before the full list is processed, but extraction and matching remain part of the query-time workload.

These alternatives correspond to different workloads. An existing knowledge graph is useful when entities and their external descriptions are available. Online indexing fits tasks whose label evidence evolves over a sequence. A reusable corpus graph supports repeated queries over shared text. Query-specific construction targets the evidence requirements of an individual multimodal question.

7.3 Evidence Quality and Computational Cost

The relevant quality criterion follows the task. A column representation must support the expected type label. Classification retrieval must retain the correct candidate label. Generation requires both sufficient answer coverage and support for the claims produced. Cache reuse must preserve answer quality while reducing processing cost.

Table 7.2 compares the main experimental conditions. KGLink evaluates a trained encoder; the other studies evaluate generative pipelines with different backbones. Shared datasets also

Table 7.2: Main experimental conditions across the four studies. Hardware denotes the reported local setup; GORAG's IFS-Rel generation uses the GPT-4o service. Full task-specific controls appear in the corresponding research chapters.

Study	Main model and hardware	Supervision and evidence state	Reported outcome
KGLink	BERT; Tesla V100	7:1:2 train/validation/test; external KG; three-run averages	Annotation accuracy and weighted F1 on a percentage scale
GORAG	Qwen2.5-7B on CAD/WOS; GPT-4o on IFS-Rel; A100 40 GB	1/5/10 support texts per label; cumulative labels; prediction-dependent index updates	Exact-label accuracy by round and support size
AGRAG	Qwen2.5-14B-Instruct; A100 80 GB	Fixed corpus graph; query-conditioned scores and subgraph; one training passage per label for classification	Task-specific QA assessment/overlap, coverage, faithfulness, and classification metrics
MGRAG	Qwen2.5-VL-7B-Instruct; A100 40 GB	Query-specific multimodal graph; semantic matching; selective KV recomputation	Answer quality, TTFT, and complete-system/module costs

have different supervision and update protocols. GORAG's WOS and IFS-Rel experiments accumulate labels and prediction-derived keyword–label associations over rounds, whereas AGRAG indexes one training passage per label for classification.

A shared metric label can also conceal a different assessment rule. GORAG's accuracy requires the predicted class to match the gold label. AGRAG's QA accuracy uses model judgments scored 0, 0.5, or 1, while its classification accuracy uses label correctness. Its source-defined ROUGE-L quantity divides longest-common-subsequence length by generated-answer length. MGRAG reports answer accuracy alongside semantic similarity and lexical overlap.

The AGRAG ablations show why coverage and faithfulness must be considered together. Without hybrid passages, the generator can produce supported claims while omitting required information. Graph connectivity helps organize facts, while passages preserve context that extraction or subgraph selection can omit.

MGRAG separates evidence retrieval from reuse of its computation. Cache comparisons measure combined retrieval and serving choices. Inverse ablations test additions within one retrieval pipeline, where improved answer quality can require extra runtime.

Indexing and query costs also need to remain separate. KGLink reports preprocessing and model-processing costs for annotation. GORAG reports the resource demands of its evolving index and retrieval. AGRAG separates corpus indexing from querying. MGRAG reports construction, matching, answer-generation, and end-to-end costs.

7.4 Information Preservation and Semantic Granularity

The hyper-relational study in Chapter 2 supplies another perspective on structural representations. A main triple and its qualifiers express different aspects of a fact. Decomposing that representation can retain some relationships while losing the association that gives them their meaning. The study examines this issue through decomposition methods, qualifier capture, graph reception, and link-prediction results.

A related design question appears in each core chapter. KGLink must translate graph-derived type descriptions into the annotation task's vocabulary. GORAG associates words with labels whose relevance depends on the classification task. AGRAG selects a graph serialization while retaining a passage channel for information absent from that serialization. MGRAG matches relations to paths that can express a corresponding meaning through a different topology. Each method specifies an evidence unit and a way to relate that unit to the task.

A column vector, candidate label, graph fact, path, and source passage preserve different information. Paths expose intermediate relations, while passages and images retain details that may disappear during extraction. Moving between these representations changes what the next stage must infer.

A useful structure must preserve the relationships needed for the task. Connectivity identifies available links, semantic matching checks their correspondence to a query, and supervision determines the required annotation category. The hyper-relational study examines how graph transformation can alter these relationships before prediction.

7.5 Error Propagation Across Structural Decisions

The location of a structural decision affects how an error can propagate. An incorrect KGLink entity link changes the descriptions offered to the encoder for a table. In GORAG, an incorrect predicted label can become an association available to later queries. AGRAG reuses a corpus graph, so an extracted relation can influence several query-specific selections. MGRAG constructs a new graph for each query, but a mistaken accepted match can end construction before later chunks are examined.

Feedback can target these error sources directly. Entity disambiguation can revise cell links, online correction can revise inserted associations, source verification can check extracted relations, and matching diagnosis can inspect entity alignment and path choices. Chapter 8 develops these directions.

7.6 Limitations and Open Questions

KGLink depends on the coverage and vocabulary of the external graph. Its no-linkage analysis tests tables without retrieved KG evidence, while broader use also depends on graph language, update state, and alignment with the target labels.

GORAG can retain errors through prediction-derived index updates. Longer streams and different patterns of label evolution remain untested.

AGRAG and MGRAG depend on extracted relations, and MGRAG additionally depends on caption-based image ranking. Missing or inaccurate facts can prevent the generator from receiving relevant evidence. Model-assessed QA metrics also depend on the evaluator and its assessment rules.

AGRAG's 2-approximation statement applies to its initial terminal-connecting tree. MGRAG's threshold condition describes accepted path similarities. Neither characterizes the answer quality of the full retrieval and generation pipeline.

CHAPTER 8

CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

In this thesis, we developed four methods that use explicit knowledge structures to select and process evidence for language models. The methods address missing table context, evolving classification labels, connected textual evidence, and semantic correspondence across multimodal sources.

We proposed KGLink to enrich column representations with graph descriptions and learn their correspondence to annotation labels. Cross-column filtering selects relevant descriptions, and contextual type-representation training helps the encoder use them. The model also retains a table-encoding pathway for columns without KG evidence.

We proposed GORAG to maintain a keyword–label graph for dynamic few-shot classification. Connected retrieval produces a candidate set for each query, while online insertion expands the index after prediction. The ablations show the benefit of these updates, with dataset-dependent effects as support and label sets grow.

We proposed AGRAG to select connected textual evidence using node influence and semantic edge costs. Expanding the initial tree supplies additional answer components, while hybrid passages retain context beyond the selected graph. The ablations show how these choices affect answer coverage and faithfulness.

We proposed MGRAG to construct query-specific multimodal graphs and match query relations to semantic paths. Graph-aware recomputation extends the use of the retrieved structure to context processing. The experiments show improved answer quality, with runtime effects that depend on the retrieval pipeline.

The accompanying hyper-relational knowledge-graph study examined how structural representations preserve fact information. Its qualifier-capture and graph-reception perspectives help explain why the form of a representation matters. Across the thesis, the common design question is which relationships must remain explicit for the downstream task.

8.2 Future Directions

8.2.1 Feedback for structure maintenance

Future work can use feedback to maintain evidence structures. A retrieved relation may prove irrelevant, its source passage may be insufficient, or a prediction inserted into an index may later be corrected. These observations could guide which associations are retained, revised, or removed.

Feedback timing determines which updates are possible. An online correction becomes available after the original prediction, and an answer assessment can guide only subsequent retrieval. Recording this timing and the feedback source would allow correction and retention policies to be evaluated under explicit supervision conditions.

8.2.2 Multimodal structure and provenance

Multimodal graph construction could retain a link from each extracted fact to its supporting text span or image region. Caption-based ranking and query-aware extraction both select visual information, and source links would make their omissions easier to identify.

Source links would help distinguish missing visual evidence from extraction, relation-representation, and path-search errors. They would also support checking whether an accepted path is grounded in the original source at the granularity required by the answer.

8.2.3 Joint retrieval and context computation

Retrieval and context computation can be optimized jointly. Retrieved evidence determines input length, modality, and relationships; cache availability and recomputation policy determine processing cost. MGRAG uses graph features for one such decision.

Further work could compare evidence structures under a fixed serving budget, measuring answer quality alongside construction, matching, cache reuse, and generation costs. Keeping the query workload and hardware fixed would show which structures provide useful information at an acceptable processing cost.

REFERENCES

[1] Y. Wang *et al.*, "KGLink: A column type annotation method that combines knowledge graph and pre-trained language model," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, IEEE, 2024, pp. 1023–1035. DOI: 10.1109/ICDE60146.2024.00083 (cit. on pp. 1, 7, 9, 17–21, 24, 30, 31).

[2] Y. Wang *et al.*, "GORAG: Graph-based online retrieval augmented generation for dynamic few-shot social media text classification," in *Proceedings of the ACM Web Conference 2026*, Association for Computing Machinery, 2026, pp. 9148–9159. DOI: 10.1145/3774904.3793011 (cit. on pp. 1, 7, 11, 12, 32–34, 43–45).

[3] Y. Wang *et al.*, "AGRAG: Advanced graph-based retrieval-augmented generation for LLMs," in *2026 IEEE 42nd International Conference on Data Engineering (ICDE)*, IEEE, 2026. DOI: 10.1109/ICDE65706.2026.00110 (cit. on pp. 2, 7, 12, 47, 48, 53, 57–59, 65).

[4] Y. Wang *et al.*, "MGRAG: Semantic subgraph matching and graph-aware caching for multimodal retrieval-augmented generation," *Proceedings of the VLDB Endowment*, vol. 19, no. 9, pp. 2480–2493, 2026. DOI: 10.14778/3819518.3819565 (cit. on pp. 2, 7, 12, 13, 67–69, 71, 76, 78, 80, 81, 84–88).

[5] Y. Wang *et al.*, "Understanding the embedding models on hyper-relational knowledge graph," in *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, 2025, pp. 3177–3187. DOI: 10.1145/3746252.3761370 (cit. on pp. 7, 13, 14).

[6] D. Vrandečić *et al.*, "Wikidata," *Communications of the ACM*, vol. 57, no. 10, pp. 78–85, 2014. DOI: 10.1145/2629489 (cit. on p. 8).

[7] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017 (cit. on pp. 8, 13).

[8] J. Devlin *et al.*, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT 2019*, 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423 (cit. on pp. 9, 23).

[9] M. Hulsebos *et al.*, "Sherlock: A deep learning approach to semantic data type detection," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 1500–1508 (cit. on p. 9).

[10] D. Zhang *et al.*, "Sato: Contextual semantic type detection in tables," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 1835–1848, 2020 (cit. on pp. 9, 26).

[11] Y. Suhara *et al.*, "Annotating columns with pre-trained language models," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 1493–1503 (cit. on pp. 9, 23, 27).

[12] J. Chen *et al.*, "Learning semantic annotations for tabular data," in *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, 2019, pp. 2088–2094 (cit. on pp. 9, 27).

[13] Y. Sun *et al.*, "Reca: Related tables enhanced column semantic type annotation framework," *Proceedings of the VLDB Endowment*, vol. 16, no. 6, pp. 1319–1331, 2023. DOI: 10.14778/3583140.3583149 (cit. on pp. 9, 27).

[14] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in neural information processing systems*, vol. 33, pp. 9459–9474, 2020 (cit. on p. 10).

[15] S. Robertson *et al.*, "The probabilistic relevance framework: Bm25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009. DOI: 10.1561/1500000019 (cit. on pp. 10, 19, 54).

[16] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550 (cit. on pp. 10, 12).

[17] D. Edge *et al.*, "From local to global: A graph rag approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024 (cit. on pp. 11, 12, 39, 44, 56).

[18] B. Jiménez Gutiérrez *et al.*, "HippoRAG: Neurobiologically inspired long-term memory for large language models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 59 532–59 569 (cit. on pp. 11, 12).

[19] Z. Guo *et al.*, "Lightrag: Simple and fast retrieval-augmented generation," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, Suzhou, China, 2025, pp. 10 746–10 761. DOI: 10.18653/v1/2025.findings-emnlp.568 (cit. on pp. 11, 12, 39, 44, 56).

[20] A. Talmor *et al.*, "Multimodalqa: Complex question answering over text, tables and images," in *International Conference on Learning Representations*, 2021 (cit. on pp. 12, 78).

[21] Q. Yu *et al.*, "Mramg-bench: A comprehensive benchmark for advancing multimodal retrieval-augmented multimodal generation," in *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2025, pp. 3616–3626 (cit. on pp. 13, 78).

[22] J. Yao *et al.*, "Cacheblend: Fast large language model serving for rag with cached knowledge fusion," in *Proceedings of the Twentieth European Conference on Computer Systems*, 2025, pp. 94–109. DOI: 10.1145/3689031.3696098 (cit. on pp. 13, 77).

[23] Y. Liu *et al.*, "Cachegen: Kv cache compression and streaming for fast large language model serving," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 38–56. DOI: 10.1145/3651890.3672274 (cit. on p. 13).

[24] M. Galkin *et al.*, "Message passing for hyper-relational knowledge graphs," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 7346–7359. DOI: 10.18653/v1/2020.emnlp-main.596 (cit. on p. 14).

[25] S. Vashishth *et al.*, "Composition-based multi-relational graph convolutional networks," in *International Conference on Learning Representations*, 2020 (cit. on p. 15).

[26] Z. Zhu *et al.*, "Neural bellman-ford networks: A general graph neural network framework for link prediction," *Advances in Neural Information Processing Systems*, vol. 34, pp. 29 476–29 490, 2021 (cit. on p. 15).

[27] Z. Hu *et al.*, "Hyperformer: Enhancing entity and relation interaction for hyper-relational knowledge graph completion," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, 2023, pp. 803–812. DOI: 10.1145/3583780.3614922 (cit. on p. 15).

[28] Q. Wang *et al.*, "Learning to extract attribute value from product via question answering: A multi-task approach," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 47–55 (cit. on p. 24).

[29] A. Kendall *et al.*, "Multi-task learning using uncertainty to weigh losses for scene geometry and semantics," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 7482–7491 (cit. on p. 25).

[30] O. Hassanzadeh *et al.*, *SemTab 2019: Semantic Web Challenge on Tabular Data to Knowledge Graph Matching Data Sets*, version 2019, Zenodo, 2019. DOI: 10.5281/zenodo.3518539 (cit. on p. 26).

[31] K. Hu *et al.*, "Viznet: Towards a large-scale visualization learning and benchmarking repository," in *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, 2019, pp. 1–12 (cit. on p. 26).

[32] P. Nguyen *et al.*, "Semtab 2021: Tabular data annotation with mtab tool," in *SemTab@ISWC 2021*, ser. CEUR Workshop Proceedings, vol. 3103, CEUR-WS.org, 2021, pp. 92–101 (cit. on p. 27).

[33] P. Yin *et al.*, "Tabert: Pretraining for joint understanding of textual and tabular data," *arXiv preprint arXiv:2005.08314*, 2020 (cit. on p. 27).

[34] R. Wang *et al.*, "Sudowoodo: Contrastive self-supervised learning for multi-purpose data integration and preparation," *arXiv preprint arXiv:2207.04122*, 2022 (cit. on p. 27).

[35] P. He *et al.*, "Deberta: Decoding-enhanced bert with disentangled attention," *arXiv preprint arXiv:2006.03654*, 2020 (cit. on p. 28).

[36] C. Xia *et al.*, "Incremental few-shot text classification with multi-round new classes: Formulation, dataset and system," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021*, K. Toutanova *et al.*, Eds., Online: Association for Computational Linguistics, 2021, pp. 1351–1360. DOI: 10.18653/V1/2021.NAACL-MAIN.106 (cit. on pp. 32, 38, 39, 55).

[37] "Tf–idf," in *Encyclopedia of Machine Learning*, C. Sammut *et al.*, Eds. Boston, MA: Springer US, 2010, pp. 986–987. DOI: 10.1007/978-0-387-30164-8_832 (cit. on pp. 34, 49).

[38] K. Mehlhorn, "A faster approximation algorithm for the steiner problem in graphs," *Information Processing Letters*, vol. 27, no. 3, pp. 125–128, 1988. DOI: 10.1016/0020-0190(88)90066-X (cit. on pp. 36, 55).

[39] B. Vidgen *et al.*, "Introducing CAD: The contextual abuse dataset," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, K. Toutanova *et al.*, Eds., Online: Association for Computational Linguistics, 2021, pp. 2289–2303. DOI: 10.18653/v1/2021.naacl-main.182 (cit. on p. 38).

[40] K. Kowsari *et al.*, "Hdltex: Hierarchical deep learning for text classification," in *2017 16th IEEE international conference on machine learning and applications (ICMLA)*, IEEE, 2017, pp. 364–371 (cit. on pp. 38, 55).

[41] H. Jiang *et al.*, "Longllmlingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku *et al.*, Eds., Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 1658–1677. DOI: 10.18653/v1/2024.acl-long.91 (cit. on pp. 39, 44, 56).

[42] B. J. Gutiérrez *et al.*, "From RAG to memory: Non-parametric continual learning for large language models," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, PMLR, 2025, pp. 21 497–21 515 (cit. on pp. 39, 56).

[43] A. Hurst *et al.*, "Gpt-4o system card," *arXiv preprint arXiv:2410.21276*, 2024 (cit. on p. 44).

[44] T. H. Haveliwala, "Topic-sensitive pagerank," in *Proceedings of the 11th international conference on World Wide Web*, 2002, pp. 517–526 (cit. on p. 51).

[45] Z. Xiang *et al.*, "When to use graphs in rag: A comprehensive analysis for graph retrieval-augmented generation," *arXiv preprint arXiv:2506.05690*, 2025 (cit. on p. 55).

[46] Qwen *et al.*, *Qwen2.5 technical report*, 2025. [Online]. Available: https://arxiv.org/abs/2412.15115 (cit. on p. 55).

[47] W. Kwon *et al.*, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626 (cit. on pp. 55, 77).

[48] G. Izacard *et al.*, "Unsupervised dense information retrieval with contrastive learning," *Transactions on Machine Learning Research*, 2022 (cit. on pp. 55, 78).

[49] P. Sarthi *et al.*, "Raptor: Recursive abstractive processing for tree-organized retrieval," *arXiv preprint arXiv:2401.18059*, 2024 (cit. on p. 56).

[50] N. Zhang *et al.*, "Sirerag: Indexing similar and related information for multihop reasoning," in *The Thirteenth International Conference on Learning Representations*, 2025 (cit. on p. 56).

[51] X. He *et al.*, "G-retriever: Retrieval-augmented generation for textual graph understanding and question answering," *Advances in Neural Information Processing Systems*, vol. 37, pp. 132 876–132 907, 2024 (cit. on p. 56).

[52] L. Liu *et al.*, "Mixrag: Mixture-of-experts retrieval-augmented generation for textual graph understanding and question answering," *arXiv preprint arXiv:2509.21391*, 2025 (cit. on p. 56).

[53] circlemind-ai. "Fast-graphrag." (2025), [Online]. Available: `https://github.com/circlemind-ai/fast-graphrag` (cit. on p. 56).

[54] C.-Y. Lin, "Rouge: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, Barcelona, Spain: Association for Computational Linguistics, 2004, pp. 74–81 (cit. on pp. 56, 79).

[55] VibrantLabs, *Ragas: Supercharge your llm application evaluations*, `https://github.com/vibrantlabsai/ragas`, 2024 (cit. on p. 56).

[56] S. Es *et al.*, "RAGAs: Automated evaluation of retrieval augmented generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024, pp. 150–158. DOI: `10.18653/v1/2024.eacl-demo.16` (cit. on pp. 56, 79).

[57] S. Xiao *et al.*, *C-pack: Packaged resources to advance general chinese embedding*, 2023 (cit. on pp. 69, 81).

[58] L. P. Cordella *et al.*, "A (sub) graph isomorphism algorithm for matching large graphs," *IEEE transactions on pattern analysis and machine intelligence*, vol. 26, no. 10, pp. 1367–1372, 2004 (cit. on pp. 73, 81).

[59] Y. Wang *et al.*, *MGRAG: Semantic subgraph matching and graph-aware caching for multimodal retrieval-augmented generation [technical report]*. [Online]. Available: `https://github.com/Wyb0627/MGRAG/blob/main/tech_report.pdf` (cit. on p. 75).

[60] S. Bai *et al.*, "Qwen2.5-VL technical report," *arXiv preprint arXiv:2502.13923*, 2025 (cit. on p. 78).

[61] M. Douze *et al.*, "The faiss library," 2024 (cit. on p. 79).

[62] "nano-graphrag: A simple, easy-to-hack GraphRAG implementation." (), [Online]. Available: `https://github.com/gusye1234/nano-graphrag` (cit. on p. 79).

[63] K. Papineni *et al.*, "Bleu: A method for automatic evaluation of machine translation," in *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, 2002, pp. 311–318 (cit. on p. 79).

[64] G. Angeli *et al.*, "Leveraging linguistic structure for open domain information extraction," in *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 2015, pp. 344–354 (cit. on p. 81).

[65] M. Houbraken *et al.*, "The index-based subgraph matching algorithm with general symmetries (ismags): Exploiting symmetry for faster subgraph enumeration," *PloS one*, vol. 9, no. 5, e97896, 2014 (cit. on p. 81).

[66] M. Leordeanu *et al.*, "A spectral technique for correspondence problems using pairwise constraints," in *Tenth IEEE International Conference on Computer Vision (ICCV'05) Volume 1*, IEEE, vol. 2, 2005, pp. 1482–1489 (cit. on p. 81).

[67] S. Fankhauser *et al.*, "Speeding up graph edit distance computation through fast bipartite matching," in *International Workshop on Graph-Based Representations in Pattern Recognition*, Springer, 2011, pp. 102–111 (cit. on p. 81).

[68] M. Cho *et al.*, "Reweighted random walks for graph matching," in *European conference on Computer vision*, Springer, 2010, pp. 492–505 (cit. on p. 81).

[69] M. Leordeanu *et al.*, "An integer projected fixed point method for graph matching and map inference," *Advances in neural information processing systems*, vol. 22, 2009 (cit. on p. 81).

[70] J. Tang *et al.*, "Robust attributed graph alignment via joint structure learning and optimal transport," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, IEEE, 2023, pp. 1638–1651 (cit. on p. 81).

[71] J. Tang *et al.*, "Fused gromov-wasserstein alignment for graph edit distance computation and beyond," *Proceedings of the VLDB Endowment*, vol. 18, no. 10, pp. 3641–3654, 2025 (cit. on p. 81).

[72] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," in *International conference on machine learning*, PmLR, 2021, pp. 8748–8763 (cit. on p. 87).